

SOURCE CODE AND OUTPUTS

Learning Community Analytics using NLP Topics and Knowledge Diffusion Networks

Capstone project: program listings with their outputs

Prateek Chauhan

Guide: _____

Department / Institute: _____

September 2026

Contents

Contents	2
1. Project Structure and How to Run	3
2. Configuration Files	4
3. Data Collection (collect_data.py)	6
4. Database Schema and Loading (schema.sql, load_to_postgres.py)	10
5. Text Preprocessing (preprocess.py)	15
6. Topic Modeling (topic_model.py)	18
7. Knowledge Network (network.py)	25
8. Knowledge Diffusion (diffusion.py)	33
9. Answer Prediction (predict.py)	41
10. Expert Finder (recommender.py)	45
11. Figures and the Pipeline Runner (figures.py, run_pipeline.py)	48
12. SQL Analysis Queries (queries.sql)	55
13. Interactive Dashboard (streamlit_app.py)	60

1. Project Structure and How to Run

The project is a pipeline of Python programs. Each program reads the output of the previous one, so they are listed below in the order they run. All programs are in the `src/` folder, the dashboard is in `app/`, and the SQL files are in `sql/`.

```

learning-community-analytics/
├── app/streamlit_app.py      Interactive dashboard (7 pages)
├── src/
│   ├── config.py           Shared paths and settings
│   ├── db.py               PostgreSQL connection (.env)
│   ├── collect_data.py     Step 1 Download data from the Stack Exchange API
│   ├── load_to_postgres.py Step 2 Load data + results into PostgreSQL
│   ├── data_loader.py      Read tables from PostgreSQL or CSV
│   ├── preprocess.py       Step 3 Text cleaning and features
│   ├── topic_model.py      Step 4 LDA and BERTopic topic modeling
│   ├── network.py          Step 5 Knowledge network analysis
│   ├── diffusion.py        Step 6 Knowledge diffusion analysis
│   ├── predict.py          Step 7 Answer prediction models
│   ├── recommender.py      Step 8 Expert finder
│   ├── figures.py          Step 9 Report figures
│   └── run_pipeline.py     Runs steps 3-9 in order
├── sql/schema.sql          Table definitions
├── sql/queries.sql         11 analysis queries (pgAdmin)
├── data/raw/cs50/          Downloaded CSV files
├── data/processed/         Cleaned data, embeddings, features
├── outputs/ , outputs/figures/ Result tables and figures
└── docs/                  Report, presentation, black book

```

Commands

```

pip install -r requirements.txt          # install libraries
python src/collect_data.py --site cs50 --max-questions 4000 # step 1 (optional, data included)
python src/run_pipeline.py              # steps 3-9 (about 10 minutes)
python src/load_to_postgres.py          # step 2: load everything into PostgreSQL
streamlit run app/streamlit_app.py      # start the dashboard

```

Software used

Python 3.14 (tested; 3.12 recommended on the laptop), PostgreSQL with pgAdmin, pandas, NumPy, scikit-learn, NLTK, BeautifulSoup, sentence-transformers (all-MiniLM-L6-v2), BERTopic, UMAP, HDBSCAN, NetworkX, SciPy, XGBoost, Matplotlib, Plotly, PyVis, Streamlit, SQLAlchemy and psycopg2.

2. Configuration Files

`config.py` holds the folder paths and settings shared by all programs: the random seed that makes results reproducible, the sentence-embedding model, and the 80/20 time split used for evaluation. `db.py` builds the PostgreSQL connection from the `.env` file, so the password is never written in the code. These two files only define settings and print nothing.

Source code: `src/config.py` (21 lines)

```
1  """Shared paths and settings for the whole pipeline."""
2
3  from pathlib import Path
4
5  PROJECT_ROOT = Path(__file__).resolve().parent.parent
6  SITE = "cs50"
7
8  RAW_DIR = PROJECT_ROOT / "data" / "raw" / SITE
9  PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
10 OUTPUT_DIR = PROJECT_ROOT / "outputs"
11 FIGURE_DIR = OUTPUT_DIR / "figures"
12
13 RANDOM_SEED = 42
14 EMBEDDING_MODEL = "all-MiniLM-L6-v2"
15
16 # Share of the timeline (oldest first) used for training in the prediction
17 # and recommender evaluations; the newest part is held out as the test set.
18 TRAIN_FRACTION = 0.8
19
20 for _d in (PROCESSED_DIR, OUTPUT_DIR, FIGURE_DIR):
21     _d.mkdir(parents=True, exist_ok=True)
```

Source code: `src/db.py` (23 lines)

```
1  """PostgreSQL connection, configured from the .env file in the project root."""
2
3  import os
4  from pathlib import Path
5
6  from dotenv import load_dotenv
7  from sqlalchemy import create_engine
8  from sqlalchemy.engine import URL
9
10 PROJECT_ROOT = Path(__file__).resolve().parent.parent
11 load_dotenv(PROJECT_ROOT / ".env")
12
13
14 def get_engine():
15     url = URL.create(
16         "postgresql+psycopg2",
17         username=os.getenv("DB_USER", "postgres"),
18         password=os.getenv("DB_PASSWORD"),
19         host=os.getenv("DB_HOST", "localhost"),
20         port=int(os.getenv("DB_PORT", "5432")),
21         database=os.getenv("DB_NAME", "learning_community"),
22     )
23     return create_engine(url)
```

Source code: `.env.example` (7 lines)

```
1  # Copy this file to .env and fill in your PostgreSQL details.
2  # .env is ignored by git, so your password never gets uploaded.
3  DB_HOST=localhost
4  DB_PORT=5432
5  DB_NAME=learning_community
6  DB_USER=postgres
7  DB_PASSWORD=your_postgres_password_here
```

Source code: `requirements.txt` (30 lines)

```
1  # Tested with Python 3.12-3.14. Install with: pip install -r requirements.txt
```

```
2
3 # data + database
4 requests>=2.31
5 pandas>=2.2,<3
6 numpy>=1.26
7 pyarrow>=15
8 SQLAlchemy>=2.0
9 psycpg2-binary>=2.9
10 python-dotenv>=1.0
11
12 # NLP + topic modeling
13 beautifulsoup4>=4.12
14 nltk>=3.8
15 scikit-learn>=1.4
16 scipy>=1.11
17 sentence-transformers>=2.7
18 bertopic>=0.16
19 umap-learn>=0.5.5
20 hdbscan>=0.8.33
21
22 # networks, diffusion, prediction
23 networkx>=3.2
24 xgboost>=2.0
25
26 # figures + dashboard
27 matplotlib>=3.8
28 plotly>=5.20
29 streamlit>=1.40
30 pyvis>=0.3.2
```

3. Data Collection (collect_data.py)

Aim: download the questions, answers and comments of the CS50 Stack Exchange site through the public Stack Exchange API and save them as CSV files. Without an API key the API serves only 25 pages per query, so the program walks backwards through time in date windows. It also builds the **interactions** table (who answered or commented on whose post), which is the edge list of the knowledge network.

Run: `python src/collect_data.py --site cs50 --max-questions 4000`

Source code: `src/collect_data.py` (205 lines)

```

1  """
2  Collect a learning-community dataset from the Stack Exchange API.
3
4  Downloads questions, answers, and comments from one Stack Exchange site and
5  saves them as CSV files that the rest of the project (topic modeling,
6  network analysis, diffusion) builds on.
7
8  Usage:
9      python src/collect_data.py --site datascience --max-questions 2000
10
11  Without an API key you get 300 requests/day, which is enough for ~5,000
12  questions. Register a free key at https://stackapps.com/apps/oauth/register
13  to get 10,000 requests/day, then pass it with --key.
14  """
15
16  import argparse
17  import html
18  import time
19  from pathlib import Path
20
21  import pandas as pd
22  import requests
23
24  API = "https://api.stackexchange.com/2.3"
25  # Built-in filter that includes the post body text.
26  FILTER = "withbody"
27  MAX_PAGE_WITHOUT_KEY = 25
28  OUT_DIR = Path(__file__).resolve().parent.parent / "data" / "raw"
29
30  class StackExchangeClient:
31      def __init__(self, site, key=None):
32          self.site = site
33          self.key = key
34          self.quota_remaining = None
35
36      def get(self, endpoint, **params):
37          """Yield every item from a paginated endpoint."""
38          params.update(site=self.site, filter=FILTER, pagesize=100, page=1)
39          if self.key:
40              params["key"] = self.key
41          # Without a key the API refuses pages above 25.
42          max_page = None if self.key else MAX_PAGE_WITHOUT_KEY
43          while max_page is None or params["page"] <= max_page:
44              resp = requests.get(f"{API}/{endpoint}", params=params, timeout=30)
45              data = resp.json()
46              if "error_id" in data:
47                  raise RuntimeError(f"API error {data['error_id']}: {data.get('error_message')}")
48              self.quota_remaining = data.get("quota_remaining")
49              yield from data.get("items", [])
50              # The API asks clients to wait when it sends a backoff value.
51              if data.get("backoff"):
52                  time.sleep(data["backoff"])
53              if not data.get("has_more"):
54                  break
55              params["page"] += 1
56              time.sleep(0.1)
57
58  def owner_fields(item):
59      owner = item.get("owner", {})
60      return {
61          "user_id": owner.get("user_id"),
62          "user_name": html.unescape(owner.get("display_name", "")),
63          "user_reputation": owner.get("reputation"),
64

```

```

65     }
66
67
68 def fetch_questions(client, max_questions):
69     """
70     Newest questions first. The API only serves 25 pages per query without a
71     key, so after each 25-page window we continue from the oldest date seen.
72     """
73     rows = {}
74     todate = None
75     while len(rows) < max_questions:
76         params = {"order": "desc", "sort": "creation"}
77         if todate:
78             params["todate"] = todate
79         before = len(rows)
80         for q in client.get("questions", **params):
81             rows[q["question_id"]] = {
82                 "post_id": q["question_id"],
83                 "title": html.unescape(q.get("title", "")),
84                 "body": q.get("body", ""),
85                 "tags": "|".join(q.get("tags", [])),
86                 "score": q.get("score"),
87                 "view_count": q.get("view_count"),
88                 "answer_count": q.get("answer_count"),
89                 "is_answered": q.get("is_answered"),
90                 "accepted_answer_id": q.get("accepted_answer_id"),
91                 "created_at": q.get("creation_date"),
92                 **owner_fields(q),
93             }
94         if len(rows) >= max_questions:
95             break
96         if len(rows) % 500 == 0:
97             print(f" questions: {len(rows)} (quota left: {client.quota_remaining})")
98         if len(rows) == before:
99             break # no older questions left
100         todate = min(r["created_at"] for r in rows.values())
101     return pd.DataFrame(list(rows.values()))
102
103
104 def fetch_by_ids(client, endpoint_template, ids, parse):
105     """Call an '{ids}' endpoint in batches of 100 ids (the API maximum)."""
106     rows = []
107     ids = [str(i) for i in ids]
108     for start in range(0, len(ids), 100):
109         batch = ";".join(ids[start:start + 100])
110         rows.extend(parse(item) for item in client.get(endpoint_template.format(ids=batch)))
111         print(f" {endpoint_template.split('/')[1]}: {min(start + 100, len(ids))}/{len(ids)} parents
112
113             f"(quota left: {client.quota_remaining})")
114     return pd.DataFrame(rows)
115
116 def parse_answer(a):
117     return {
118         "post_id": a["answer_id"],
119         "question_id": a["question_id"],
120         "body": a.get("body", ""),
121         "score": a.get("score"),
122         "is_accepted": a.get("is_accepted"),
123         "created_at": a.get("creation_date"),
124         **owner_fields(a),
125     }
126
127
128 def parse_comment(c):
129     reply_to = c.get("reply_to_user", {})
130     return {
131         "comment_id": c["comment_id"],
132         "post_id": c["post_id"],
133         "body": c.get("body", ""),
134         "score": c.get("score"),
135         "reply_to_user_id": reply_to.get("user_id"),
136         "created_at": c.get("creation_date"),
137         **owner_fields(c),
138     }
139
140
141 def build_interactions(questions, answers, comments):
142     """
143     One row per 'source user helped/replied to target user' event.
144     This is the edge list for the knowledge network.
145     """
146     post_owner = pd.concat([

```

```

147     questions[["post_id", "user_id"]],
148     answers[["post_id", "user_id"]],
149     ]).dropna().drop_duplicates("post_id").set_index("post_id")["user_id"]
150     q_owner = questions.set_index("post_id")["user_id"]
151
152     edges = []
153     for a in answers.itertuples():
154         edges.append((a.user_id, q_owner.get(a.question_id), "answer", a.post_id, a.question_id,
155 a.created_at))
156     for c in comments.itertuples():
157         target = c.reply_to_user_id if pd.notna(c.reply_to_user_id) else post_owner.get(c.post_id)
158         edges.append((c.user_id, target, "comment", c.comment_id, c.post_id, c.created_at))
159
160     df = pd.DataFrame(edges, columns=["source_user", "target_user", "type", "id", "parent_post_id",
161 "created_at"])
162     df = df.dropna(subset=["source_user", "target_user"])
163     return df[df.source_user != df.target_user]
164
165 def main():
166     parser = argparse.ArgumentParser(description=__doc__,
167 formatter_class=argparse.RawDescriptionHelpFormatter)
168     parser.add_argument("--site", default="datascience", help="Stack Exchange site, e.g. datascience,
169 ai, stats, cs50")
170     parser.add_argument("--max-questions", type=int, default=2000)
171     parser.add_argument("--key", default=None, help="Optional Stack Apps API key for a higher quota")
172     args = parser.parse_args()
173
174     out = OUT_DIR / args.site
175     out.mkdir(parents=True, exist_ok=True)
176     client = StackExchangeClient(args.site, args.key)
177
178     print(f"Fetching up to {args.max_questions} questions from {args.site}.stackexchange.com ...")
179     questions = fetch_questions(client, args.max_questions)
180     questions.to_csv(out / "questions.csv", index=False)
181     print(f"Saved {len(questions)} questions")
182
183     print("Fetching answers ...")
184     answers = fetch_by_ids(client, "questions/{ids}/answers", questions.post_id, parse_answer)
185     answers.to_csv(out / "answers.csv", index=False)
186     print(f"Saved {len(answers)} answers")
187
188     print("Fetching comments ...")
189     all_post_ids = pd.concat([questions.post_id, answers.post_id])
190     comments = fetch_by_ids(client, "posts/{ids}/comments", all_post_ids, parse_comment)
191     comments.to_csv(out / "comments.csv", index=False)
192     print(f"Saved {len(comments)} comments")
193
194     interactions = build_interactions(questions, answers, comments)
195     interactions.to_csv(out / "interactions.csv", index=False)
196
197     users = pd.concat([questions, answers, comments])[["user_id", "user_name", "user_reputation"]]
198     users = users.dropna(subset=["user_id"]).sort_values("user_reputation").drop_duplicates("user_id", keep="last")
199     users.to_csv(out / "users.csv", index=False)
200
201     print(f"\nDone. Files in {out}")
202     print(f"  questions={len(questions)} answers={len(answers)} comments={len(comments)} "
203           f"users={len(users)} interactions={len(interactions)}")
204     print(f"  API quota remaining today: {client.quota_remaining}")
205
206 if __name__ == "__main__":
207     main()

```

Output (final lines of the run that produced the project's dataset)

```

Fetching up to 4000 questions from cs50.stackexchange.com ...
...
Fetching answers ...
Saved 4253 answers
Fetching comments ...
Saved 7168 comments

Done. Files in C:\Users\pc\python codes\Capstone project\data\raw\cs50
  questions=4000 answers=4253 comments=7168 users=3003 interactions=10477
  API quota remaining today: 15

```

Table 1. *Sample rows of data/raw/cs50/questions.csv*

post_id	title	tags	score	answer_count	created_at
45579	How to Choose the Right Creative 2D Animation Service	2danimation creativeservices	0	0	2026-09-18
45577	adieu.py check50 fails on first "input of EOF halts program" check despite correct output	check50 python cs50p	0	0	2026-09-13
45573	Revounts Australia	digital	0	0	2026-08-19
45572	Why use a for loop instead of string multiplication?	cs50p	0	0	2026-08-18
45571	Why is submit50 claiming I have no links, when I have at least 4 links in all of my html pages?	pset8 html homepage	0	0	2026-08-13
45563	CS50x 2026 gradebook not showing my previous submissions	cs50x	1	0	2026-07-24

Table 2. *Sample rows of data/raw/cs50/interactions.csv (knowledge-network edges)*

source_user	target_user	type	id	parent_post_id	created_at
48554	43820	answer	45574	45343	2026-08-25
1090	47412	answer	45528	45527	2026-04-03
4847	47311	answer	45525	45524	2026-03-07
4847	47251	answer	45520	45517	2026-02-19
47251	45886	answer	45519	45455	2026-02-19
38661	46831	answer	45500	45492	2025-12-17

4. Database Schema and Loading (schema.sql, load_to_postgres.py)

Aim: store the raw data in PostgreSQL with primary and foreign keys, then load the analysis results (topics, user metrics, model evaluations) as extra tables. The program recreates the tables on every run, so it is safe to run again. The data loader lets every analysis program read from PostgreSQL when it is connected, and from the CSV files otherwise.

Run: `python src/load_to_postgres.py` (needs the `.env` file with the PostgreSQL password)

Source code: `sql/schema.sql` (63 lines)

```

1  -- Schema for the learning-community database.
2  -- Running this file recreates the tables from scratch (existing data is removed).
3
4  DROP TABLE IF EXISTS interactions, comments, answers, questions, users CASCADE;
5
6  CREATE TABLE users (
7      user_id          BIGINT PRIMARY KEY,
8      user_name        TEXT,
9      user_reputation  INTEGER
10 );
11
12 CREATE TABLE questions (
13     post_id          BIGINT PRIMARY KEY,
14     title            TEXT,
15     body             TEXT,
16     tags             TEXT,          -- pipe-separated, e.g. 'pset1|c|cs50'
17     score            INTEGER,
18     view_count       INTEGER,
19     answer_count     INTEGER,
20     is_answered      BOOLEAN,
21     accepted_answer_id BIGINT,
22     created_at       TIMESTAMP,
23     user_id          BIGINT REFERENCES users(user_id)
24 );
25
26 CREATE TABLE answers (
27     post_id          BIGINT PRIMARY KEY,
28     question_id     BIGINT NOT NULL REFERENCES questions(post_id),
29     body            TEXT,
30     score           INTEGER,
31     is_accepted     BOOLEAN,
32     created_at      TIMESTAMP,
33     user_id         BIGINT REFERENCES users(user_id)
34 );
35
36 -- A comment can belong to a question or an answer, so post_id has no foreign key.
37 CREATE TABLE comments (
38     comment_id      BIGINT PRIMARY KEY,
39     post_id         BIGINT NOT NULL,
40     body            TEXT,
41     score           INTEGER,
42     reply_to_user_id BIGINT,
43     created_at      TIMESTAMP,
44     user_id         BIGINT REFERENCES users(user_id)
45 );
46
47 -- Edge list of the knowledge network: source_user helped / replied to target_user.
48 -- target_user may be someone with no posts in the sample, so no foreign key here.
49 CREATE TABLE interactions (
50     interaction_id   SERIAL PRIMARY KEY,
51     source_user      BIGINT NOT NULL,
52     target_user      BIGINT NOT NULL,
53     type            TEXT NOT NULL CHECK (type IN ('answer', 'comment')),
54     id              BIGINT NOT NULL,    -- answer or comment id
55     parent_post_id   BIGINT NOT NULL,
56     created_at      TIMESTAMP
57 );
58
59 CREATE INDEX idx_questions_created ON questions(created_at);
60 CREATE INDEX idx_answers_question ON answers(question_id);
61 CREATE INDEX idx_comments_post ON comments(post_id);
62 CREATE INDEX idx_interactions_src ON interactions(source_user);
63 CREATE INDEX idx_interactions_tgt ON interactions(target_user);

```

Source code: src/load_to_postgres.py (114 lines)

```

1  """
2  Load the raw CSV files (src/collect_data.py) and the analysis results (outputs/) into PostgreSQL.
3
4  Usage:
5      python src/load_to_postgres.py --site cs50
6
7  Needs a .env file with your database details (copy .env.example).
8  The tables are recreated each run, so it is safe to run again.
9
10 Raw tables:      users, questions, answers, comments, interactions  (sql/schema.sql)
11 Result tables:   topics, question_topics, user_metrics, topic_trends, help_edges, and the
12                  model evaluation tables – created from the files in outputs/.
13 """
14
15 import argparse
16 from pathlib import Path
17
18 import pandas as pd
19 from sqlalchemy import text
20
21 from db import PROJECT_ROOT, get_engine
22
23 SCHEMA_FILE = PROJECT_ROOT / "sql" / "schema.sql"
24 # Load order matters: users first, because other tables reference them.
25 TABLES = ["users", "questions", "answers", "comments", "interactions"]
26 ID_COLUMNS = ["user_id", "post_id", "question_id", "comment_id", "accepted_answer_id",
27               "reply_to_user_id", "source_user", "target_user", "id", "parent_post_id",
28               "user_reputation", "score", "view_count", "answer_count"]
29 PRIMARY_KEYS = {"users": "user_id", "questions": "post_id", "answers": "post_id", "comments":
30 "comment_id"}
31
32 OUTPUT_DIR = PROJECT_ROOT / "outputs"
33 PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
34 # table name -> (file, primary key or None, extra SQL run after loading)
35 RESULT_TABLES = {
36     "topics": (OUTPUT_DIR / "topics.csv", "topic", None),
37     "question_topics": (PROCESSED_DIR / "question_topics.parquet", "post_id",
38                         "ALTER TABLE question_topics ADD FOREIGN KEY (post_id) REFERENCES
39 questions(post_id)"),
40     "user_metrics": (OUTPUT_DIR / "user_metrics.csv", "user_id", None),
41     "help_edges": (OUTPUT_DIR / "help_edges.csv", None, None),
42     "topic_trends": (OUTPUT_DIR / "topic_trends.csv", None, None),
43     "network_over_time": (OUTPUT_DIR / "network_over_time.csv", "year", None),
44     "topic_model_comparison": (OUTPUT_DIR / "topic_model_comparison.csv", None, None),
45     "diffusion_exposure_test": (OUTPUT_DIR / "diffusion_exposure_test.csv", "topic", None),
46     "influence_maximization": (OUTPUT_DIR / "influence_maximization.csv", None, None),
47     "prediction_metrics": (OUTPUT_DIR / "prediction_metrics.csv", "model", None),
48     "recommender_eval": (OUTPUT_DIR / "recommender_eval.csv", "method", None),
49 }
50
51 def prepare_tables(data_dir: Path) -> dict[str, pd.DataFrame]:
52     """Read the CSVs and convert types so they match sql/schema.sql."""
53     tables = {}
54     for name in TABLES:
55         df = pd.read_csv(data_dir / f"{name}.csv")
56         if name != "users":
57             # User details live only in the users table.
58             df = df.drop(columns=["user_name", "user_reputation"], errors="ignore")
59             # IDs are read as floats when a column has blanks; make them whole numbers again.
60             for col in df.columns.intersection(ID_COLUMNS):
61                 df[col] = df[col].astype("Int64")
62             if "created_at" in df:
63                 df["created_at"] = pd.to_datetime(df["created_at"], unit="s")
64             if name in PRIMARY_KEYS:
65                 # The API can return the same post twice while paging.
66                 df = df.drop_duplicates(PRIMARY_KEYS[name])
67             tables[name] = df
68     return tables
69
70 def load_results(conn):
71     """Load analysis results when they exist (run src/run_pipeline.py to create them)."""
72     present = {name: spec for name, spec in RESULT_TABLES.items() if spec[0].exists()}
73     if not present:
74         print("No analysis results found in outputs/ (run python src/run_pipeline.py first).")
75         return
76     print("Loading analysis results ...")
77     for name, (path, key, extra) in present.items():
78         df = pd.read_parquet(path) if path.suffix == ".parquet" else pd.read_csv(path)
79         for col in df.columns.intersection(["first_at", "last_at", "first_seen", "last_seen"]):

```

```

80         df[col] = pd.to_datetime(df[col])
81         conn.execute(text(f"DROP TABLE IF EXISTS {name} CASCADE"))
82         df.to_sql(name, conn, index=False, chunksize=1000)
83         if key:
84             conn.execute(text(f"ALTER TABLE {name} ADD PRIMARY KEY ({key}))")
85         if extra:
86             conn.execute(text(extra))
87         print(f" {name}: {len(df)} rows")
88
89
90 def main():
91     parser = argparse.ArgumentParser(description=__doc__,
92 formatter_class=argparse.RawDescriptionHelpFormatter)
93     parser.add_argument("--site", default="cs50", help="Folder name under data/raw/")
94     args = parser.parse_args()
95     data_dir = PROJECT_ROOT / "data" / "raw" / args.site
96     if not data_dir.exists():
97         raise SystemExit(f"No data in {data_dir}. Run: python src/collect_data.py --site
{args.site}")
98
99     tables = prepare_tables(data_dir)
100    engine = get_engine()
101
102    with engine.begin() as conn:
103        print("Creating tables ...")
104        conn.execute(text(SCHEMA_FILE.read_text()))
105        for name in TABLES:
106            tables[name].to_sql(name, conn, if_exists="append", index=False, chunksize=1000)
107            print(f" {name}: {len(tables[name])} rows")
108        load_results(conn)
109
110    print("\nDone. Open pgAdmin -> learning_community -> Schemas -> public -> Tables to see the
data.")
111
112
113 if __name__ == "__main__":
114     main()

```

Source code: src/data_loader.py (82 lines)

```

1  """
2  Load the raw community tables from PostgreSQL when it is configured, and from
3  the CSV files otherwise, so the pipeline runs with or without a database.
4  """
5
6  import os
7  from functools import lru_cache
8
9  import pandas as pd
10
11  from config import PROJECT_ROOT, RAW_DIR
12
13  RAW_TABLES = ["users", "questions", "answers", "comments", "interactions"]
14
15
16  @lru_cache(maxsize=1)
17  def get_db_engine():
18      """Return a working SQLAlchemy engine, or None when no database is reachable."""
19      if not (PROJECT_ROOT / ".env").exists():
20          return None
21      try:
22          from sqlalchemy import text
23
24          from db import get_engine
25
26          engine = get_engine()
27          with engine.connect() as conn:
28              conn.execute(text("SELECT 1"))
29          return engine
30      except Exception as exc: # no driver, wrong password, server down, ...
31          if os.getenv("DEBUG_DB"):
32              print(f"[data_loader] PostgreSQL not available: {exc}")
33          return None
34
35
36  def data_source() -> str:
37      return "PostgreSQL" if get_db_engine() is not None else "CSV files"
38
39

```

```

40 def _table_in_db(engine, name):
41     from sqlalchemy import inspect
42     return inspect(engine).has_table(name)
43
44
45 def load_table(name: str) -> pd.DataFrame:
46     engine = get_db_engine()
47     if engine is not None and _table_in_db(engine, name):
48         df = pd.read_sql_table(name, engine)
49         df = df.drop(columns=["interaction_id"], errors="ignore")
50     else:
51         df = pd.read_csv(RAW_DIR / f"{name}.csv")
52         if "created_at" in df:
53             df["created_at"] = pd.to_datetime(df["created_at"], unit="s")
54     for col in ("user_id", "source_user", "target_user", "reply_to_user_id", "accepted_answer_id"):
55         if col in df:
56             df[col] = df[col].astype("Int64")
57     return df
58
59
60 def load_raw() -> dict[str, pd.DataFrame]:
61     return {name: load_table(name) for name in RAW_TABLES}
62
63
64 def help_interactions(raw: dict[str, pd.DataFrame]) -> pd.DataFrame:
65     """
66     Interactions that pass knowledge from a helper to a learner: every answer, plus
67     comments written by someone other than the person who asked the thread's question.
68     (An asker replying "thanks, still broken" to a helper is not the asker helping.)
69     """
70     q, a = raw["questions"], raw["answers"]
71     inter = raw["interactions"].dropna(subset=["source_user", "target_user"]).copy()
72     post_to_question = pd.concat([
73         pd.Series(q["post_id"].to_numpy(), index=q["post_id"]),
74         pd.Series(a["question_id"].to_numpy(), index=a["post_id"]),
75     ])
76     asker = q.set_index("post_id")["user_id"]
77     thread_asker = inter["parent_post_id"].map(post_to_question).map(asker)
78     keep = (inter["type"] == "answer") | (inter["source_user"] != thread_asker)
79     inter = inter[keep.fillna(True)]
80     inter["source_user"] = inter["source_user"].astype("int64")
81     inter["target_user"] = inter["target_user"].astype("int64")
82     return inter

```

Expected output (row counts from the dataset; replace with a screenshot of your own run)

```

Creating tables ...
users: 3003 rows
questions: 4000 rows
answers: 4253 rows
comments: 7168 rows
interactions: 10477 rows
Loading analysis results ...
topics: 19 rows
question_topics: 3960 rows
user_metrics: 2891 rows
...

Done. Open pgAdmin -> learning_community -> Schemas -> public -> Tables to see the data.

```

Paste screenshot: Command Prompt after running `python src/load_to_postgres.py`

Paste screenshot: pgAdmin → learning_community → Schemas → public → Tables (16 tables)

5. Text Preprocessing (preprocess.py)

Aim: clean the question text and compute features. HTML and code blocks are removed (the number of code lines is kept as a feature), the text is lemmatised and filtered with English and domain stopwords, and each question gets a VADER sentiment score, a confusion score (phrases such as “stuck” or “don’t understand”), an error flag and a check50-failure flag. The time to the first answer is also computed.

Run: `python src/preprocess.py` (or as step 1 of `run_pipeline.py`)

Source code: `src/preprocess.py` (152 lines)

```

1  """
2  Step 1: clean the question text and build per-question features.
3
4  Output: data/processed/questions_clean.parquet
5      text_clean    natural text (title + body without code blocks), for BERTopic and embeddings
6      tokens        lower-cased lemmas without stopwords, for LDA and TF-IDF
7      + structure, sentiment/confusion and outcome columns used by later steps
8  """
9
10 import html
11 import re
12 import warnings
13
14 import numpy as np
15 import pandas as pd
16 from bs4 import BeautifulSoup, MarkupResemblesLocatorWarning
17
18 from config import PROCESSED_DIR
19 from data_loader import load_raw
20
21 warnings.filterwarnings("ignore", category=MarkupResemblesLocatorWarning)
22
23 URL_RE = re.compile(r"https?://\S+|www\.\S+")
24 TOKEN_RE = re.compile(r"[a-z][a-z0-9_]+")
25 ERROR_RE = re.compile(r"error|segmentation fault|traceback|exception|undefined reference|"
26                       r"core dumped|valgrind|leak|warning:", re.I)
27 # check50 prints ":" for each failed test.
28 CHECK50_FAIL_RE = re.compile(r":\(")
29
30 # Words that appear in almost every question and carry no topic information.
31 DOMAIN_STOPWORDS = {
32     "cs50", "cs50x", "cs50p", "cs50w", "cs50ai", "pset", "problem", "set", "hi", "hello", "hey",
33     "thanks", "thank", "please", "help", "question", "trying", "try", "tried", "get", "got", "know",
34     "like", "want", "need", "would", "could", "work", "working", "works", "way", "anyone", "one",
35     "also", "use", "using", "used", "make", "see", "seem", "seems", "im", "ive", "dont", "doesnt",
36     "cant", "didnt", "isnt", "wont", "thats", "it", "code", "program", "run", "running", "fine",
37     "able", "still", "time", "thing", "something", "anything", "everything", "think", "idea",
38     "sure", "understand", "problem", "issue", "wrong", "right", "correct", "help", "appreciated",
39     "advance", "much", "lot", "really", "just", "even", "though", "however", "first", "second",
40     "new", "line", "lines", "every", "since", "keep", "keeps", "going", "look", "looks", "take",
41     "said", "say", "says", "well", "good", "okay", "ok", "yes", "no", "without", "back", "made",
42     "doe", "wa", "ha", "u", "le", "go", "come", "give", "gives", "given", "put", "let", "went",
43     "getting", "following", "find", "found", "result", "correctly", "expected", "check", "error",
44     "output", "print", "printing", "return", "returns", "function", "file", "value", "number",
45     "actual", "instead", "part", "far", "done", "write", "wrote", "written", "change", "changed",
46     "reason", "happen", "happens", "example", "may", "might", "must", "another", "anymore", "ever",
47     "hope", "somebody", "someone", "maybe", "probably", "already", "yet", "able", "currently",
48     "tell", "told", "stuck", "confused", "call", "called", "don", "doesn", "didn", "isn",
49 }
50
51 # Phrases that signal a confused or stuck learner.
52 CONFUSION_TERMS = [
53     "confused", "confusing", "stuck", "don't understand", "dont understand", "do not understand",
54     "not sure", "no idea", "lost", "struggling", "frustrated", "can't figure", "cant figure",
55     "cannot figure", "doesn't work", "doesnt work", "not working", "what am i doing wrong",
56     "what is wrong", "what's wrong", "why does", "why is", "hours", "days", "desperate", "hopeless",
57 ]
58
59
60 def _nltk_resources():
61     """Stopwords, lemmatizer and sentiment analyser, downloading NLTK data if needed."""
62     import nltk
63     for pkg, path in [("stopwords", "corpora/stopwords"), ("wordnet", "corpora/wordnet"),
64                       ("omw-1.4", "corpora/omw-1.4"), ("vader_lexicon", "sentiment/vader_lexicon")]:

```

```

65     try:
66         nltk.data.find(path)
67     except LookupError:
68         nltk.download(pkg, quiet=True)
69 from nltk.corpus import stopwords
70 from nltk.sentiment import SentimentIntensityAnalyzer
71 from nltk.stem import WordNetLemmatizer
72 return set(stopwords.words("english")), WordNetLemmatizer(), SentimentIntensityAnalyzer()
73
74
75 def split_html(body: str) -> tuple[str, int]:
76     """Return (text without code blocks, number of code lines)."""
77     soup = BeautifulSoup(body or "", "html.parser")
78     code_lines = 0
79     for pre in soup.find_all("pre"):
80         code_lines += len([ln for ln in pre.get_text().splitlines() if ln.strip()])
81         pre.decompose()
82     text = soup.get_text(" ")
83     text = URL_RE.sub(" ", html.unescape(text))
84     return re.sub(r"\s+", " ", text).strip(), code_lines
85
86
87 def clean_questions(questions: pd.DataFrame, answers: pd.DataFrame) -> pd.DataFrame:
88     stop_words, lemmatizer, vader = _nltk_resources()
89     stop_words |= DOMAIN_STOPWORDS
90
91     df = questions.copy()
92     df["title"] = df["title"].fillna("").map(html.unescape)
93     parts = df["body"].fillna("").map(split_html)
94     df["body_text"] = parts.str[0]
95     df["n_code_lines"] = parts.str[1]
96     df["text_clean"] = (df["title"] + ". " + df["body_text"]).str.strip()
97
98     lemma_cache = {}
99
100    def tokenize(text):
101        out = []
102        for tok in TOKEN_RE.findall(text.lower()):
103            if tok not in lemma_cache:
104                lemma_cache[tok] = lemmatizer.lemmatize(tok)
105            lem = lemma_cache[tok]
106            if lem not in stop_words and tok not in stop_words and len(lem) > 2:
107                out.append(lem)
108        return " ".join(out)
109
110    df["tokens"] = df["text_clean"].map(tokenize)
111
112    raw_text = df["body"].fillna("").map(lambda b: BeautifulSoup(b, "html.parser").get_text(" "))
113    lower = df["text_clean"].str.lower()
114    df["n_words"] = df["text_clean"].str.split().str.len()
115    df["title_words"] = df["title"].str.split().str.len()
116    df["has_code"] = df["n_code_lines"] > 0
117    df["has_error_msg"] = raw_text.str.contains(ERROR_RE)
118    df["has_check50_fail"] = raw_text.str.contains(CHECK50_FAIL_RE)
119    df["n_question_marks"] = df["text_clean"].str.count(r"?\")
120    df["n_tags"] = df["tags"].fillna("").str.split("|").map(lambda t: len([x for x in t if x]))
121    df["sentiment"] = df["text_clean"].map(lambda t: vader.polarity_scores(t)["compound"])
122    df["confusion_score"] = lower.map(lambda t: sum(t.count(term) for term in CONFUSION_TERMS))
123    df["confusion_score"] = df["confusion_score"] / np.sqrt(df["n_words"].clip(lower=1)) * 10
124
125    # Outcomes: first answer time and who gave it.
126    first = (answers.sort_values("created_at")
127            .groupby("question_id")
128            .agg(first_answer_at=("created_at", "first"), first_answerer=("user_id", "first")))
129    df = df.merge(first, left_on="post_id", right_index=True, how="left")
130    df["hours_to_first_answer"] = (df["first_answer_at"] - df["created_at"]).dt.total_seconds() /
3600
131    df["has_accepted"] = df["accepted_answer_id"].notna()
132    df["is_answered"] = df["is_answered"].astype(bool)
133
134    keep = ["post_id", "user_id", "created_at", "title", "tags", "text_clean", "tokens",
135            "n_words", "title_words", "n_code_lines", "has_code", "has_error_msg",
136            "has_check50_fail",
137            "n_question_marks", "n_tags", "sentiment", "confusion_score", "score", "view_count",
138            "answer_count", "is_answered", "has_accepted", "accepted_answer_id",
139            "first_answer_at", "first_answerer", "hours_to_first_answer"]
140    return df[keep].sort_values("created_at").reset_index(drop=True)
141
142 def run():
143     raw = load_raw()
144     df = clean_questions(raw["questions"], raw["answers"])
145     df.to_parquet(PROCESSED_DIR / "questions_clean.parquet", index=False)

```

```

146     print(f"[preprocess] {len(df)} questions cleaned "
147           f"({df.has_code.mean():.0%} contain code, {df.has_error_msg.mean():.0%} mention an error)")
148     return df
149
150
151 if __name__ == "__main__":
152     run()

```

Output

```

=== preprocess: clean text and build question features
[preprocess] 4000 questions cleaned (80% contain code, 46% mention an error)
=== preprocess done in 29s

```

Table 3. *Sample rows of data/processed/questions_clean.parquet*

post_id	n_words	n_code_lines	has_error_msg	sentiment	confusion_score	tokens
32630	27	32	False	0.908	0.0	compile pointer...
32633	135	20	False	-0.264	0.861	key looping vigenere reached final step vigenere loop key pl...
32634	50	24	False	-0.392	1.414	recover explanation implement byte jpeg completely implement...
32639	34	53	True	-0.365	3.43	recovered image match pset3 figure recovered image jpg jpg s...
32642	87	0	False	0.504	0.0	expression always true test four byte buffer match header jp...
32644	63	21	False	0.271	0.0	pset8 finance calculaton sell implementing sell calculation ...

6. Topic Modeling (topic_model.py)

Aim: discover the discussion topics. LDA is fitted for $k = 6$ to 20 and the best k is chosen by NPMI coherence $\times$ diversity. BERTopic embeds each question with a sentence transformer, reduces the embeddings with UMAP and clusters them with HDBSCAN. Both models are scored on the same vocabulary with NPMI coherence, topic diversity and agreement (NMI) with the course's problem-set tags. The program also computes a difficulty index and topic trends for each topic.

Run: `python src/topic_model.py`

Source code: `src/topic_model.py` (285 lines)

```

1  """
2  Step 2: discover discussion topics with LDA (baseline) and BERTopic (transformer based).
3
4  Both models are scored with the same metrics so they can be compared fairly:
5      NPMI coherence    do a topic's top words really occur together? (-1..1, higher is better)
6      Topic diversity   share of unique words across all topics' top-25 words (0..1)
7      NMI vs tags       agreement with the course's own problem-set tags (0..1)
8
9  Outputs:
10     data/processed/question_topics.parquet    topic of every question (both models)
11     data/processed/embeddings.npy             sentence embeddings, reused by the recommender
12     outputs/topics.csv                       BERTopic topics + difficulty metrics
13     outputs/lda_topics.csv, outputs/lda_model_selection.csv
14     outputs/topic_model_comparison.csv, outputs/topic_trends.csv, outputs/topic_map.csv
15 """
16
17 import itertools
18 import warnings
19
20 import numpy as np
21 import pandas as pd
22 from scipy import sparse
23 from sklearn.decomposition import LatentDirichletAllocation
24 from sklearn.feature_extraction.text import CountVectorizer
25 from sklearn.metrics import normalized_mutual_info_score
26
27 from config import EMBEDDING_MODEL, OUTPUT_DIR, PROCESSED_DIR, RANDOM_SEED
28
29 warnings.filterwarnings("ignore", category=FutureWarning)
30
31 LDA_K_GRID = [6, 8, 10, 12, 14, 16, 18, 20]
32 TOP_N_COHERENCE = 10
33 TOP_N_DIVERSITY = 25
34 # 20 requested; one of them is the outlier topic (-1), so 19 real topics remain.
35 BERTOPIC_N_TOPICS = 20
36
37 # Course-structure tags used to validate topics against the CS50 syllabus.
38 COURSE_TAGS = {
39     "pset1": "Week 1 (C basics)", "mario": "Week 1 (C basics)", "cash": "Week 1 (C basics)",
40     "credit": "Week 1 (C basics)", "pset2": "Week 2 (Arrays)", "caesar": "Week 2 (Arrays)",
41     "readability": "Week 2 (Arrays)", "substitution": "Week 2 (Arrays)", "vigenere": "Week 2
42 (Arrays)", "scrabble": "Week 2 (Arrays)", "pset3": "Week 3 (Algorithms)", "plurality": "Week 3
43 (Algorithms)", "runoff": "Week 3 (Algorithms)", "tideman": "Week 3 (Algorithms)", "sort": "Week 3 (Algorithms)",
44     "pset4": "Week 4 (Memory)", "recover": "Week 4 (Memory)", "filter": "Week 4 (Memory)",
45     "volume": "Week 4 (Memory)", "blur": "Week 4 (Memory)", "pset4recover": "Week 4 (Memory)",
46     "pset4filter": "Week 4 (Memory)", "pset5": "Week 5 (Data structures)",
47     "speller": "Week 5 (Data structures)", "inheritance": "Week 5 (Data structures)",
48     "pset6": "Week 6 (Python)", "dna": "Week 6 (Python)", "python": "Week 6 (Python)",
49     "pset7": "Week 7 (SQL)", "sql": "Week 7 (SQL)", "movies": "Week 7 (SQL)", "fiftyville": "Week 7
50 (SQL)", "songs": "Week 7 (SQL)", "pset8": "Week 8-9 (Web / Flask)", "pset9": "Week 8-9 (Web / Flask)",
51     "finance": "Week 8-9 (Web / Flask)", "flask": "Week 8-9 (Web / Flask)", "html": "Week 8-9 (Web /
52 Flask)", "homepage": "Week 8-9 (Web / Flask)", "birthdays": "Week 8-9 (Web / Flask)",
53     "javascript": "Week 8-9 (Web / Flask)", "cs50p": "CS50P (Python course)",
54     "cs50w": "CS50W (Web course)", "cs50ai": "CS50AI (AI course)", "final-project": "Final project",
55     "check50": "Tools (check50 / IDE)", "cs50-ide": "Tools (check50 / IDE)",
56     "codespaces": "Tools (check50 / IDE)", "submit50": "Tools (check50 / IDE)",
57 }
58
59
60 def course_unit(tags: str) -> str | None:

```

```

61     """Map a question's tags to its course unit (specific tags win over generic ones)."""
62     units = [COURSE_TAGS[t] for t in str(tags).split("|") if t in COURSE_TAGS]
63     specific = [u for u in units if not u.startswith("Tools")]
64     return (specific or units or [None])[0]
65
66
67 # ----- metrics
68
69 def npmi_coherence(topic_words: list[list[str]], doc_term: sparse.csr_matrix, vocab: dict) ->
list[float]:
70     """Average normalised PMI of all word pairs, using document co-occurrence."""
71     binary = (doc_term > 0).astype(np.float32).tocsc()
72     n_docs = binary.shape[0]
73     doc_freq = np.asarray(binary.sum(axis=0)).ravel()
74     scores = []
75     for words in topic_words:
76         idx = [vocab[w] for w in words if w in vocab]
77         pair_scores = []
78         for i, j in itertools.combinations(idx, 2):
79             co = binary[:, i].multiply(binary[:, j]).sum()
80             if co == 0:
81                 pair_scores.append(-1.0)
82                 continue
83             p_ij = co / n_docs
84             pmi = np.log(p_ij / ((doc_freq[i] / n_docs) * (doc_freq[j] / n_docs)))
85             pair_scores.append(pmi / -np.log(p_ij))
86         scores.append(float(np.mean(pair_scores)) if pair_scores else 0.0)
87     return scores
88
89
90 def topic_diversity(topic_words: list[list[str]]) -> float:
91     words = [w for topic in topic_words for w in topic[:TOP_N_DIVERSITY]]
92     return len(set(words)) / max(len(words), 1)
93
94
95 def tag_nmi(topics: pd.Series, units: pd.Series) -> float:
96     mask = units.notna() & (topics != -1)
97     return float(normalized_mutual_info_score(units[mask], topics[mask]))
98
99
100 # ----- LDA
101
102 def make_vectorizer(min_df=5):
103     return CountVectorizer(token_pattern=r"\S+", ngram_range=(1, 2), min_df=min_df,
104                             max_df=0.4, max_features=6000)
105
106
107 def fit_lda(tokens: pd.Series, units: pd.Series):
108     vectorizer = make_vectorizer()
109     doc_term = vectorizer.fit_transform(tokens)
110     vocab = vectorizer.vocabulary_
111     words = vectorizer.get_feature_names_out()
112
113     rows, best = [], None
114     for k in LDA_K_GRID:
115         lda = LatentDirichletAllocation(n_components=k, learning_method="batch", max_iter=40,
116                                         random_state=RANDOM_SEED, n_jobs=-1)
117         doc_topic = lda.fit_transform(doc_term)
118         top_words = [[words[i] for i in comp.argsort()[::-1][:TOP_N_DIVERSITY]] for comp in
lda.components_]
119         npmi = float(np.mean(npmi_coherence([w[:TOP_N_COHERENCE] for w in top_words], doc_term,
vocab)))
120         row = {"k": k, "npmi": npmi, "diversity": topic_diversity(top_words),
121               "perplexity": float(lda.perplexity(doc_term)),
122               "nmi_vs_tags": tag_nmi(pd.Series(doc_topic.argmax(1)), units)}
123         rows.append(row)
124         print(f" LDA k={k:2d} NPMI={npmi:.3f} diversity={row['diversity']:.2f}")
125         # Select on coherence x diversity so we don't pick many near-duplicate topics.
126         score = npmi * row["diversity"]
127         if best is None or score > best[0]:
128             best = (score, k, lda, doc_topic, top_words)
129
130     _, k, lda, doc_topic, top_words = best
131     selection = pd.DataFrame(rows)
132     topics = pd.DataFrame({
133         "lda_topic": range(k),
134         "top_words": [" ".join(w[:10]) for w in top_words],
135         "npmi": npmi_coherence([w[:TOP_N_COHERENCE] for w in top_words], doc_term, vocab),
136         "n_questions": np.bincount(doc_topic.argmax(1), minlength=k),
137     })
138     return selection, topics, doc_topic, doc_term, vocab
139
140

```

```

141 # ----- BERTopic
142
143 def compute_embeddings(texts: list[str], post_ids: np.ndarray) -> np.ndarray:
144     """Sentence embeddings, cached on disk and reused when the questions are unchanged."""
145     path, ids_path = PROCESSED_DIR / "embeddings.npy", PROCESSED_DIR / "embedding_post_ids.npy"
146     if path.exists() and ids_path.exists() and np.array_equal(np.load(ids_path), post_ids):
147         return np.load(path)
148     from sentence_transformers import SentenceTransformer
149     model = SentenceTransformer(EMBEDDING_MODEL)
150     emb = model.encode(texts, batch_size=64, show_progress_bar=True, normalize_embeddings=True)
151     np.save(path, emb.astype(np.float32))
152     np.save(ids_path, post_ids)
153     return emb
154
155
156 def ctfidf_vectorizer():
157     # BERTopic fits this on one merged document per topic, so min_df counts topics, not
158     # questions: keep it at 1 and filter to the shared reference vocabulary afterwards.
159     return CountVectorizer(token_pattern=r"\S+", ngram_range=(1, 2), min_df=1)
160
161
162 def fit_bertopic(tokens: list[str], embeddings: np.ndarray):
163     from bertopic import BERTopic
164     from bertopic.vectorizers import ClassTfidfTransformer
165     from hdbscan import HDBSCAN
166     from umap import UMAP
167
168     umap_model = UMAP(n_neighbors=15, n_components=5, min_dist=0.0, metric="cosine",
169                       random_state=RANDOM_SEED)
170     hdbscan_model = HDBSCAN(min_cluster_size=30, min_samples=10, metric="euclidean",
171                             cluster_selection_method="eom", prediction_data=True)
172     model = BERTopic(
173         embedding_model=None, # embeddings are precomputed
174         umap_model=umap_model,
175         hdbscan_model=hdbscan_model,
176         vectorizer_model=ctfidf_vectorizer(),
177         ctfidf_model=ClassTfidfTransformer(reduce_frequent_words=True),
178         top_n_words=100,
179         calculate_probabilities=False,
180     )
181     # Topic words come from the cleaned tokens; clustering uses the full-sentence embeddings.
182     topics, _ = model.fit_transform(tokens, embeddings)
183     outlier_share = float(np.mean(np.array(topics) == -1))
184     # Merge near-duplicate clusters (e.g. several 'recover' clusters) to a size comparable with LDA.
185     model.reduce_topics(tokens, nr_topics=BERTOPICTOPICS)
186     # Give outlier questions the topic whose centre is closest in embedding space.
187     topics = model.reduce_outliers(tokens, model.topics_, strategy="embeddings",
188     embeddings=embeddings)
189     model.update_topics(tokens, topics=topics, vectorizer_model=ctfidf_vectorizer(),
190     ctfidf_model=ClassTfidfTransformer(reduce_frequent_words=True),
191     top_n_words=100)
192     return model, np.array(topics), outlier_share
193
194 # ----- analysis
195
196 def topic_difficulty(q: pd.DataFrame) -> pd.DataFrame:
197     """Per-topic signals of how hard learners find it."""
198     g = q.groupby("topic")
199     out = pd.DataFrame({
200         "n_questions": g.size(),
201         "pct_unanswered": 100 * (1 - g["is_answered"].mean()),
202         "pct_accepted": 100 * g["has_accepted"].mean(),
203         "median_hours_to_answer": g["hours_to_first_answer"].median(),
204         "avg_confusion": g["confusion_score"].mean(),
205         "avg_sentiment": g["sentiment"].mean(),
206         "pct_with_error": 100 * g["has_error_msg"].mean(),
207         "avg_views": g["view_count"].mean(),
208     })
209     z = lambda s: (s - s.mean()) / (s.std() or 1)
210     out["difficulty_index"] = (z(out["pct_unanswered"]) + z(np.log1p(out["median_hours_to_answer"]))
211     + z(out["avg_confusion"])) / 3
212     return out.round(3)
213
214 def run():
215     q = pd.read_parquet(PROCESSED_DIR / "questions_clean.parquet")
216     q = q[q["tokens"].str.split().str.len() >= 3].reset_index(drop=True)
217     q["course_unit"] = q["tags"].map(course_unit)
218     print(f"[Topics] {len(q)} questions, {q.course_unit.notna().mean():.0%} mapped to a course unit")
219
220     # ---- LDA baseline
221     selection, lda_topics, doc_topic, doc_term, vocab = fit_lda(q["tokens"], q["course_unit"])

```

```

222     q["lda_topic"] = doc_topic.argmax(1)
223     q["lda_prob"] = doc_topic.max(1)
224     selection.to_csv(OUTPUT_DIR / "lda_model_selection.csv", index=False)
225     lda_topics.to_csv(OUTPUT_DIR / "lda_topics.csv", index=False)
226
227     # ---- BERTopic
228     embeddings = compute_embeddings(q["text_clean"].tolist(), q["post_id"].to_numpy())
229     model, topics, outlier_share = fit_bertopic(q["tokens"].tolist(), embeddings)
230     q["topic"] = topics
231     topic_ids = sorted(t for t in set(topics) if t != -1)
232     # Keep words from the shared reference vocabulary (>= 5 questions) so both models are scored
233     # alike.
234     top_words = {t: [w for w, _ in model.get_topic(t) if w in vocab][:TOP_N_DIVERSITY] for t in
235     topic_ids}
236
237     # ---- labels: dominant course unit + top words
238     info = pd.DataFrame({"topic": topic_ids})
239     info["top_words"] = info["topic"].map(lambda t: ", ".join(top_words[t][:10]))
240     dominant = (q.dropna(subset=["course_unit"]).groupby("topic")["course_unit"]
241     .agg(lambda s: s.value_counts().index[0]))
242     purity = (q.dropna(subset=["course_unit"]).groupby("topic")["course_unit"]
243     .agg(lambda s: s.value_counts(normalize=True).iloc[0]))
244     info["course_unit"] = info["topic"].map(dominant)
245     info["unit_purity"] = info["topic"].map(purity).round(3)
246     info["label"] = info["topic"].map(lambda t: " / ".join(top_words[t][:3]))
247     info["npmi"] = npmi_coherence([top_words[t][:TOP_N_COHERENCE] for t in topic_ids], doc_term,
248     vocab)
249
250     info = info.merge(topic_difficulty(q), left_on="topic", right_index=True)
251     info = info.sort_values("n_questions", ascending=False)
252     info.to_csv(OUTPUT_DIR / "topics.csv", index=False)
253     q["topic_label"] = q["topic"].map(info.set_index("topic")["label"])
254
255     # ---- model comparison
256     best_lda = selection.loc[selection["k"] == len(lda_topics)].iloc[0]
257     comparison = pd.DataFrame([
258     {"model": f"LDA (k={len(lda_topics)})", "n_topics": len(lda_topics), "npmi":
259     best_lda["npmi"],
260     "diversity": best_lda["diversity"], "nmi_vs_course_tags": best_lda["nmi_vs_tags"],
261     "outlier_share_before_reduction": 0.0},
262     {"model": "BERTopic (MiniLM + UMAP + HDBSCAN)", "n_topics": len(topic_ids),
263     "npmi": float(info["npmi"].mean()), "diversity": topic_diversity(list(top_words.values())),
264     "nmi_vs_course_tags": tag_nmi(q["topic"], q["course_unit"]),
265     "outlier_share_before_reduction": outlier_share},
266     ]).round(4)
267     comparison.to_csv(OUTPUT_DIR / "topic_model_comparison.csv", index=False)
268
269     # ---- topic trends per quarter
270     q["period"] = q["created_at"].dt.to_period("Q").astype(str)
271     trends = q.groupby(["period", "topic"]).size().rename("n_questions").reset_index()
272     trends["share"] = trends["n_questions"] /
273     trends.groupby("period")["n_questions"].transform("sum")
274     trends.to_csv(OUTPUT_DIR / "topic_trends.csv", index=False)
275
276     # ---- 2-D map of all questions for the dashboard
277     from umap import UMAP
278     xy = UMAP(n_neighbors=15, n_components=2, min_dist=0.1, metric="cosine",
279     random_state=RANDOM_SEED).fit_transform(embeddings)
280     pd.DataFrame({"post_id": q["post_id"], "x": xy[:, 0], "y": xy[:, 1], "topic": q["topic"],
281     "title": q["title"]}).to_csv(OUTPUT_DIR / "topic_map.csv", index=False)
282
283     q[["post_id", "topic", "topic_label", "lda_topic", "lda_prob", "course_unit"]].to_parquet(
284     PROCESSED_DIR / "question_topics.parquet", index=False)
285
286     print(comparison.to_string(index=False))
287     return info
288
289 if __name__ == "__main__":
290     run()

```

Output

```

=== topics: LDA + BERTopic topic modeling (~5 min)
[topics] 3960 questions, 82% mapped to a course unit
  LDA k= 6  NPMI=0.063  diversity=0.89
  LDA k= 8  NPMI=0.142  diversity=0.84
  LDA k=10  NPMI=0.119  diversity=0.82
  LDA k=12  NPMI=0.176  diversity=0.77
  LDA k=14  NPMI=0.185  diversity=0.78
  LDA k=16  NPMI=0.176  diversity=0.77

```

```

LDA k=18  NPMI=0.173  diversity=0.74
LDA k=20  NPMI=0.243  diversity=0.73
                                model  n_topics  npmi  diversity  nmi_vs_course_tags
outlier_share_before_reduction
                                LDA (k=20)          20  0.2426      0.7300          0.3839
0.0000
BERTopic (MiniLM + UMAP + HDBSCAN)          19  0.3099      0.8653          0.5613
0.1192
=== topics done in 134s

```

Table 4. LDA model selection (outputs/lda_model_selection.csv)

k	npmi	diversity	perplexity	nmi_vs_tags
6	0.063	0.893	1184.368	0.263
8	0.142	0.84	1118.858	0.312
10	0.119	0.82	1068.342	0.329
12	0.176	0.77	1041.21	0.329
14	0.185	0.783	1020.49	0.345
16	0.176	0.768	1009.161	0.345
18	0.173	0.744	1008.433	0.348
20	0.243	0.73	964.648	0.384

Table 5. LDA vs BERTopic (outputs/topic_model_comparison.csv)

model	n_topics	npmi	diversity	nmi_vs_course_tags
LDA (k=20)	20	0.243	0.73	0.384
BERTopic (MiniLM + UMAP + HDBSCAN)	19	0.31	0.865	0.561

Table 6. BERTopic topics with difficulty measures (outputs/topics.csv)

topic	label	course_unit	n_questions	pct_unanswered	median_hours_to_answer	difficulty_index
0	mario / ide / project	Week 1 (C basics)	758	41.69	5.56	0.04
1	speller / word / dictionary	Week 5 (Data structures)	553	28.57	2.96	-0.41
3	caesar / key / character	Week 2 (Arrays)	334	25.45	2.89	-0.68
2	filter / blur / pixel	Week 4 (Memory)	301	29.24	4.13	-0.04
4	recover / pset4 recover / pset4	Week 4 (Memory)	290	26.55	2.7	-0.45
5	credit / cash / cent	Week 1 (C basics)	230	26.52	4.68	-0.26
6	buy / finance / stock	Week 8-9 (Web / Flask)	220	42.73	7.76	0.47
7	vote / candidate / winner	Week 3 (Algorithms)	189	33.86	4.78	-0.16
9	loop / array / string	Week 1 (C basics)	178	33.15	3.84	-0.29
8	readability / grade / sentence	Week 2 (Arrays)	161	34.16	2.35	-0.32
17	html / flask / form	Week 8-9 (Web / Flask)	113	55.75	12.1	1.1
14	identifier / int / lecture	Week 2 (Arrays)	112	30.36	2.61	-0.49
10	dna / sequence / str	Week 6 (Python)	90	44.44	9.07	0.5
13	jpg / recover / image	Week 4 (Memory)	81	19.75	2.52	-0.29
11	resize / bmp / resize less	Week 3 (Algorithms)	79	21.52	3.67	0.1
12	register / username / register html	Week 8-9 (Web / Flask)	77	44.16	8.28	0.92
16	import / sql / database	Week 7 (SQL)	70	44.29	2.62	-0.69

topic	label	course_unit	n_questions	pct_unanswered	median_hours_to_answer	difficulty_index
18	pair / tideman / sort	Week 3 (Algorithms)	67	44.78	12.68	1.3
15	sql / movie / query	Week 7 (SQL)	57	35.09	5.07	-0.35

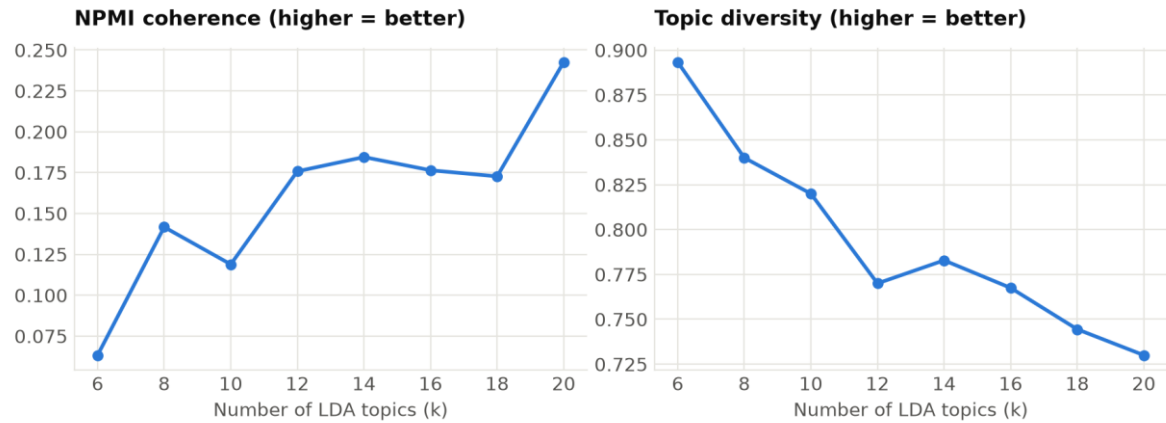


Figure 1. LDA coherence and diversity for each number of topics

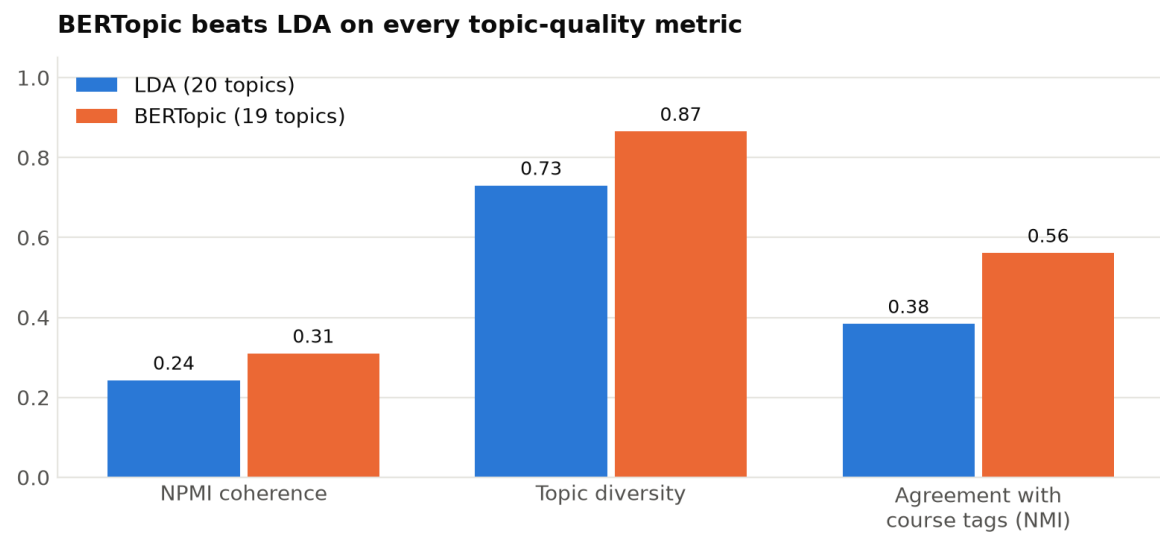


Figure 2. BERTopic vs LDA on the three quality metrics

Topic difficulty index
(unanswered rate + time to answer + learner confusion)

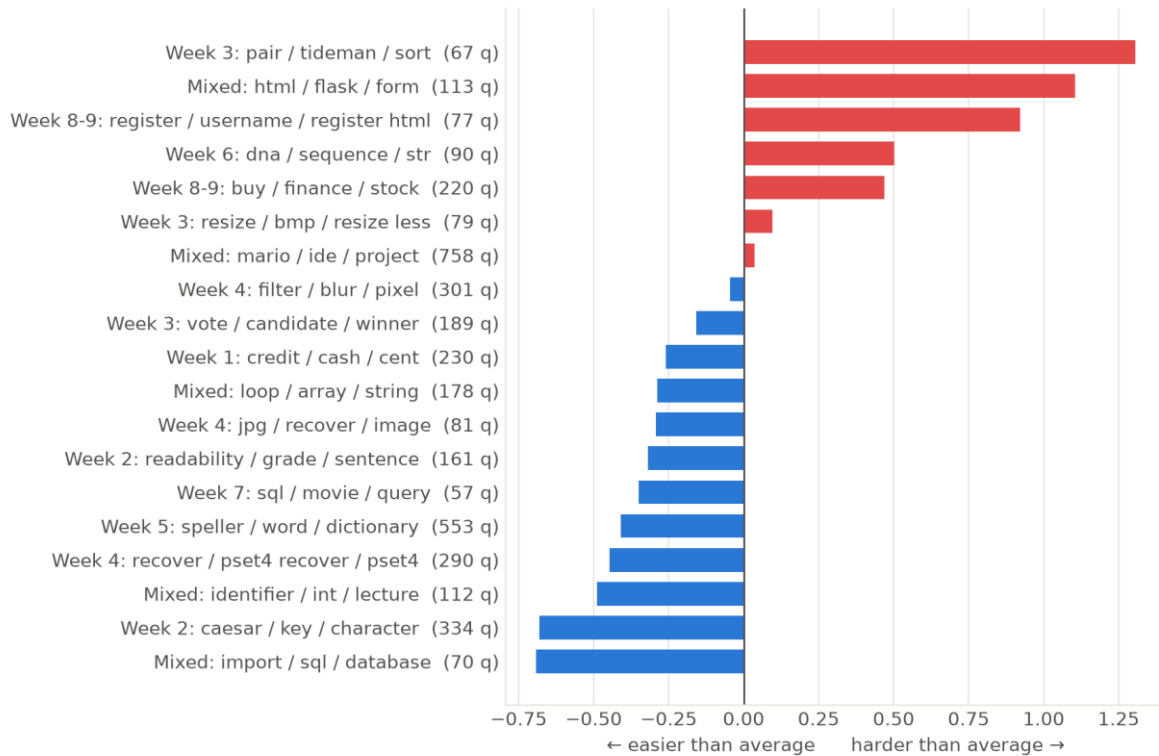


Figure 3. Topic difficulty index

How the discussion mix changed over the years
(years with 100+ questions)

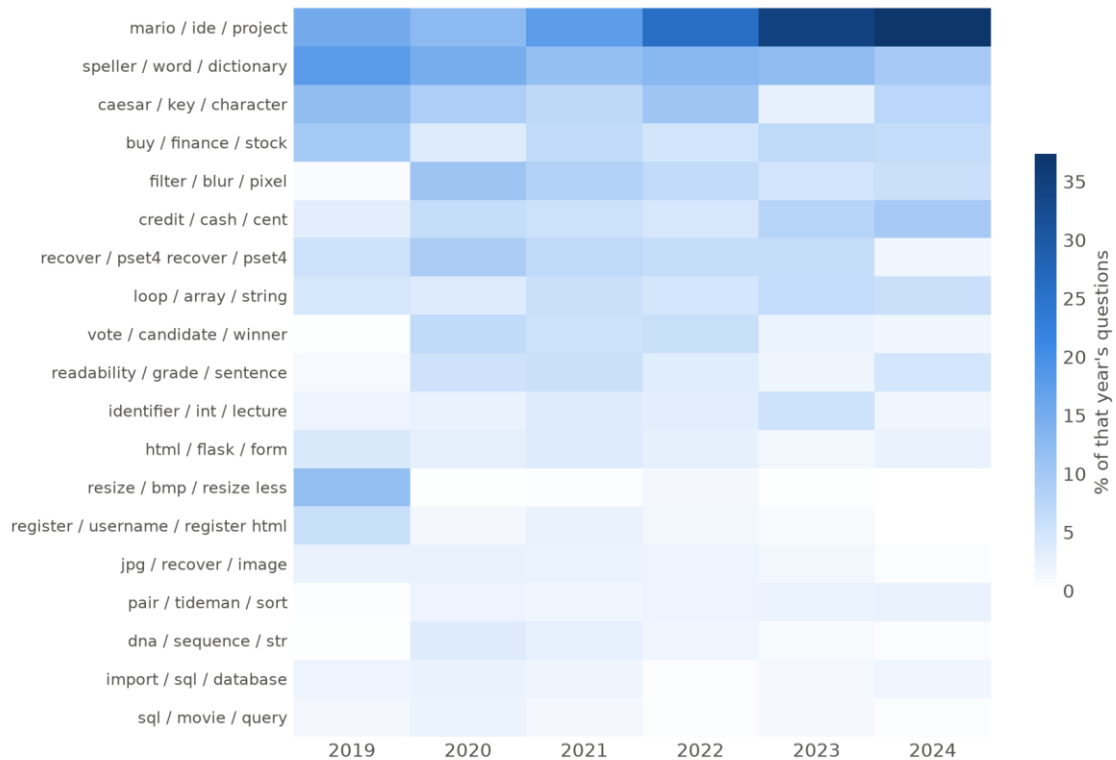


Figure 4. Topic share by year

7. Knowledge Network (network.py)

Aim: build the directed helper → learner network (answers, and comments by someone other than the thread's asker) and measure it. For every user the program computes degree and strength, ExpertiseRank (PageRank on learner → helper links), PageRank, betweenness, clustering, k-core number, Louvain community, participation coefficient and a rule-based role. It then validates the measures against Stack Exchange reputation and accepted answers, and builds yearly snapshots.

Run: `python src/network.py`

Source code: `src/network.py` (267 lines)

```

1  """
2  Step 3: build the knowledge network (who helps whom) and measure its structure.
3
4  Graph: directed, edge helper -> learner, weight = number of interactions. Only
5  knowledge-giving interactions are used (answers, and comments by someone other than the
6  thread's asker), see data_loader.help_interactions.
7
8  Outputs:
9      outputs/user_metrics.csv          centrality, community and role of every user
10     outputs/help_edges.csv            aggregated weighted edge list
11     outputs/network_stats.json        global statistics of the network
12     outputs/network_over_time.csv     yearly snapshots
13     outputs/community_topics.csv      what each community talks about
14     outputs/topic_network_edges.csv   topics linked by shared helpers
15     outputs/network_validation.csv     do centrality scores agree with reputation?
16  """
17
18  import json
19
20  import networkx as nx
21  import numpy as np
22  import pandas as pd
23  from scipy.stats import spearmanr
24
25  from config import OUTPUT_DIR, PROCESSED_DIR, RANDOM_SEED
26  from data_loader import help_interactions, load_raw
27
28
29  def gini(values) -> float:
30      x = np.sort(np.asarray(values, dtype=float))
31      if x.sum() == 0:
32          return 0.0
33      n = len(x)
34      return float((2 * np.arange(1, n + 1) - n - 1).dot(x) / (n * x.sum()))
35
36
37  def aggregate_edges(interactions: pd.DataFrame) -> pd.DataFrame:
38      return (interactions.groupby(["source_user", "target_user"])
39              .agg(weight=("id", "size"),
40                  n_answers=("type", lambda s: int((s == "answer").sum())),
41                  first_at=("created_at", "min"), last_at=("created_at", "max"))
42              .reset_index())
43
44
45  def build_graph(edges: pd.DataFrame) -> nx.DiGraph:
46      g = nx.DiGraph()
47      for e in edges.itertuples():
48          g.add_edge(int(e.source_user), int(e.target_user), weight=int(e.weight))
49      return g
50
51
52  def user_activity(raw) -> pd.DataFrame:
53      q, a, c = raw["questions"], raw["answers"], raw["comments"]
54      act = pd.DataFrame({
55          "n_questions": q.groupby("user_id").size(),
56          "n_answers": a.groupby("user_id").size(),
57          "n_accepted": a[a["is_accepted"].astype(bool)].groupby("user_id").size(),
58          "n_comments": c.groupby("user_id").size(),
59          "answer_score": a.groupby("user_id")["score"].sum(),
60      }).fillna(0).astype(int)
61      first_seen = pd.concat([q[["user_id", "created_at"]], a[["user_id", "created_at"]],
62                             c[["user_id", "created_at"]]]).groupby("user_id")["created_at"]
63      act["first_seen"] = first_seen.min()
64      act["last_seen"] = first_seen.max()

```

```

65     act.index = act.index.astype("int64")
66     return act
67
68
69 def participation_coefficient(ug: nx.Graph, community: dict) -> dict:
70     """Share of a node's links that go to other communities (0 = inward, ~1 = bridge)."""
71     out = {}
72     for node in ug:
73         nbrs = list(ug[node])
74         if not nbrs:
75             out[node] = 0.0
76             continue
77         counts = pd.Series([community[n] for n in nbrs]).value_counts(normalize=True)
78         out[node] = float(1 - (counts ** 2).sum())
79     return out
80
81
82 def assign_roles(m: pd.DataFrame) -> pd.Series:
83     """Rule-based roles, from most to least specific."""
84     top_help = m["out_strength"].quantile(0.98)
85     top_betw = m["betweenness"].quantile(0.95)
86     role = pd.Series("Peripheral learner", index=m.index)
87     role[(m["out_strength"] >= 1) & (m["in_strength"] >= 1)] = "Active participant"
88     role[(m["out_strength"] >= 3) & (m["out_strength"] > m["in_strength"])] = "Helper"
89     role[(m["betweenness"] >= top_betw) & (m["participation_coef"] >= 0.3) & (m["out_strength"] >=
3)] = "Bridge"
90     role[(m["out_strength"] >= max(top_help, 10)) & (m["n_answers"] >= 10)] = "Core expert"
91     role[(m["out_strength"] == 0) & (m["in_strength"] == 0)] = "Isolated"
92     return role
93
94
95 def global_stats(g: nx.DiGraph, ug: nx.Graph, communities, m: pd.DataFrame) -> dict:
96     giant = max(nx.weakly_connected_components(g), key=len)
97     help_given = m["out_strength"].sort_values(ascending=False)
98     total = help_given.sum()
99     n = len(help_given)
100     out_deg = np.array([d for _, d in g.out_degree() if d > 0])
101     # Slope of the log-log complementary CDF: a straight line means a heavy-tailed (scale-free-like)
network.
102     vals = np.sort(np.unique(out_deg))
103     ccdf = np.array([(out_deg >= v).mean() for v in vals])
104     slope = float(np.polyfit(np.log(vals), np.log(ccdf), 1)[0]) if len(vals) > 2 else None
105     helpers = (m["out_strength"] > 0).sum()
106     return {
107         "n_users": g.number_of_nodes(),
108         "n_edges": g.number_of_edges(),
109         "n_interactions": int(sum(d["weight"] for _, _, d in g.edges(data=True))),
110         "density": nx.density(g),
111         "reciprocity": nx.reciprocity(g),
112         "avg_clustering": nx.average_clustering(ug),
113         "transitivity": nx.transitivity(ug),
114         "degree_assortativity": nx.degree_assortativity_coefficient(ug),
115         "giant_component_share": len(giant) / g.number_of_nodes(),
116         "n_communities": len(communities),
117         "n_communities_10plus": sum(len(c) >= 10 for c in communities),
118         "modularity": nx.community.modularity(ug, communities, weight="weight"),
119         "n_helpers": int(helpers),
120         "pct_users_who_help": helpers / n,
121         "help_share_top1pct": float(help_given.iloc[: max(1, n // 100)].sum() / total),
122         "help_share_top10pct": float(help_given.iloc[: max(1, n // 10)].sum() / total),
123         "gini_help_given": gini(help_given),
124         "out_degree_ccdf_slope": slope,
125     }
126
127
128 def snapshots(interactions: pd.DataFrame) -> pd.DataFrame:
129     rows = []
130     for year, part in interactions.groupby(interactions["created_at"].dt.year):
131         e = aggregate_edges(part)
132         g = build_graph(e)
133         if g.number_of_nodes() < 5:
134             continue
135         ug = g.to_undirected()
136         comms = nx.community.louvain_communities(ug, weight="weight", seed=RANDOM_SEED)
137         strength = pd.Series(dict(g.out_degree(weight="weight"))).sort_values(ascending=False)
138         rows.append({
139             "year": int(year), "active_users": g.number_of_nodes(), "edges": g.number_of_edges(),
140             "interactions": int(e["weight"].sum()), "density": nx.density(g),
141             "reciprocity": nx.reciprocity(g), "avg_clustering": nx.average_clustering(ug),
142             "giant_component_share": len(max(nx.weakly_connected_components(g), key=len)) /
g.number_of_nodes(),
143             "modularity": nx.community.modularity(ug, comms, weight="weight"),
144             "n_helpers": int((strength > 0).sum()),

```

```

145         "help_share_top10pct": float(strength.iloc[: max(1, len(strength) // 10)].sum() /
146         strength.sum()),
147     })
148     return pd.DataFrame(rows)
149
150 def topic_participation(raw, qt: pd.DataFrame) -> pd.DataFrame:
151     """One row per (user, topic, role): who asks about and who helps with each topic."""
152     q = raw["questions"][["post_id", "user_id"]].merge(qt[["post_id", "topic"]], on="post_id")
153     a = raw["answers"][["question_id", "user_id"]].merge(
154         qt[["post_id", "topic"]], left_on="question_id", right_on="post_id")
155     asks = q.assign(role="ask")[["user_id", "topic", "role"]]
156     helps = a.assign(role="help")[["user_id", "topic", "role"]]
157     return pd.concat([asks, helps]).dropna(subset=["user_id"])
158
159
160 def run():
161     raw = load_raw()
162     inter = help_interactions(raw)
163
164     edges = aggregate_edges(inter)
165     edges.to_csv(OUTPUT_DIR / "help_edges.csv", index=False)
166     g = build_graph(edges)
167     ug = nx.Graph()
168     for u, v, d in g.edges(data=True):
169         w = ug[u][v]["weight"] + d["weight"] if ug.has_edge(u, v) else d["weight"]
170         ug.add_edge(u, v, weight=w)
171     print(f"[network] {g.number_of_nodes()} users, {g.number_of_edges()} helper->learner links")
172
173     # ---- centrality
174     m = pd.DataFrame(index=pd.Index(list(g.nodes), name="user_id"))
175     m["out_degree"] = pd.Series(dict(g.out_degree()))
176     m["in_degree"] = pd.Series(dict(g.in_degree()))
177     m["out_strength"] = pd.Series(dict(g.out_degree(weight="weight")))
178     m["in_strength"] = pd.Series(dict(g.in_degree(weight="weight")))
179     # ExpertiseRank: PageRank on learner -> helper links, so helping people who themselves
180     # help others counts more (Zhang, Ackerman & Adamic, 2007).
181     m["expertise_rank"] = pd.Series(nx.pagerank(g.reverse(copy=False), weight="weight"))
182     m["pagerank"] = pd.Series(nx.pagerank(g, weight="weight"))
183     m["betweenness"] = pd.Series(nx.betweenness_centrality(ug, k=min(1000, len(ug))),
184     seed=RANDOM_SEED))
185     m["clustering"] = pd.Series(nx.clustering(ug))
186     simple = nx.Graph(ug)
187     simple.remove_edges_from(nx.selfloop_edges(simple))
188     m["core_number"] = pd.Series(nx.core_number(simple))
189
190     # ---- communities
191     communities = nx.community.louvain_communities(ug, weight="weight", seed=RANDOM_SEED)
192     communities = sorted(communities, key=len, reverse=True)
193     community_of = {node: i for i, c in enumerate(communities) for node in c}
194     m["community"] = pd.Series(communities)
195     m["participation_coef"] = pd.Series(participation_coefficient(ug, community_of))
196
197     # ---- activity, reputation, roles
198     users = raw["users"].dropna(subset=["user_id"]).assign(user_id=lambda d:
199     d.user_id.astype("int64"))
200     m = m.join(users.set_index("user_id")[["user_name", "user_reputation"]]).join(user_activity(raw))
201     m[["n_questions", "n_answers", "n_accepted", "n_comments", "answer_score"]] = (
202         m[["n_questions", "n_answers", "n_accepted", "n_comments",
203         "answer_score"]].fillna(0).astype(int))
204     m["role"] = assign_roles(m)
205
206     # Main topic each user helps with (from the topic model, when available).
207     qt_path = PROCESSED_DIR / "question_topics.parquet"
208     if qt_path.exists():
209         qt = pd.read_parquet(qt_path)
210         part = topic_participation(raw, qt)
211         part["user_id"] = part["user_id"].astype("int64")
212         main_help = part[part.role == "help"].groupby("user_id")["topic"].agg(lambda s:
213         s.value_counts().index[0])
214         m["main_help_topic"] = main_help.astype("Int64")
215         m["n_topics_helped"] = part[part.role == "help"].groupby("user_id")["topic"].nunique()
216         m["n_topics_helped"] = m["n_topics_helped"].fillna(0).astype(int)
217
218     # Community profiles: which topics each community's members ask about or help with.
219     cp = part.merge(m[["community"]], left_on="user_id", right_index=True)
220     prof = cp.groupby(["community", "topic"]).size().rename("n").reset_index()
221     prof["share"] = prof["n"] / prof.groupby("community")["n"].transform("sum")
222     prof = prof[prof["community"].isin(range(15))].sort_values(["community", "share"],
223     ascending=[True, False])
224     prof.groupby("community").head(5).to_csv(OUTPUT_DIR / "community_topics.csv", index=False)
225
226     # Topic network: two topics are linked when the same people help with both.

```

```

222     helpers = part[part.role == "help"].drop_duplicates(["user_id", "topic"])
223     pairs = helpers.merge(helpers, on="user_id")
224     pairs = pairs[pairs.topic_x < pairs.topic_y]
225     tn = pairs.groupby(["topic_x", "topic_y"]).size().rename("shared_helpers").reset_index()
226     tn.rename(columns={"topic_x": "topic_a", "topic_y": "topic_b"}).to_csv(
227         OUTPUT_DIR / "topic_network_edges.csv", index=False)
228
229     m = m.sort_values("expertise_rank", ascending=False)
230     m.reset_index().to_csv(OUTPUT_DIR / "user_metrics.csv", index=False)
231
232     # ---- communities summary
233     comm = (m.groupby("community")
234             .agg(size=("out_strength", "size"), help_given=("out_strength", "sum"),
235                 top_member=("user_name", "first"))
236             .sort_values("size", ascending=False))
237     comm.reset_index().to_csv(OUTPUT_DIR / "communities.csv", index=False)
238
239     # ---- global stats, validation, time
240     stats = global_stats(g, ug, communities, m)
241     stats["role_counts"] = m["role"].value_counts().to_dict()
242     with open(OUTPUT_DIR / "network_stats.json", "w") as f:
243         json.dump(stats, f, indent=2, default=float)
244
245     # Validate on helpers only: most users never answer, and thousands of tied zeros would
246     # swamp the rank correlation.
247     val = []
248     helpers = m[(m["n_answers"] > 0) & m["user_reputation"].notna()]
249     for metric in ["expertise_rank", "pagerank", "out_strength", "betweenness", "core_number"]:
250         for target in ["user_reputation", "n_accepted"]:
251             rho, p = spearmanr(helpers[metric], helpers[target])
252             top_metric = set(helpers.nlargest(20, metric).index)
253             top_target = set(helpers.nlargest(20, target).index)
254             val.append({"metric": metric, "compared_with": target, "n_helpers": len(helpers),
255                        "spearman_rho": rho, "p_value": p,
256                        "top20_overlap": len(top_metric & top_target) / 20})
257     pd.DataFrame(val).round(4).to_csv(OUTPUT_DIR / "network_validation.csv", index=False)
258
259     snapshots(inter).round(4).to_csv(OUTPUT_DIR / "network_over_time.csv", index=False)
260
261     print(f"[network] {stats['n_communities']} communities (modularity {stats['modularity']:.2f}); "
262           f"top 10% of users give {stats['help_share_top10pct']:.0%} of all help")
263     return m
264
265
266 if __name__ == "__main__":
267     run()

```

Output

```

=== network: knowledge network, centrality, communities (~1 min)
[network] 2891 users, 4245 helper->learner links
[network] 118 communities (modularity 0.60); top 10% of users give 92% of all help
=== network done in 8s

```

Table 7. Global network statistics (outputs/network_stats.json)

Measure	Value
n_users	2891
n_edges	4245
n_interactions	6930
density	0.0005
reciprocity	0.0353
avg_clustering	0.1575
transitivity	0.0043
degree_assortativity	-0.3045
giant_component_share	0.9339
n_communities	118
n_communities_10plus	16
modularity	0.6042

Measure	Value
n_helpers	872
pct_users_who_help	0.3016
help_share_top1pct	0.8063
help_share_top10pct	0.9159
gini_help_given	0.955
out_degree_ccdf_slope	-0.7879

Table 8. *Top 10 users by ExpertiseRank (outputs/user_metrics.csv)*

user_name	role	out_strength	in_strength	n_answers	n_accepted	expertise_rank	betweenness	community
Cliff B	Core expert	1861	107	1160	809	0.1756	0.5724	0
UpAndAdam	Core expert	558	19	140	52	0.075	0.0808	2
DinoCoderSaurus	Core expert	1043	50	523	282	0.0591	0.2944	1
curiouskiwi	Core expert	295	35	191	112	0.0276	0.0916	5
kcw78	Core expert	97	22	44	16	0.022	0.0252	9
Blauelf	Core expert	409	24	249	160	0.0217	0.0807	3
Robert S. Pratt	Core expert	182	17	84	59	0.0139	0.031	7
Tritum	Core expert	157	13	101	51	0.0127	0.0359	4
Andreas Wenzel	Core expert	32	6	11	4	0.0109	0.0071	2
Fuelled_By_Coffee	Core expert	70	5	38	15	0.0093	0.021	10

Table 9. *Largest communities (outputs/communities.csv)*

community	size	help_given	top_member
0	648	2000	Cliff B
1	474	1258	DinoCoderSaurus
2	220	682	UpAndAdam
3	215	509	Blauelf
4	190	457	Tritum
5	167	372	curiouskiwi
6	129	277	Vsjain
7	99	268	Robert S. Pratt

Table 10. *Validation of network measures (outputs/network_validation.csv)*

metric	compared_with	n_helpers	spearman_rho	top20_overlap
expertise_rank	user_reputation	821	0.232	0.7
expertise_rank	n_accepted	821	0.154	0.85
pagerank	user_reputation	821	0.298	0.35
pagerank	n_accepted	821	0.204	0.4
out_strength	user_reputation	821	0.253	0.7
out_strength	n_accepted	821	0.206	0.95
betweenness	user_reputation	821	0.407	0.7

metric	compared_with	n_helpers	spearman_rho	top20_overlap
betweenness	n_accepted	821	0.309	0.85
core_number	user_reputation	821	0.369	0.5
core_number	n_accepted	821	0.346	0.7

Table 11. Network by year (outputs/network_over_time.csv)

year	active_users	edges	interactions	reciprocity	modularity	n_helpers	help_share_top10pct
2019	345	510	885	0.043	0.513	69	0.96
2020	1180	1805	2871	0.038	0.569	354	0.912
2021	522	702	1115	0.043	0.601	144	0.918
2022	472	506	682	0.028	0.651	148	0.852
2023	442	496	787	0.024	0.616	135	0.884
2024	213	249	511	0.04	0.408	60	0.924
2025	52	50	64	0.04	0.582	16	0.812
2026	18	13	15	0.0	0.731	7	0.267

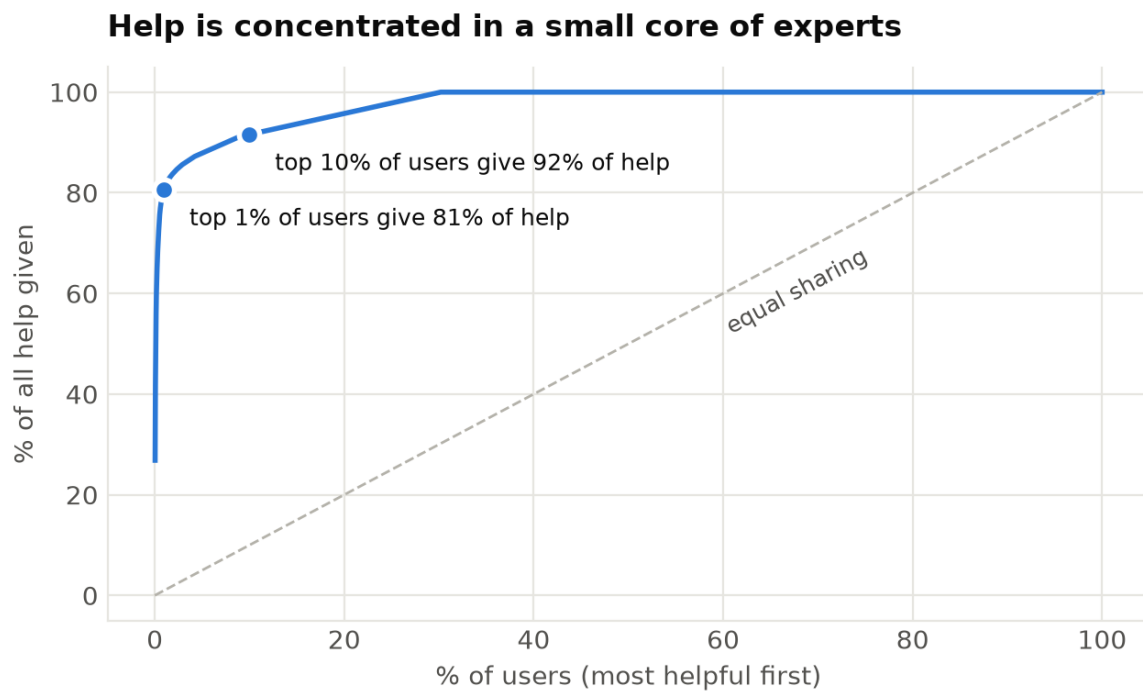


Figure 5. Help concentration

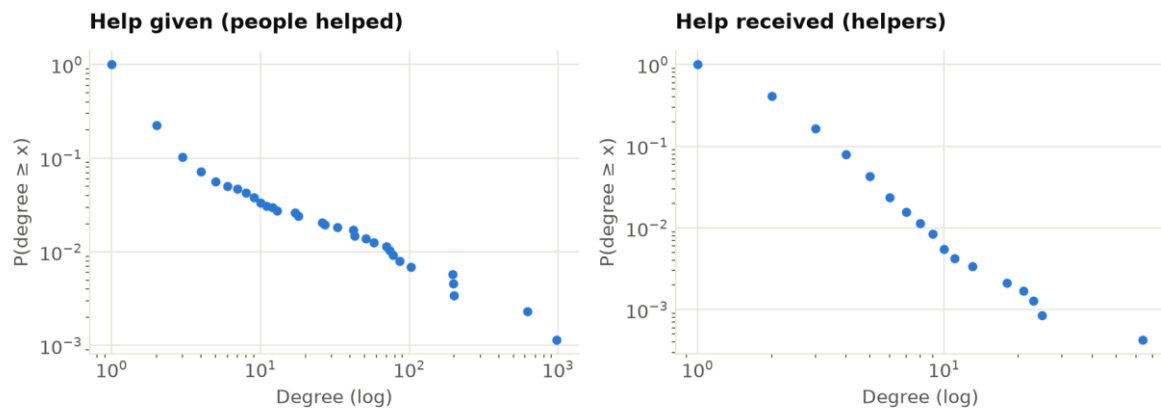
Heavy-tailed degree distributions: a few hubs, many one-off learners

Figure 6. Degree distributions (log scale)

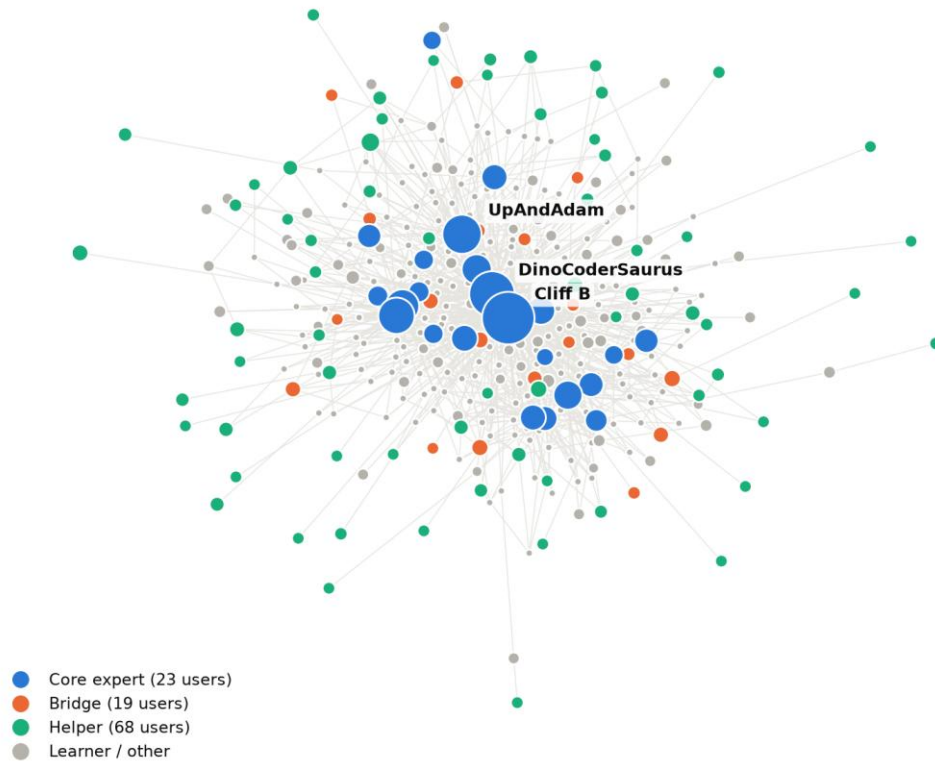
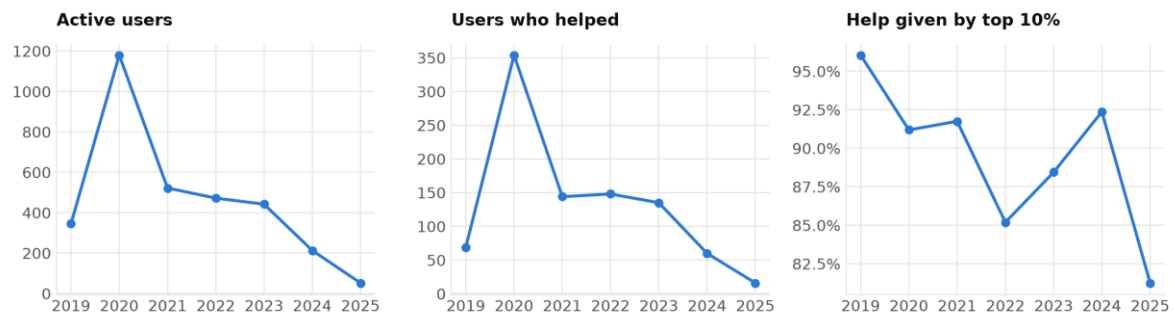
Knowledge network core: top 150 helpers and the learners they help (node size = help given)

Figure 7. Core of the knowledge network, coloured by role

The community peaked in 2020 and shrank afterwards**Figure 8.** *The network over time*

8. Knowledge Diffusion (diffusion.py)

Aim: test whether knowledge spreads through contact, and simulate diffusion. The **exposure test** compares how often adopters of a topic had earlier contact with an adopter against 200 random shuffles of the adoption times, and combines the topics with Stouffer's method. The **learner → helper** test uses a chi-square test. The **Independent Cascade** and **Linear Threshold** models are simulated with Monte Carlo runs, seeds are chosen by CELF greedy and compared with centrality rankings, and the simulation is validated against observed time-respecting reach.

Run: `python src/diffusion.py` (about 6 minutes)

Source code: `src/diffusion.py` (364 lines)

```

1  """
2  Step 4: knowledge diffusion – how topics and help spread through the community.
3
4  A. Observed diffusion
5      - Topic adoption curves: when each user first takes part in a topic.
6      - Social exposure test: were adopters more often in contact with earlier adopters than
7        chance predicts? (time-shuffle test, Anagnostopoulos et al., 2008)
8      - Learner → helper transition: do learners who receive help go on to help others?
9      - Thread cascades: how large the discussion grows around each question.
10 B. Simulated diffusion on the help network
11     - Independent Cascade (IC,  $p = 1 - (1 - \beta)^{\text{interactions}}$ ) and Linear Threshold (LT)
12       models (Kempe et al., 2003).
13     - Influence maximisation with CELF greedy vs. centrality heuristics and random seeds.
14     - Validation: does simulated influence match observed time-respecting reach?
15 """
16
17 import json
18
19 import numpy as np
20 import pandas as pd
21 from scipy.stats import spearmanr
22
23 from config import OUTPUT_DIR, PROCESSED_DIR, RANDOM_SEED
24 from data_loader import help_interactions, load_raw
25
26 N_SHUFFLES = 200
27 MC_RUNS_SELECT = 200      # Monte Carlo runs while choosing seeds
28 MC_RUNS_EVAL = 300       # Monte Carlo runs when reporting spread
29 N_CANDIDATES = 150       # seed candidates: most active helpers
30 MAX_SEEDS = 10
31 IC_BETA = 0.1            # IC: chance that one interaction passes the knowledge on
32
33 rng = np.random.default_rng(RANDOM_SEED)
34
35
36 # ===== A. observed
37
38 def participation_events(raw, qt):
39     """(user, topic, time, kind): asking a question or answering one in a topic."""
40     q = raw[["questions"]][["post_id", "user_id", "created_at"]].merge(qt[["post_id", "topic"]],
41 on="post_id")
42     a = raw[["answers"]][["question_id", "user_id", "created_at"]].merge(
43 qt[["post_id", "topic"]], left_on="question_id", right_on="post_id")
44     ev = pd.concat([q.assign(kind="ask")[["user_id", "topic", "created_at", "kind"]],
45 a.assign(kind="help")[["user_id", "topic", "created_at", "kind"]]])
46     ev = ev.dropna(subset=["user_id"])
47     ev["user_id"] = ev["user_id"].astype("int64")
48     return ev
49
50 def contact_arrays(inter, user_index):
51     """Every interaction in both directions as parallel arrays (u, v, time in days)."""
52     s = inter["source_user"].map(user_index).to_numpy()
53     t = inter["target_user"].map(user_index).to_numpy()
54     ts = (inter["created_at"].astype("int64") // 10**9 / 86400).to_numpy()
55     ok = ~(np.isnan(s.astype(float)) | np.isnan(t.astype(float)))
56     s, t, ts = s[ok].astype(int), t[ok].astype(int), ts[ok]
57     return np.concatenate([s, t]), np.concatenate([t, s]), np.concatenate([ts, ts])
58
59
60 def exposed_share(adopt_time, adopters, cu, cv, ct):
61     """Share of adopters who, before adopting, had contact with someone who adopted earlier."""

```

```

62     hit = (adopt_time[cv] < ct) & (ct < adopt_time[cu])
63     exposed = np.intersect1d(np.unique(cu[hit]), adopters)
64     first = adopters[np.argmax(adopt_time[adopters])]
65     return len(np.setdiff1d(exposed, [first])) / max(len(adopters) - 1, 1)
66
67
68 def exposure_test(events, inter, topics):
69     users = np.unique(np.concatenate([events["user_id"].unique(),
70                                     inter["source_user"].unique(), inter["target_user"].unique()]))
71     user_index = pd.Series(np.arange(len(users)), index=users)
72     cu, cv, ct = contact_arrays(inter, user_index)
73     first = events.groupby(["topic", "user_id"])["created_at"].min().reset_index()
74     first["day"] = first["created_at"].astype("int64") // 10**9 / 86400
75
76     rows = []
77     for topic, grp in first.groupby("topic"):
78         if len(grp) < 30:
79             continue
80         idx = user_index[grp["user_id"]].to_numpy()
81         adopt = np.full(len(users), np.inf)
82         adopt[idx] = grp["day"].to_numpy()
83         observed = exposed_share(adopt, idx, cu, cv, ct)
84         null = []
85         for _ in range(N_SHUFFLES):
86             shuffled = np.full(len(users), np.inf)
87             shuffled[idx] = rng.permutation(grp["day"].to_numpy())
88             null.append(exposed_share(shuffled, idx, cu, cv, ct))
89         null = np.array(null)
90         rows.append({
91             "topic": topic, "n_adopters": len(grp), "observed_exposed_share": observed,
92             "null_mean": null.mean(), "null_std": null.std(),
93             "z_score": (observed - null.mean()) / (null.std() or np.nan),
94             "p_value": (np.sum(null >= observed) + 1) / (N_SHUFFLES + 1),
95         })
96     out = pd.DataFrame(rows)
97     return out.merge(topics[["topic", "label"]], on="topic", how="left") if len(out) else out
98
99
100 def combine_exposure(exposure: pd.DataFrame) -> dict:
101     """Stouffer's method: one overall z-score from the per-topic tests."""
102     from scipy.stats import norm
103     z = exposure["z_score"].replace([np.inf, -np.inf], np.nan).dropna()
104     z_all = float(z.sum() / np.sqrt(len(z)))
105     return {"exposure_stouffer_z": z_all, "exposure_stouffer_p": float(norm.sf(z_all)),
106           "exposure_significant_topics": exposure.loc[exposure["p_value"] < 0.05,
107           "label"].tolist()}
108
109 def adoption_curves(events, communities):
110     first = events.groupby(["topic", "user_id"])["created_at"].min().reset_index()
111     first["period"] = first["created_at"].dt.to_period("Q").astype(str)
112     first["community"] = first["user_id"].map(communities)
113     curve = first.groupby(["topic", "period"]).agg(
114         new_adopters=("user_id", "size"),
115         communities_reached=("community", lambda s: s.dropna().nunique()).reset_index()
116     )
117     curve["cumulative_adopters"] = curve.groupby("topic")["new_adopters"].cumsum()
118     curve["share_of_final"] = curve["cumulative_adopters"] /
119     curve.groupby("topic")["new_adopters"].transform("sum")
120     return curve
121
122 def learner_to_helper(row):
123     """Do learners whose first question got answered later become helpers themselves?"""
124     q = row["questions"].dropna(subset=["user_id"]).sort_values("created_at")
125     a = row["answers"].dropna(subset=["user_id"])
126     first_q = q.groupby("user_id").first()[["post_id", "created_at"]]
127     answered_ids = set(a["question_id"])
128     accepted_ids = set(q.loc[q["accepted_answer_id"].notna(), "post_id"])
129     first_q["got_answer"] = first_q["post_id"].isin(answered_ids)
130     first_q["got_accepted"] = first_q["post_id"].isin(accepted_ids)
131     asker = q.set_index("post_id")["user_id"]
132     a = a[a["question_id"].map(asker) != a["user_id"]] # answering your own question is not helping
133     # others
134     first_help = a.groupby("user_id")["created_at"].min()
135     first_q["first_help_at"] = first_help.reindex(first_q.index)
136     first_q["became_helper"] = first_q["first_help_at"] > first_q["created_at"]
137     first_q["days_to_helper"] = (first_q["first_help_at"] - first_q["created_at"]).dt.days
138
139     rate = first_q.groupby("got_answer")["became_helper"].mean()
140     table = pd.crosstab(first_q["got_answer"], first_q["became_helper"])
141     from scipy.stats import chi2_contingency
142     chi2, p, _, _ = chi2_contingency(table) if table.shape == (2, 2) else (np.nan, np.nan, None,
143     None)

```

```

141     helpers = first_q[first_q["became_helper"]]
142     return {
143         "n_learners": int(len(first_q)),
144         "n_became_helpers": int(first_q["became_helper"].sum()),
145         "pct_became_helpers": float(first_q["became_helper"].mean()),
146         "pct_helper_if_first_question_answered": float(rate.get(True, np.nan)),
147         "pct_helper_if_first_question_unanswered": float(rate.get(False, np.nan)),
148         "chi2": float(chi2), "p_value": float(p),
149         "median_days_to_become_helper": float(helpers["days_to_helper"].median()) if len(helpers)
150     }
151 else None,
152 }
153
154 def thread_cascades(raw, qt):
155     """Size of the discussion around each question (answers + comments on the thread)."""
156     q, a, c = raw["questions"], raw["answers"], raw["comments"]
157     post_to_q = pd.concat([q[["post_id"]].assign(question_id=q["post_id"]),
158                           a[["post_id", "question_id"]]])
159     cm = c.merge(post_to_q, on="post_id")
160     size = (a.groupby("question_id").size().rename("n_answers").to_frame()
161            .join(cm.groupby("question_id").size().rename("n_comments"), how="outer").fillna(0))
162     people = pd.concat([a[["question_id", "user_id"]], cm[["question_id", "user_id"]]])
163     size["n_participants"] = people.dropna().groupby("question_id")["user_id"].nunique()
164     size = size.fillna(0)
165     size["cascade_size"] = size["n_answers"] + size["n_comments"]
166     size = size.reset_index().rename(columns={"index": "question_id"})
167     size = q[["post_id"]].merge(size, left_on="post_id", right_on="question_id",
168                               how="left").fillna(0)
169     size = size.merge(qt[["post_id", "topic"]], on="post_id", how="left")
170     size = size.dropna(subset=["topic"]).astype({"topic": int})
171     per_topic = size.groupby("topic").agg(
172         n_threads=("post_id", "size", mean_cascade_size=("cascade_size", "mean"),
173         max_cascade_size=("cascade_size", "max"), mean_participants=("n_participants",
174         "mean")).reset_index()
175     distribution = size["cascade_size"].value_counts().sort_index().rename("n_threads").reset_index()
176     return per_topic, distribution
177
178 # ===== B. simulated
179
180 class HelpNetwork:
181     """Compact adjacency lists for fast Monte Carlo simulation (knowledge flows helper ->
182     learner)."""
183
184     def __init__(self, edges: pd.DataFrame):
185         nodes = pd.Index(pd.unique(pd.concat([edges.source_user, edges.target_user])))
186         self.nodes = nodes
187         self.n = len(nodes)
188         src = nodes.get_indexer(edges.source_user)
189         dst = nodes.get_indexer(edges.target_user)
190         w = edges["weight"].to_numpy(float)
191         in_strength = np.bincount(dst, weights=w, minlength=self.n)
192         # IC: each of the w interactions independently passes knowledge with probability beta.
193         p_ic = 1 - (1 - IC_BETA) ** w
194         # LT: influence weights w_uv / in_strength(v), which sum to 1 over v's helpers.
195         b_lt = w / in_strength[dst]
196         order = np.argsort(src, kind="stable")
197         self.src, self.dst = src[order], dst[order]
198         self.p_ic, self.b_lt = p_ic[order], b_lt[order]
199         self.ptr = np.searchsorted(self.src, np.arange(self.n + 1))
200
201     def out(self, u, weights):
202         a, b = self.ptr[u], self.ptr[u + 1]
203         return self.dst[a:b], weights[a:b]
204
205     def ic(self, seeds, runs):
206         total = 0
207         for _ in range(runs):
208             active = np.zeros(self.n, bool)
209             active[seeds] = True
210             frontier = list(seeds)
211             while frontier:
212                 nxt = []
213                 for u in frontier:
214                     v, p = self.out(u, self.p_ic)
215                     hit = v[(~active[v]) & (rng.random(len(v)) < p)]
216                     active[hit] = True
217                     nxt.extend(hit.tolist())
218                 frontier = nxt
219             total += active.sum()
220         return total / runs
221
222     def lt(self, seeds, runs):

```

```

220     total = 0
221     for _ in range(runs):
222         threshold = rng.random(self.n)
223         pressure = np.zeros(self.n)
224         active = np.zeros(self.n, bool)
225         active[seeds] = True
226         frontier = list(seeds)
227         while frontier:
228             nxt = []
229             for u in frontier:
230                 v, p = self.out(u, self.b_lt)
231                 pressure[v] += p
232                 new = v[(~active[v]) & (pressure[v] >= threshold[v])]
233                 active[new] = True
234                 nxt.extend(new.tolist())
235             frontier = nxt
236         total += active.sum()
237     return total / runs
238
239
240 def celf(net: HelpNetwork, candidates, k, model="ic"):
241     """CELF lazy-greedy influence maximisation (Leskovec et al., 2007)."""
242     sim = net.ic if model == "ic" else net.lt
243     gains = sorted((sim([c], MC_RUNS_SELECT), c) for c in candidates), reverse=True)
244     seeds, spread = [gains[0][1]], gains[0][0]
245     gains = gains[1:]
246     while len(seeds) < k and gains:
247         while True:
248             _, c = gains.pop(0)
249             gain = sim(seeds + [c], MC_RUNS_SELECT) - spread
250             if not gains or gain >= gains[0][0]:
251                 seeds.append(c)
252                 spread += gain
253                 break
254             gains.append((gain, c))
255         gains.sort(reverse=True)
256     return seeds
257
258
259 def observed_reach(inter, sources):
260     """Users reachable from each source by time-respecting helper -> learner paths."""
261     ev = inter.sort_values("created_at")
262     s, t = ev["source_user"].to_numpy(), ev["target_user"].to_numpy()
263     ts = ev["created_at"].astype("int64").to_numpy()
264     reach = {}
265     for src in sources:
266         arrival = {src: -np.inf}
267         for a, b, time in zip(s, t, ts):
268             if a in arrival and arrival[a] <= time and b not in arrival:
269                 arrival[b] = time
270         reach[src] = len(arrival) - 1
271     return pd.Series(reach)
272
273
274 def run():
275     raw = load_raw()
276     qt = pd.read_parquet(PROCESSED_DIR / "question_topics.parquet")
277     topics = pd.read_csv(OUTPUT_DIR / "topics.csv")
278     metrics = pd.read_csv(OUTPUT_DIR / "user_metrics.csv").set_index("user_id")
279     edges = pd.read_csv(OUTPUT_DIR / "help_edges.csv")
280     # Any contact (either direction) can expose a user to a topic; only help carries knowledge on.
281     contacts = raw["interactions"].dropna(subset=["source_user", "target_user"]).copy()
282     contacts["source_user"] = contacts["source_user"].astype("int64")
283     contacts["target_user"] = contacts["target_user"].astype("int64")
284     inter = help_interactions(raw)
285     summary = {}
286
287     # ----- A. observed
288     events = participation_events(raw, qt)
289     exposure = exposure_test(events, contacts, topics)
290     exposure.round(4).to_csv(OUTPUT_DIR / "diffusion_exposure_test.csv", index=False)
291     summary["exposure_topics_tested"] = int(len(exposure))
292     summary["exposure_topics_significant"] = int((exposure["p_value"] < 0.05).sum())
293     summary["exposure_mean_observed"] = float(exposure["observed_exposed_share"].mean())
294     summary["exposure_mean_null"] = float(exposure["null_mean"].mean())
295     summary.update(combine_exposure(exposure))
296     print(f"[diffusion] exposure test: {summary['exposure_topics_significant']}/{len(exposure)}")
297
298     topics = "
299         f"show more contact with earlier adopters than chance")
300     adoption_curves(events, metrics["community"]).to_csv(OUTPUT_DIR / "topic_adoption_curves.csv",
301 index=False)
302     summary["learner_to_helper"] = learner_to_helper(raw)

```

```

301     per_topic, dist = thread_cascades(raw, qt)
302     per_topic.round(3).to_csv(OUTPUT_DIR / "cascade_by_topic.csv", index=False)
303     dist.to_csv(OUTPUT_DIR / "cascade_size_distribution.csv", index=False)
304
305     # ----- B. simulated
306     net = HelpNetwork(edges)
307     idx = pd.Series(np.arange(net.n), index=net.nodes)
308     in_net = metrics.index.intersection(net.nodes)
309     candidates_ids = metrics.loc[in_net].sort_values("out_strength",
ascending=False).index[:N_CANDIDATES]
310     candidates = idx[candidates_ids].tolist()
311
312     strategies = {}
313     for name, col in [("Top out-degree", "out_strength"), ("Top ExpertiseRank", "expertise_rank"),
314                     ("Top betweenness", "betweenness"), ("Top PageRank", "pagerank")]:
315         strategies[name] = idx[metrics.loc[in_net].sort_values(col,
ascending=False).index[:MAX_SEEDS]].tolist()
316
317     rows, seed_sets = [], {}
318     for model in ["ic", "lt"]:
319         print(f"[diffusion] influence maximisation ({model.upper()}) ...")
320         model_strategies = dict(strategies)
321         model_strategies["CELF greedy"] = celf(net, candidates, MAX_SEEDS, model)
322         sim = net.ic if model == "ic" else net.lt
323         for name, seeds in model_strategies.items():
324             for k in range(1, MAX_SEEDS + 1):
325                 rows.append({"model": model.upper(), "strategy": name, "k": k, "spread":
sim(seeds[:k], MC_RUNS_EVAL)})
326                 random_spread = np.zeros(MAX_SEEDS)
327                 for _ in range(20):
328                     seeds = rng.choice(net.n, MAX_SEEDS, replace=False).tolist()
329                     random_spread += [sim(seeds[:k], MC_RUNS_EVAL // 10) for k in range(1, MAX_SEEDS + 1)]
330                 rows += [{"model": model.upper(), "strategy": "Random", "k": k + 1, "spread": s / 20}
331                         for k, s in enumerate(random_spread)]
332                 seed_sets[model.upper()] = {name: [int(net.nodes[i]) for i in s] for name, s in
model_strategies.items()}
333
334     im = pd.DataFrame(rows)
335     im["spread_pct_of_users"] = 100 * im["spread"] / net.n
336     im.round(3).to_csv(OUTPUT_DIR / "influence_maximization.csv", index=False)
337     with open(OUTPUT_DIR / "seed_sets.json", "w") as f:
338         json.dump(seed_sets, f, indent=2)
339
340     # ----- validation: simulated vs observed
341     print("[diffusion] validating simulated influence against observed reach ...")
342     sim_single = pd.Series({net.nodes[c]: net.ic[c], MC_RUNS_SELECT) for c in candidates})
343     obs = observed_reach(inter, sim_single.index)
344     valid = pd.DataFrame({"simulated_ic_spread": sim_single, "observed_reach": obs})
345     valid = valid.join(metrics[["user_name", "out_strength", "expertise_rank", "betweenness"]])
346     valid.index.name = "user_id"
347     valid.reset_index().round(4).to_csv(OUTPUT_DIR / "diffusion_validation.csv", index=False)
348     summary["validation"] = {
349         col: float(spearmanr(valid[col], valid["observed_reach"])[0])
350         for col in ["simulated_ic_spread", "out_strength", "expertise_rank", "betweenness"]}
351
352     k10 = im[im.k == MAX_SEEDS].set_index(["model", "strategy"])["spread"]
353     summary["spread_at_k10"] = {f"{m}{s}": float(v) for (m, s), v in k10.items()}
354     summary["network_users"] = int(net.n)
355
356     with open(OUTPUT_DIR / "diffusion_summary.json", "w") as f:
357         json.dump(summary, f, indent=2, default=float)
358     print(f"[diffusion] simulated influence vs observed reach: Spearman rho = "
359           f"{summary['validation']['simulated_ic_spread']:.2f}")
360     return summary
361
362
363 if __name__ == "__main__":
364     run()

```

Output

```

=== diffusion: diffusion tests and IC/LT simulation (~8 min)
[diffusion] exposure test: 3/19 topics show more contact with earlier adopters than chance
[diffusion] influence maximisation (IC) ...
[diffusion] influence maximisation (LT) ...
[diffusion] validating simulated influence against observed reach ...
[diffusion] simulated influence vs observed reach: Spearman rho = 0.86
=== diffusion done in 336s

```

Table 12. *Social exposure test per topic (outputs/diffusion_exposure_test.csv)*

label	n_adopters	observed_exposed_share	null_mean	z_score	p_value
sql / movie / query	79	0.487	0.294	2.766	0.005
dna / sequence / str	116	0.487	0.293	3.513	0.005
buy / finance / stock	197	0.424	0.276	1.673	0.03
register / username / register.html	77	0.447	0.315	1.568	0.06
import / sql / database	86	0.329	0.244	1.508	0.09
speller / word / dictionary	496	0.346	0.256	1.187	0.144
pair / tideman / sort	96	0.295	0.259	0.916	0.224
identifier / int / lecture	128	0.339	0.292	0.726	0.264
html / flask / form	139	0.239	0.216	0.451	0.338
loop / array / string	191	0.295	0.281	0.267	0.383
resize / bmp / resize less	86	0.306	0.288	0.191	0.438
recover / pset4 recover / pset4	296	0.271	0.266	0.087	0.453
caesar / key / character	335	0.254	0.257	-0.047	0.502
readability / grade / sentence	187	0.296	0.291	0.081	0.517
filter / blur / pixel	319	0.255	0.26	-0.103	0.517
vote / candidate / winner	210	0.278	0.283	-0.128	0.567
jpg / recover / image	96	0.263	0.279	-0.19	0.592
mario / ide / project	831	0.17	0.23	-1.369	0.93
credit / cash / cent	256	0.188	0.268	-1.402	0.94

Table 13. *Diffusion summary (outputs/diffusion_summary.json)*

Measure	Value
n_learners	2374
n_became_helpers	125
pct_became_helpers	0.0527
pct_helper_if_first_question_answered	0.0519
pct_helper_if_first_question_unanswered	0.0599
chi2	0.1173
p_value	0.732
median_days_to_become_helper	4
exposure_stouffer_z	2.683
exposure_stouffer_p	0.0036
validation_spearman_simulated_ic	0.8576

Table 14. *Share of users reached with 10 seed users (outputs/influence_maximization.csv)*

model	strategy	spread	spread_pct_of_users
IC	Top out-degree	415.75	14.38
IC	Top ExpertiseRank	415.01	14.36
IC	Top betweenness	418.3	14.47
IC	Top PageRank	391.06	13.53
IC	CELF greedy	421.56	14.58
IC	Random	22.26	0.77
LT	Top out-degree	1960.54	67.82
LT	Top ExpertiseRank	1969.0	68.11
LT	Top betweenness	1971.49	68.19
LT	Top PageRank	1866.03	64.55

model	strategy	spread	spread_pct_of_users
LT	CELF greedy	1953.48	67.57
LT	Random	26.8	0.93

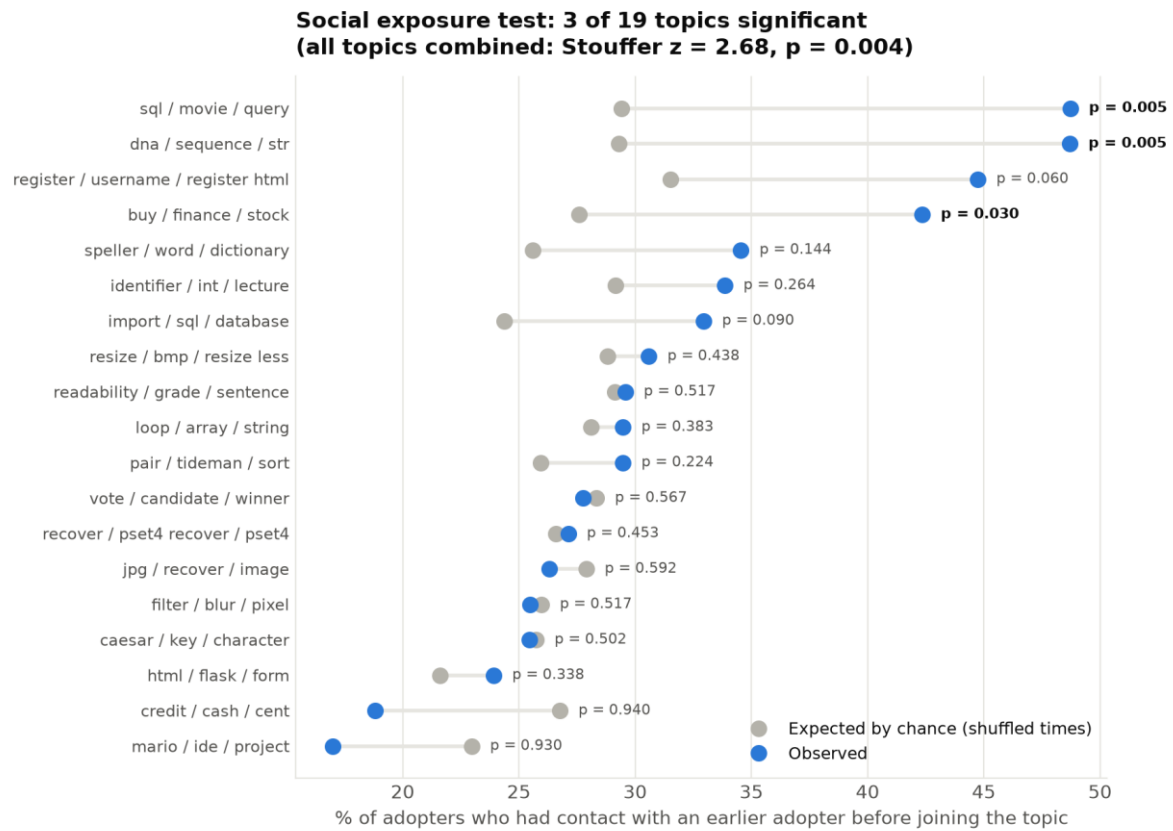
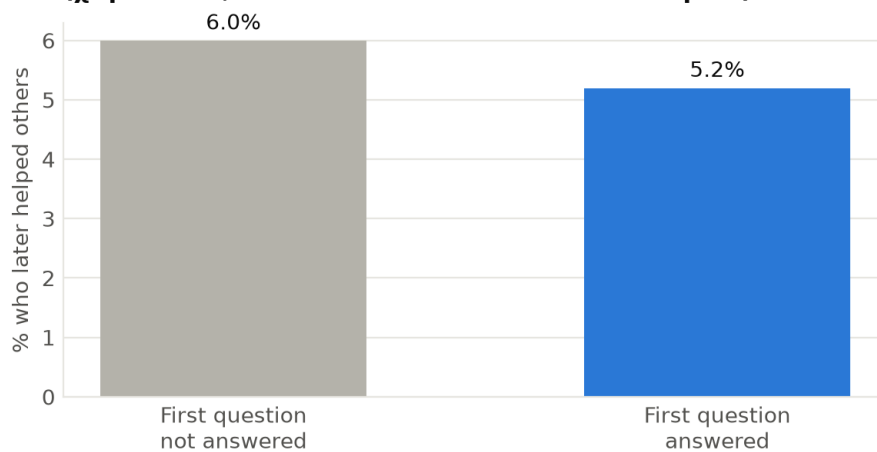
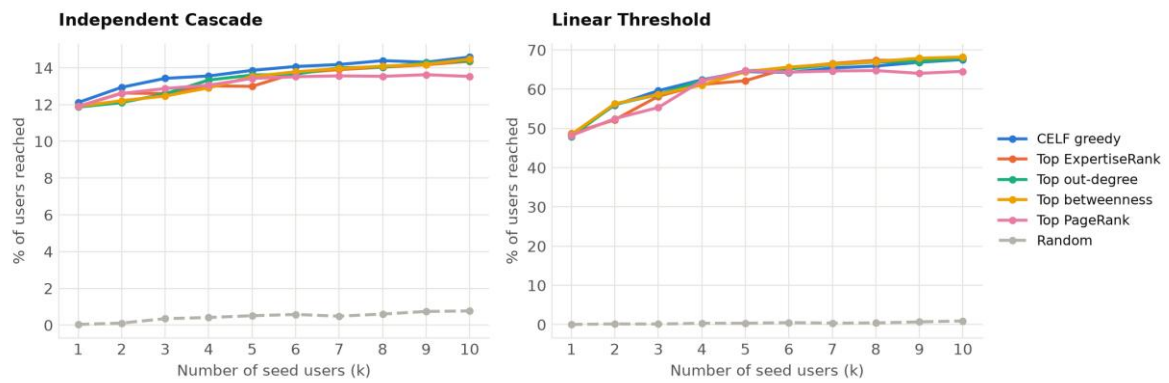
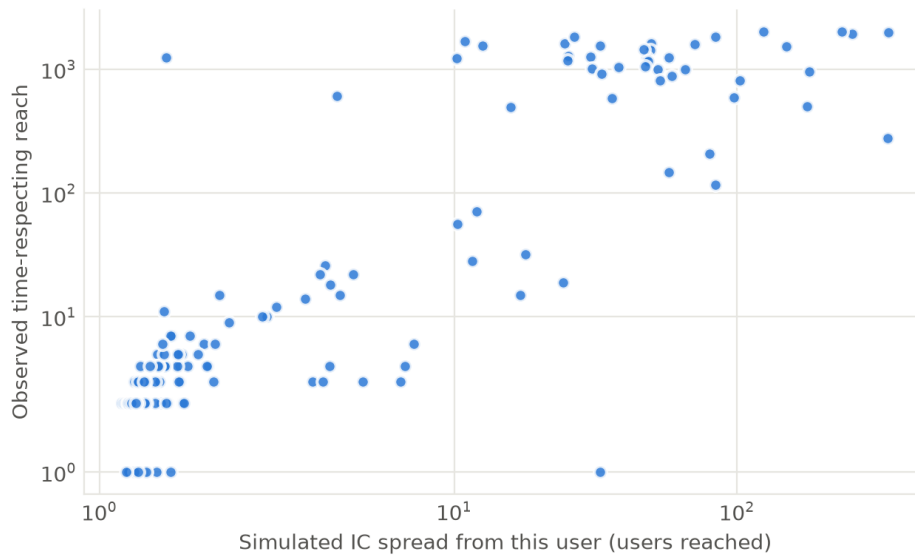


Figure 9. Social exposure test

**Getting an answer does not change who becomes a helper
(χ^2 $p = 0.73$, 125 of 2374 learners became helpers)**

Figure 10. Learner $\rightarrow$ helper transition

Influence maximisation: which seed users spread knowledge furthest**Figure 11.** *Influence maximisation***Simulated vs observed diffusion
(Spearman $\rho = 0.86$, 150 most active helpers)****Figure 12.** *Simulated vs observed reach*

9. Answer Prediction (predict.py)

Aim: predict whether a new question will be answered, using only information available when it is posted. The data is split in time (oldest 80% for training, newest 20% for testing). The program compares a majority-class baseline, Logistic Regression, Random Forest and XGBoost, runs 5-fold cross-validation on the training period, and measures permutation feature importance.

Run: `python src/predict.py`

Source code: `src/predict.py` (164 lines)

```

1  """
2  Step 5: predict whether a new question will get a good answer (Stack Exchange's `is_answered`).
3
4  Only information available at the moment a question is posted is used (no scores, views
5  or answer counts), and the evaluation is temporal: train on the oldest 80% of questions,
6  test on the newest 20%, the way the model would be used in practice.
7
8  Outputs:
9      outputs/prediction_metrics.csv      model comparison on the held-out future questions
10     outputs/prediction_cv.csv          5-fold cross-validation on the training period
11     outputs/prediction_importance.csv   permutation importance of each feature
12     outputs/prediction_roc.csv         ROC curve points for every model
13     data/processed/prediction_features.parquet
14  """
15
16  import numpy as np
17  import pandas as pd
18  from sklearn.compose import ColumnTransformer
19  from sklearn.dummy import DummyClassifier
20  from sklearn.ensemble import RandomForestClassifier
21  from sklearn.inspection import permutation_importance
22  from sklearn.linear_model import LogisticRegression
23  from sklearn.metrics import (accuracy_score, average_precision_score, balanced_accuracy_score,
24                              f1_score, roc_auc_score, roc_curve)
25  from sklearn.model_selection import StratifiedKFold, cross_val_score
26  from sklearn.pipeline import Pipeline
27  from sklearn.preprocessing import OneHotEncoder, StandardScaler
28  from xgboost import XGBClassifier
29
30  from config import OUTPUT_DIR, PROCESSED_DIR, RANDOM_SEED, TRAIN_FRACTION
31  from data_loader import load_raw
32
33  TARGET = "is_answered"
34  NUMERIC = [
35      "n_words", "title_words", "n_code_lines", "n_question_marks", "n_tags", "sentiment",
36      "confusion_score", "has_code", "has_error_msg", "has_check50_fail",
37      "asker_prior_questions", "asker_prior_answers", "asker_is_new", "asker_prior_answered_rate",
38      "questions_last_30d", "helpers_last_30d", "topic_prior_answered_rate", "hour", "weekday",
39  ]
40  CATEGORICAL = ["topic", "course_unit"]
41
42
43  def count_before(times_by_user: dict, users, at) -> np.ndarray:
44      """For each (user, time): how many of the user's events happened strictly before time."""
45      out = np.zeros(len(users), dtype=int)
46      for i, (u, t) in enumerate(zip(users, at)):
47          arr = times_by_user.get(u)
48          if arr is not None:
49              out[i] = np.searchsorted(arr, t, side="left")
50      return out
51
52
53  def build_features(q: pd.DataFrame, answers: pd.DataFrame) -> pd.DataFrame:
54      q = q.sort_values("created_at").reset_index(drop=True).copy()
55      t = q["created_at"].astype("int64").to_numpy()
56      user = q["user_id"].fillna(-1).astype("int64").to_numpy()
57
58      # Asker's history before this question.
59      q["asker_prior_questions"] = q.groupby(user).cumcount()
60      q.loc[user == -1, "asker_prior_questions"] = 0
61      q["asker_is_new"] = (q["asker_prior_questions"] == 0).astype(int)
62      ans = answers.dropna(subset=["user_id"]).sort_values("created_at")
63      ans_times = {u: g["created_at"].astype("int64").to_numpy()
64                  for u, g in ans.groupby(ans["user_id"].astype("int64"))}
65      q["asker_prior_answers"] = count_before(ans_times, user, t)
66      prior_rate = q.groupby(user)[TARGET].transform(lambda s: s.shift().expanding().mean())

```

```

67     q["asker_prior_answered_rate"] = prior_rate.fillna(-1) # -1 = no earlier question
68
69     # How busy the community was in the 30 days before the question.
70     month = np.int64(30 * 86400 * 10**9)
71     q["questions_last_30d"] = np.searchsorted(t, t, side="left") - np.searchsorted(t, t - month,
side="left")
72     a_t = ans["created_at"].astype("int64").to_numpy()
73     a_u = ans["user_id"].astype("int64").to_numpy()
74     lo, hi = np.searchsorted(a_t, t - month), np.searchsorted(a_t, t)
75     q["helpers_last_30d"] = [len(np.unique(a_u[i:j])) for i, j in zip(lo, hi)]
76
77     # How often this topic's earlier questions got answered.
78     q["topic_prior_answered_rate"] = (q.groupby("topic")[TARGET]
79                                     .transform(lambda s: s.shift().expanding().mean()))
80     q["topic_prior_answered_rate"] = q["topic_prior_answered_rate"].fillna(q[TARGET].mean())
81
82     q["hour"] = q["created_at"].dt.hour
83     q["weekday"] = q["created_at"].dt.weekday
84     for col in ["has_code", "has_error_msg", "has_check50_fail"]:
85         q[col] = q[col].astype(int)
86     q["topic"] = q["topic"].astype(str)
87     q["course_unit"] = q["course_unit"].fillna("Unknown")
88     return q
89
90
91 def models():
92     pre_scaled = ColumnTransformer([
93         ("num", StandardScaler(), NUMERIC),
94         ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL)])
95     pre_trees = ColumnTransformer([
96         ("num", "passthrough", NUMERIC),
97         ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL)])
98     return {
99         "Baseline (majority class)": Pipeline([("pre", pre_trees), ("clf", DummyClassifier())]),
100         "Logistic Regression": Pipeline([("pre", pre_scaled), ("clf", LogisticRegression(
101             max_iter=2000, class_weight="balanced"))]),
102         "Random Forest": Pipeline([("pre", pre_trees), ("clf", RandomForestClassifier(
103             n_estimators=400, min_samples_leaf=5, class_weight="balanced", n_jobs=-1,
104             random_state=RANDOM_SEED))]),
105         "XGBoost": Pipeline([("pre", pre_trees), ("clf", XGBClassifier(
106             n_estimators=300, max_depth=4, learning_rate=0.05, subsample=0.8,
107             colsample_bytree=0.8, eval_metric="logloss", random_state=RANDOM_SEED, n_jobs=-1))]),
108     }
109
110
111 def run():
112     raw = load_raw()
113     q = pd.read_parquet(PROCESSED_DIR / "questions_clean.parquet")
114     qt = pd.read_parquet(PROCESSED_DIR / "question_topics.parquet")
115     q = q.merge(qt[["post_id", "topic", "course_unit"]], on="post_id", how="inner")
116     df = build_features(q, raw["answers"])
117     df.to_parquet(PROCESSED_DIR / "prediction_features.parquet", index=False)
118
119     cut = int(len(df) * TRAIN_FRACTION)
120     train, test = df.iloc[:cut], df.iloc[cut:]
121     X_train, y_train = train[NUMERIC + CATEGORICAL], train[TARGET].astype(int)
122     X_test, y_test = test[NUMERIC + CATEGORICAL], test[TARGET].astype(int)
123     print(f"[predict] train {len(train)} questions (until {train.created_at.max():%Y-%m}), "
124           f"test {len(test)} (answered rate {y_test.mean():.0%})")
125
126     rows, cv_rows, roc_rows, fitted = [], [], [], {}
127     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
128     for name, model in models().items():
129         cv_auc = cross_val_score(model, X_train, y_train, cv=cv, scoring="roc_auc")
130         cv_rows.append({"model": name, "cv_roc_auc_mean": cv_auc.mean(), "cv_roc_auc_std":
cv_auc.std()})
131         model.fit(X_train, y_train)
132         fitted[name] = model
133         prob = model.predict_proba(X_test)[: , 1]
134         pred = (prob >= 0.5).astype(int)
135         rows.append({
136             "model": name,
137             "roc_auc": roc_auc_score(y_test, prob),
138             "pr_auc": average_precision_score(y_test, prob),
139             "accuracy": accuracy_score(y_test, pred),
140             "balanced_accuracy": balanced_accuracy_score(y_test, pred),
141             "f1": f1_score(y_test, pred),
142         })
143         fpr, tpr, _ = roc_curve(y_test, prob)
144         roc_rows += [{"model": name, "fpr": a, "tpr": b} for a, b in zip(fpr, tpr)]
145         print(f" {name:28s} test ROC-AUC {rows[-1]['roc_auc']:.3f}   CV {cv_auc.mean():.3f}")
146
147     metrics = pd.DataFrame(rows).round(4)

```

```

148     metrics.to_csv(OUTPUT_DIR / "prediction_metrics.csv", index=False)
149     pd.DataFrame(cv_rows).round(4).to_csv(OUTPUT_DIR / "prediction_cv.csv", index=False)
150     pd.DataFrame(roc_rows).round(4).to_csv(OUTPUT_DIR / "prediction_roc.csv", index=False)
151
152     best = metrics.iloc[1:].sort_values("roc_auc", ascending=False).iloc[0]["model"]
153     imp = permutation_importance(fitted[best], X_test, y_test, scoring="roc_auc", n_repeats=20,
154                                random_state=RANDOM_SEED, n_jobs=-1)
155     importance = pd.DataFrame({"feature": NUMERIC + CATEGORICAL, "importance_mean":
imp.importances_mean,
156                               "importance_std": imp.importances_std, "model": best})
157     importance.sort_values("importance_mean", ascending=False).round(5).to_csv(
158         OUTPUT_DIR / "prediction_importance.csv", index=False)
159     print(f"[predict] best model: {best}")
160     return metrics
161
162
163 if __name__ == "__main__":
164     run()

```

Output

```

=== predict: answer prediction models (~1 min)
[predict] train 3168 questions (until 2022-08), test 792 (answered rate 59%)
  Baseline (majority class)   test ROC-AUC 0.500   CV 0.500
  Logistic Regression        test ROC-AUC 0.673   CV 0.592
  Random Forest              test ROC-AUC 0.675   CV 0.602
  XGBoost                    test ROC-AUC 0.656   CV 0.599
[predict] best model: Random Forest
=== predict done in 29s

```

Table 15. Test-set results (outputs/prediction_metrics.csv)

model	roc_auc	pr_auc	accuracy	balanced_accuracy	f1
Baseline (majority class)	0.5	0.591	0.591	0.5	0.743
Logistic Regression	0.673	0.725	0.624	0.626	0.658
Random Forest	0.675	0.725	0.61	0.629	0.613
XGBoost	0.656	0.715	0.639	0.6	0.727

Table 16. Cross-validation on the training period (outputs/prediction_cv.csv)

model	cv_roc_auc_mean	cv_roc_auc_std
Baseline (majority class)	0.5	0.0
Logistic Regression	0.592	0.02
Random Forest	0.602	0.013
XGBoost	0.598	0.015

Table 17. Top 10 features (outputs/prediction_importance.csv)

feature	importance_mean	importance_std
topic_prior_answered_rate	0.0238	0.0068
n_tags	0.02	0.0063
asker_prior_answered_rate	0.0172	0.0097
has_code	0.0106	0.0036
n_code_lines	0.0076	0.0072
topic	0.0074	0.0034
n_words	0.0073	0.0046
weekday	0.0054	0.0026
title_words	0.0052	0.0038
course_unit	0.0032	0.0031

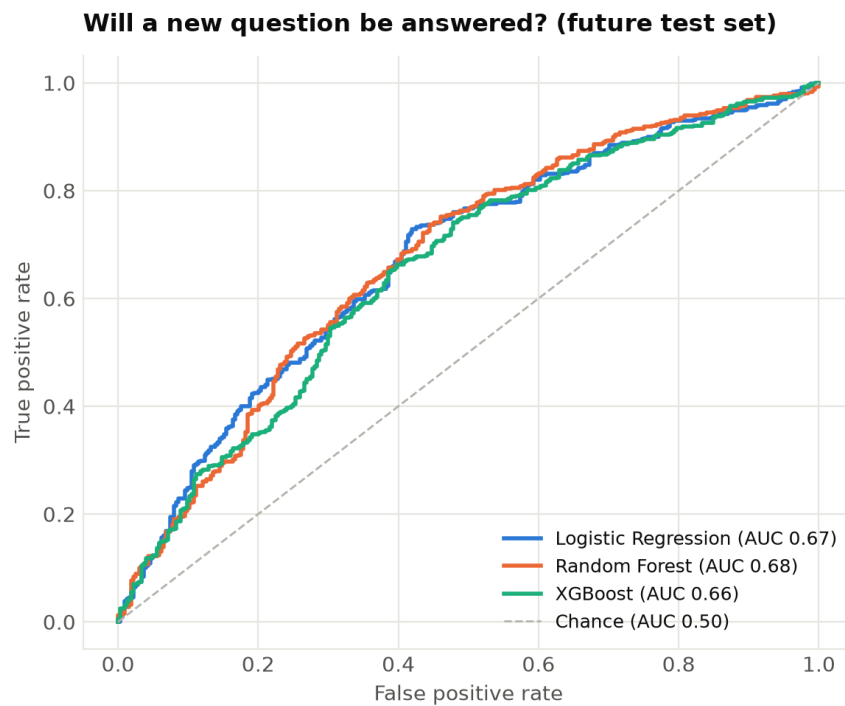


Figure 13. ROC curves

What predicts an answer (Random Forest, permutation importance)

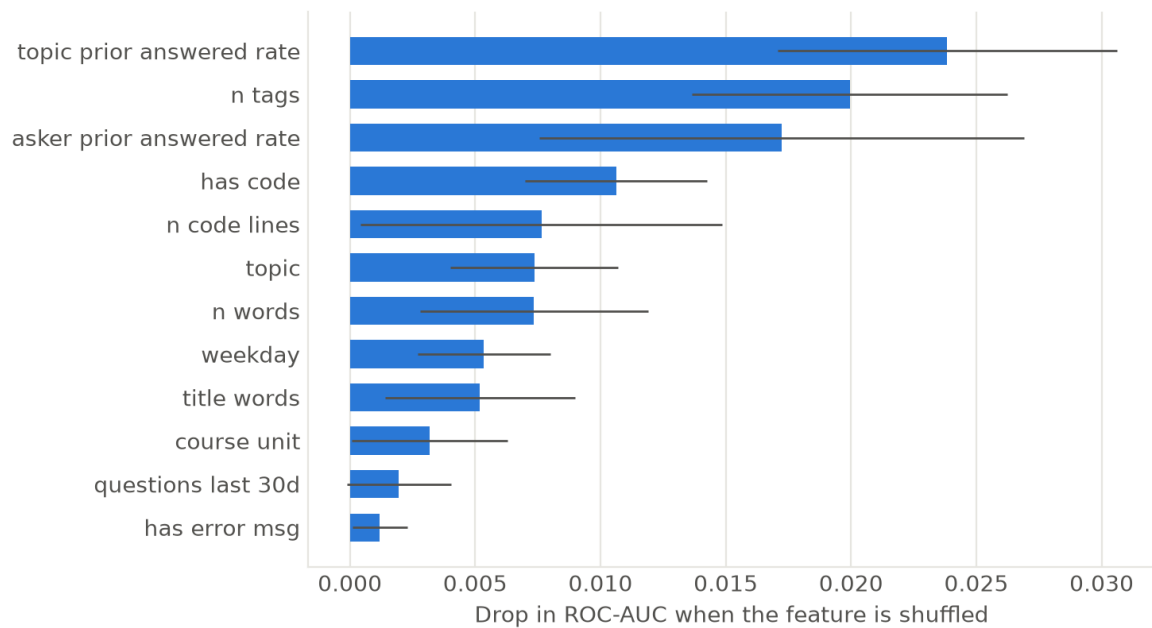


Figure 14. Permutation feature importance

10. Expert Finder (recommender.py)

Aim: recommend who can answer a new question. Similar past questions are found by cosine similarity (sentence embeddings or TF-IDF), each helper is scored on the similar questions they answered (accepted answers count double), and the hybrid score adds authority and recency. Evaluation on future questions reports Hit@1/5/10 and MRR.

Run: `python src/recommender.py`

Source code: `src/recommender.py` (144 lines)

```

1  """
2  Step 6: expert finder – recommend who can answer a new question.
3
4  For a new question we find similar past questions (by meaning) and score each helper by
5  how well they answered those questions, then blend in how active and how recently active
6  the helper is.
7
8  Evaluation (temporal): experts are learned from the oldest 80% of questions; for each newer
9  question we check whether a person who really answered it appears in the top-k list.
10 Metrics: Hit@k (share of questions where a real answerer is in the top k) and MRR.
11
12 Outputs: outputs/recommender_eval.csv
13 """
14
15 import numpy as np
16 import pandas as pd
17 from sklearn.feature_extraction.text import TfidfVectorizer
18 from sklearn.preprocessing import normalize
19
20 from config import EMBEDDING_MODEL, OUTPUT_DIR, PROCESSED_DIR, TRAIN_FRACTION
21 from data_loader import load_raw
22
23 N_SIMILAR = 50          # similar past questions considered per query
24 ACCEPTED_BONUS = 1.0    # an accepted answer counts double
25 RECENTY_DAYS = 365      # activity half-life style decay
26 HYBRID_WEIGHTS = {"content": 0.6, "authority": 0.2, "recency": 0.2}
27 KS = [1, 5, 10]
28
29
30 def _minmax(s: pd.Series) -> pd.Series:
31     rng = s.max() - s.min()
32     return (s - s.min()) / rng if rng > 0 else s * 0
33
34
35 class ExpertFinder:
36     """Content + authority + recency expert recommender over a set of past Q&A."""
37
38     def __init__(self, questions: pd.DataFrame, answers: pd.DataFrame, vectors: np.ndarray):
39         # questions: post_id, user_id, title, created_at (aligned row-by-row with vectors)
40         self.questions = questions.reset_index(drop=True)
41         self.vectors = normalize(vectors)
42         a = answers.dropna(subset=["user_id"]).copy()
43         a["user_id"] = a["user_id"].astype("int64")
44         asker = self.questions.set_index("post_id")["user_id"]
45         a = a[a["question_id"].isin(asker.index)]
46         a = a[a["question_id"].map(asker) != a["user_id"]] # ignore self-answers
47         a["credit"] = 1 + ACCEPTED_BONUS * a["is_accepted"].astype(float)
48         self.answers = a
49         row_of = pd.Series(np.arange(len(self.questions)), index=self.questions["post_id"])
50         self.answer_rows = row_of[a["question_id"]].to_numpy()
51         self.n_answers = a.groupby("user_id").size()
52         self.last_active = a.groupby("user_id")["created_at"].max()
53
54     def similar(self, query_vec: np.ndarray, n=N_SIMILAR):
55         sims = self.vectors @ normalize(query_vec.reshape(1, -1)).ravel()
56         top = np.argsort(sims)[::-1][:n]
57         return top, sims[top]
58
59     def score(self, query_vec, at=None, method="hybrid", exclude_user=None) -> pd.Series:
60         if method == "popularity":
61             scores = self.n_answers.astype(float)
62         else:
63             rows, sims = self.similar(query_vec)
64             sim_of_row = pd.Series(sims, index=rows)
65             hit = np.isin(self.answer_rows, rows)
66             part = self.answers[hit]

```

```

67         content = (part["credit"].to_numpy() * sim_of_row[self.answer_rows[hit]].to_numpy())
68         scores = pd.Series(content, index=part["user_id"].to_numpy()).groupby(level=0).sum()
69         if method == "hybrid":
70             users = self.n_answers.index
71             content = scores.reindex(users, fill_value=0)
72             authority = np.log1p(self.n_answers)
73             at = at if at is not None else self.last_active.max()
74             days = (at - self.last_active).dt.days.clip(lower=0)
75             recency = np.exp(-days / RECENCY_DAYS)
76             scores = (HYBRID_WEIGHTS["content"] * _minmax(content)
77                     + HYBRID_WEIGHTS["authority"] * _minmax(authority)
78                     + HYBRID_WEIGHTS["recency"] * _minmax(recency))
79         if exclude_user is not None:
80             scores = scores.drop(exclude_user, errors="ignore")
81         return scores.sort_values(ascending=False)
82
83
84     def load_vectors(kind: str, questions: pd.DataFrame, n_train=None):
85         """Embeddings from the topic step, or TF-IDF vectors (fitted on the first n_train rows)."""
86         if kind == "embedding":
87             emb = np.load(PROCESSED_DIR / "embeddings.npy")
88             ids = np.load(PROCESSED_DIR / "embedding_post_ids.npy")
89             idx = pd.Series(np.arange(len(ids)), index=ids)
90             return emb[idx[questions["post_id"]].to_numpy()], None
91         vec = TfidfVectorizer(token_pattern=r"\S+", ngram_range=(1, 2), min_df=2, sublinear_tf=True)
92         vec.fit(questions["tokens"].iloc[:n_train])
93         return vec.transform(questions["tokens"]).toarray().astype(np.float32), vec
94
95
96     def evaluate():
97         raw = load_raw()
98         q = pd.read_parquet(PROCESSED_DIR / "questions_clean.parquet")
99         ids = np.load(PROCESSED_DIR / "embedding_post_ids.npy")
100         q = q[q["post_id"].isin(ids)].sort_values("created_at").reset_index(drop=True)
101         cut = int(len(q) * TRAIN_FRACTION)
102         train, test = q.iloc[:cut], q.iloc[cut:]
103         answers = raw["answers"].dropna(subset=["user_id"]).copy()
104         answers["user_id"] = answers["user_id"].astype("int64")
105
106         truth =
answers[answers["question_id"].isin(test["post_id"])].groupby("question_id")["user_id"].agg(set)
107         known = set(answers.loc[answers["question_id"].isin(train["post_id"]), "user_id"])
108         test = test[test["post_id"].isin(truth.index)].copy()
109         test["truth"] = test["post_id"].map(truth).map(lambda s: s & known)
110         reachable = test["truth"].str.len() > 0
111         print(f"[recommender] {len(test)} answered test questions, "
112               f"{reachable.mean():.0%} answered by someone already active before the cut-off")
113         test = test[reachable]
114
115         rows = []
116         for kind in ["tfidf", "embedding"]:
117             vectors, _ = load_vectors(kind, q, n_train=cut)
118             finder = ExpertFinder(train, answers, vectors[:cut])
119             methods = ["popularity", "content", "hybrid"] if kind == "embedding" else ["content",
"hybrid"]
120             for method in methods:
121                 ranks = []
122                 for i, row in test.iterrows():
123                     asker = int(row.user_id) if pd.notna(row.user_id) else None
124                     ranking = finder.score(vectors[i], at=row.created_at, method=method,
exclude_user=asker)
125                     pos = [k for k, u in enumerate(ranking.index[:100], start=1) if u in row.truth]
126                     ranks.append(pos[0] if pos else np.inf)
127                 ranks = np.array(ranks)
128                 name = "Popularity (most answers)" if method == "popularity" else f"{method.title()}
({kind})"
129                 rec = {"method": name, "n_test_questions": len(ranks)}
130                 rec.update({f"hit@{k}": float(np.mean(ranks <= k)) for k in KS})
131                 rec["mrr"] = float(np.mean(np.where(np.isinf(ranks), 0, 1 / ranks)))
132                 rows.append(rec)
133                 print(f" {name:28s} Hit@5 {rec['hit@5']:.3f} Hit@10 {rec['hit@10']:.3f} MRR
{rec['mrr']:.3f}")
134         out = pd.DataFrame(rows).drop_duplicates("method").round(4)
135         out.to_csv(OUTPUT_DIR / "recommender_eval.csv", index=False)
136         return out
137
138
139     def run():
140         return evaluate()
141
142
143     if __name__ == "__main__":
144         run()

```

Output

```

=== recommender: expert recommender evaluation
[recommender] 698 answered test questions, 77% answered by someone already active before the cut-off
Content (tfidf)           Hit@5 0.689 Hit@10 0.710 MRR 0.596
Hybrid (tfidf)           Hit@5 0.717 Hit@10 0.798 MRR 0.628
Popularity (most answers) Hit@5 0.702 Hit@10 0.702 MRR 0.582
Content (embedding)      Hit@5 0.708 Hit@10 0.727 MRR 0.611
Hybrid (embedding)      Hit@5 0.725 Hit@10 0.811 MRR 0.637
=== recommender done in 13s

```

Table 18. Expert finder evaluation (outputs/recommender_eval.csv)

method	n_test_questions	hit@1	hit@5	hit@10	mrr
Content (tfidf)	534	0.524	0.689	0.71	0.596
Hybrid (tfidf)	534	0.528	0.717	0.798	0.628
Popularity (most answers)	534	0.472	0.702	0.702	0.582
Content (embedding)	534	0.53	0.708	0.727	0.61
Hybrid (embedding)	534	0.536	0.725	0.811	0.637

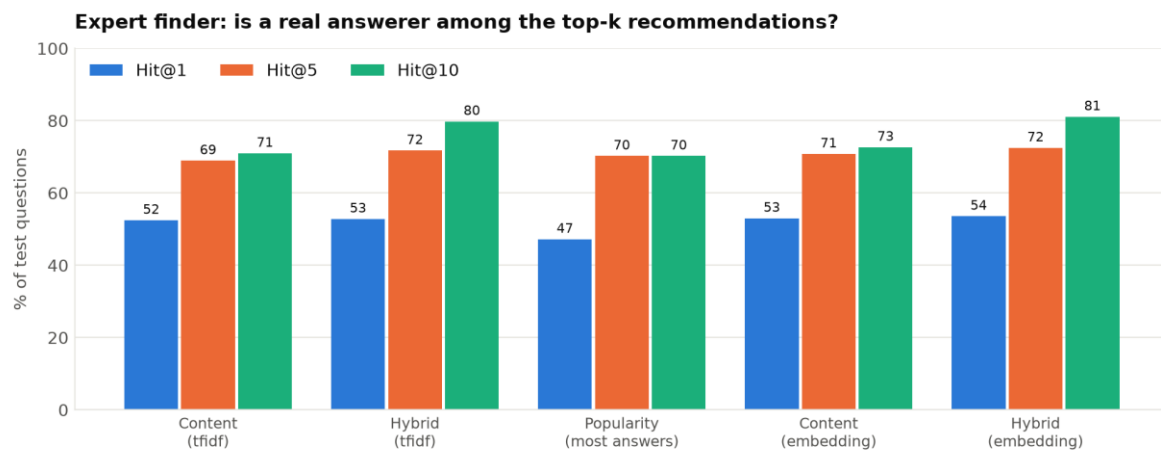


Figure 15. Hit@k of every method

11. Figures and the Pipeline Runner (figures.py, run_pipeline.py)

`figures.py` draws the 16 report figures shown in the previous chapters, using one colour-blind-safe palette. `run_pipeline.py` runs steps 3 to 9 in order with a single command and prints the time taken by each step.

Run: `python src/run_pipeline.py`

Source code: `src/figures.py` (374 lines)

```

1  """
2  Step 7: static figures for the report (PNG, 200 dpi) in outputs/figures/.
3
4  Colour roles follow one validated categorical order (never cycled); magnitude uses one
5  blue ramp, and signed values use a blue/red diverging pair around grey.
6  """
7
8  import json
9  import warnings
10
11  import matplotlib
12
13  matplotlib.use("Agg")
14  import matplotlib.pyplot as plt # noqa: E402
15  import networkx as nx # noqa: E402
16  import numpy as np # noqa: E402
17  import pandas as pd # noqa: E402
18
19  from config import FIGURE_DIR, OUTPUT_DIR, PROCESSED_DIR # noqa: E402
20
21  warnings.filterwarnings("ignore")
22
23  SERIES = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300", "#4a3aa7", "#e34948"]
24  OTHER = "#b4b2aa"
25  TEXT, TEXT_2, GRID, SURFACE = "#0b0b0b", "#52514e", "#e6e5e0", "#ffffff"
26  BLUE_RAMP = ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#256abf", "#184f95", "#0d366b"]
27  NEG, NEUTRAL, POS = "#2a78d6", "#f0efec", "#e34948"
28
29  plt.rcParams.update({
30      "figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SURFACE,
31      "font.family": "DejaVu Sans", "font.size": 10, "text.color": TEXT,
32      "axes.edgecolor": GRID, "axes.labelcolor": TEXT_2, "axes.titleweight": "bold",
33      "axes.titlesize": 12, "axes.titlelocation": "left", "axes.titlepad": 12,
34      "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
35      "grid.color": GRID, "grid.linewidth": 0.8, "axes.axisbelow": True,
36      "xtick.color": TEXT_2, "ytick.color": TEXT_2, "xtick.major.size": 0, "ytick.major.size": 0,
37      "legend.frameon": False, "lines.linewidth": 2,
38  })
39
40  def save(fig, name):
41      fig.tight_layout()
42      fig.savefig(FIGURE_DIR / f"{name}.png", dpi=200)
43      plt.close(fig)
44
45  def topic_names():
46
47      t = pd.read_csv(OUTPUT_DIR / "topics.csv")
48      return dict(zip(t["topic"], t["label"]))
49
50
51  # ----- community & topics
52
53  def fig_activity():
54      q = pd.read_parquet(PROCESSED_DIR / "questions_clean.parquet")
55      per = q.groupby(q["created_at"].dt.to_period("Q")).size()
56      fig, ax = plt.subplots(figsize=(9, 3.6))
57      ax.bar(per.index.astype(str), per.values, color=SERIES[0], width=0.8)
58      peak = per.idxmax()
59      ax.annotate(f"{per.max()} questions in {peak}\n(COVID-19 lockdown)",
60      xy=(list(per.index).index(peak), per.max()),
61      xytext=(10, -5), textcoords="offset points", color=TEXT_2, fontsize=9, va="top")
62      ax.set_title("Questions asked per quarter, CS50 Stack Exchange")
63      ax.set_ylabel("Questions")
64      ax.set_xticks(range(0, len(per), 4))
65      ax.set_xticklabels([str(p) for p in per.index[::4]])
66      ax.grid(axis="x", visible=False)
67      save(fig, "01_activity_over_time")

```

```

68
69
70 def fig_lda_selection():
71     s = pd.read_csv(OUTPUT_DIR / "lda_model_selection.csv")
72     fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
73     for ax, col, title in [(axes[0], "nmpi", "NPMI coherence (higher = better)",
74                             (axes[1], "diversity", "Topic diversity (higher = better)"))]:
75         ax.plot(s["k"], s[col], color=SERIES[0], marker="o", markersize=5)
76         ax.set_title(title, fontsize=11)
77         ax.set_xlabel("Number of LDA topics (k)")
78         ax.set_xticks(s["k"])
79     save(fig, "02_lda_model_selection")
80
81
82 def fig_model_comparison():
83     c = pd.read_csv(OUTPUT_DIR / "topic_model_comparison.csv")
84     metrics = [{"nmpi", "NPMI coherence"}, {"diversity", "Topic diversity"}, {"nmi_vs_course_tags",
85 "Agreement with\ncourse tags (NMI)"}]
86     x = np.arange(len(metrics))
87     fig, ax = plt.subplots(figsize=(8, 3.8))
88     w = 0.36
89     for i, (_, row) in enumerate(c.iterrows()):
90         vals = [row[m] for m, _ in metrics]
91         label = row["model"].split(" ")[0] + (f" ({int(row['n_topics'])} topics)")
92         bars = ax.bar(x + (i - 0.5) * (w + 0.02), vals, w, color=SERIES[i], label=label)
93         ax.bar_label(bars, fmt="%.2f", padding=3, color=TEXT, fontsize=9)
94     ax.set_xticks(x)
95     ax.set_xticklabels([m[1] for m in metrics])
96     ax.set_ylim(0, 1.05)
97     ax.set_title("BERTopic beats LDA on every topic-quality metric")
98     ax.legend(loc="upper left")
99     ax.grid(axis="x", visible=False)
100     save(fig, "03_topic_model_comparison")
101
102 def fig_topic_difficulty():
103     t = pd.read_csv(OUTPUT_DIR / "topics.csv").sort_values("difficulty_index")
104     fig, ax = plt.subplots(figsize=(9, 6.5))
105     colors = [POS if v > 0 else NEG for v in t["difficulty_index"]]
106     # Name the course unit only when most of the topic's questions come from it.
107     names = [f"{u.split(' ')[0]} if isinstance(u, str) and pur >= 0.5 else 'Mixed': {lbl} ({n} q)"
108             for u, pur, lbl, n in zip(t["course_unit"], t["unit_purity"], t["label"],
109 t["n_questions"])]
110     ax.barh(names, t["difficulty_index"], color=colors, height=0.7)
111     ax.axvline(0, color=TEXT_2, linewidth=1)
112     fig.suptitle("Topic difficulty index\n( unanswered rate + time to answer + learner confusion)",
113 x=0.02, ha="left", fontweight="bold", fontsize=12)
114     ax.set_xlabel("← easier than average      harder than average →")
115     ax.grid(axis="y", visible=False)
116     save(fig, "04_topic_difficulty")
117
118 def fig_topic_trends():
119     tr = pd.read_csv(OUTPUT_DIR / "topic_trends.csv")
120     tr["year"] = tr["period"].str[:4]
121     y = tr.groupby(["topic", "year"])[["n_questions"]].sum().unstack(fill_value=0)
122     y = y.loc[:, y.sum(axis=0) >= 100] # years with too few questions give misleading shares
123     share = y / y.sum(axis=0)
124     names = topic_names()
125     share = share.loc[share.sum(axis=1).sort_values(ascending=False).index]
126     fig, ax = plt.subplots(figsize=(9, 6.5))
127     from matplotlib.colors import LinearSegmentedColormap
128     cmap = LinearSegmentedColormap.from_list("blue", ["#ffffff"] + BLUE_RAMP)
129     im = ax.imshow(share.values * 100, aspect="auto", cmap=cmap)
130     ax.set_yticks(range(len(share)))
131     ax.set_yticklabels([names.get(t, t) for t in share.index], fontsize=8.5)
132     ax.set_xticks(range(len(share.columns)))
133     ax.set_xticklabels(share.columns)
134     ax.grid(False)
135     for s in ax.spines.values():
136         s.set_visible(False)
137     cb = fig.colorbar(im, ax=ax, shrink=0.6)
138     cb.set_label("% of that year's questions")
139     cb.outline.set_visible(False)
140     ax.set_title("How the discussion mix changed over the years\n(years with 100+ questions)")
141     save(fig, "05_topic_trends")
142
143 # ----- network
144
145 def fig_help_concentration():
146     m = pd.read_csv(OUTPUT_DIR / "user_metrics.csv")
147     h = np.sort(m["out_strength"].to_numpy())[:-1]

```

```

148     cum = np.cumsum(h) / h.sum()
149     users = np.arange(1, len(h) + 1) / len(h)
150     fig, ax = plt.subplots(figsize=(6.5, 4))
151     ax.plot(users * 100, cum * 100, color=SERIES[0])
152     ax.plot([0, 100], [0, 100], color=OTHER, linestyle="--", linewidth=1)
153     for pct in [1, 10]:
154         i = max(int(len(h) * pct / 100) - 1, 0)
155         ax.plot(users[i] * 100, cum[i] * 100, "o", color=SERIES[0], markersize=8,
156               markeredgecolor=SURFACE, markeredgewidth=2)
157         ax.annotate(f"top {pct}% of users give {cum[i]:.0%} of help", (users[i] * 100, cum[i] * 100),
158               xytext=(10, -14), textcoords="offset points", fontsize=9, color=TEXT)
159     ax.text(60, 52, "equal sharing", color=TEXT_2, fontsize=9, rotation=28)
160     ax.set_xlabel("% of users (most helpful first)")
161     ax.set_ylabel("% of all help given")
162     ax.set_title("Help is concentrated in a small core of experts")
163     save(fig, "06_help_concentration")
164
165
166 def fig_degree_distribution():
167     m = pd.read_csv(OUTPUT_DIR / "user_metrics.csv")
168     fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
169     for ax, col, title in [(axes[0], "out_degree", "Help given (people helped)"),
170                           (axes[1], "in_degree", "Help received (helpers)"]):
171         d = m[col][m[col] > 0].to_numpy()
172         vals = np.sort(np.unique(d))
173         ccdf = [(d >= v).mean() for v in vals]
174         ax.loglog(vals, ccdf, "o", color=SERIES[0], markersize=4)
175         ax.set_title(title, fontsize=11)
176         ax.set_xlabel("Degree (log)")
177         ax.set_ylabel("P(degree ≥ x)")
178     fig.suptitle("Heavy-tailed degree distributions: a few hubs, many one-off learners",
179               x=0.02, ha="left", fontweight="bold", fontsize=12)
180     save(fig, "07_degree_distribution")
181
182
183 def fig_network_graph():
184     m = pd.read_csv(OUTPUT_DIR / "user_metrics.csv").set_index("user_id")
185     e = pd.read_csv(OUTPUT_DIR / "help_edges.csv")
186     top = m.nlargest(150, "out_strength").index
187     sub = e[e.source_user.isin(top) | e.target_user.isin(top)]
188     keep = set(top) | set(sub.target_user.value_counts().index[:250])
189     sub = sub[sub.source_user.isin(keep) & sub.target_user.isin(keep)]
190     g = nx.Graph()
191     for r in sub.itertuples():
192         g.add_edge(r.source_user, r.target_user, weight=r.weight)
193     pos = nx.spring_layout(g, k=0.6, seed=42, weight="weight", iterations=150)
194     roles = ["Core expert", "Bridge", "Helper"]
195     color = {r: SERIES[i] for i, r in enumerate(roles)}
196     nodes = list(g.nodes)
197     fig, ax = plt.subplots(figsize=(8, 7))
198     nx.draw_networkx_edges(g, pos, ax=ax, edge_color=GRID, width=0.6, alpha=0.9)
199     sizes = [15 + 18 * np.sqrt(m.at[n, "out_strength"]) if n in m.index else 15 for n in nodes]
200     cols = [color.get(m.at[n, "role"], OTHER) if n in m.index else OTHER for n in nodes]
201     order = np.argsort(sizes) # draw big nodes last so they stay visible
202     nx.draw_networkx_nodes(g, pos, nodelist=[nodes[i] for i in order], node_size=[sizes[i] for i in
203 order],
204                           node_color=[cols[i] for i in order], ax=ax, edgecolors=SURFACE,
205 linewidths=1)
206     for n in m.loc[m.index.isin(nodes)].nlargest(3, "out_strength").index:
207         ax.annotate(m.at[n, "user_name"], pos[n], xytext=(14, 10), textcoords="offset points",
208               fontsize=9, color=TEXT, fontweight="bold",
209               bbox=dict(boxstyle="round,pad=0.2", fc=SURFACE, ec="none", alpha=0.85))
210     counts = m["role"].value_counts()
211     handles = [plt.Line2D([], [], marker="o", linestyle="", color=color[r], markersize=8,
212           label=f"{r} ({counts.get(r, 0)} users)") for r in roles]
213     handles.append(plt.Line2D([], [], marker="o", linestyle="", color=OTHER, markersize=8,
214           label="Learner / other"))
215     ax.legend(handles=handles, loc="lower left", fontsize=8.5)
216     ax.set_title("Knowledge network core: top 150 helpers and the learners they help\n(node size =
217 help given)")
218     ax.axis("off")
219     save(fig, "08_network_core")
220
221
222 def fig_network_over_time():
223     t = pd.read_csv(OUTPUT_DIR / "network_over_time.csv")
224     t = t[t["active_users"] >= 50]
225     fig, axes = plt.subplots(1, 3, figsize=(11, 3.4))
226     for ax, col, title, fmt in [(axes[0], "active_users", "Active users", "{:.0f}"),
227                               (axes[1], "n_helpers", "Users who helped", "{:.0f}"),
228                               (axes[2], "help_share_top10pct", "Help given by top 10%", "{:.0%}")]
229         ax.plot(t["year"], t[col], color=SERIES[0], marker="o", markersize=5)
230     ax.set_title(title, fontsize=11)

```

```

227     ax.set_xticks(t["year"])
228     if "share" in col:
229         ax.yaxis.set_major_formatter(matplotlib.ticker.PercentFormatter(1.0))
230     fig.suptitle("The community peaked in 2020 and shrank afterwards", x=0.02, ha="left",
231                fontweight="bold", fontsize=12)
232     save(fig, "09_network_over_time")
233
234
235 # ----- diffusion
236
237 def fig_exposure():
238     e = pd.read_csv(OUTPUT_DIR / "diffusion_exposure_test.csv").sort_values("observed_exposed_share")
239     summ = json.load(open(OUTPUT_DIR / "diffusion_summary.json"))
240     fig, ax = plt.subplots(figsize=(9, 6.5))
241     y = np.arange(len(e))
242     ax.hlines(y, e["null_mean"] * 100, e["observed_exposed_share"] * 100, color=GRID, linewidth=2)
243     ax.plot(e["null_mean"] * 100, y, "o", color=OTHER, markersize=8, label="Expected by chance
244            (shuffled times)")
245     ax.plot(e["observed_exposed_share"] * 100, y, "o", color=SERIES[0], markersize=8,
246            label="Observed")
247     for yi, (obs, null, p) in enumerate(zip(e["observed_exposed_share"], e["null_mean"],
248            e["p_value"])):
249         ax.text(max(obs, null) * 100 + 0.8, yi, f"p = {p:.3f}", va="center", fontsize=8,
250                color=TEXT if p < 0.05 else TEXT_2, fontweight="bold" if p < 0.05 else "normal")
251     ax.set_yticks(y)
252     ax.set_yticklabels(e["label"], fontsize=8.5)
253     ax.set_xlabel("% of adopters who had contact with an earlier adopter before joining the topic")
254     ax.set_title(f"Social exposure test: {int((e.p_value < 0.05).sum())} of {len(e)} topics
255            significant\n"
256            f"(all topics combined: Stouffer z = {summ['exposure_stouffer_z']:.2f}, "
257            f"p = {summ['exposure_stouffer_p']:.3f})")
258     ax.legend(loc="lower right", fontsize=9)
259     ax.grid(axis="y", visible=False)
260     save(fig, "10_exposure_test")
261
262
263 def fig_learner_to_helper():
264     s = json.load(open(OUTPUT_DIR / "diffusion_summary.json"))["learner_to_helper"]
265     vals = [s["pct_helper_if_first_question_unanswered"], s["pct_helper_if_first_question_answered"]]
266     fig, ax = plt.subplots(figsize=(6, 3.6))
267     bars = ax.bar(["First question\nnot answered", "First question\nanswered"], [v * 100 for v in
268            vals],
269                color=[OTHER, SERIES[0]], width=0.55)
270     ax.bar_label(bars, labels=[f"{v:.1%}" for v in vals], padding=3, color=TEXT)
271     ax.set_ylabel("% who later helped others")
272     verdict = ("Learners whose first question was answered help others more often" if s["p_value"] <
273            0.05
274            else "Getting an answer does not change who becomes a helper")
275     ax.set_title(f"{verdict}\n( $\chi^2$  p = {s['p_value']:.2f}, {s['n_became_helpers']} of
276            {s['n_learners']} learners became helpers)",
277                fontsize=11)
278     ax.grid(axis="x", visible=False)
279     save(fig, "11_learner_to_helper")
280
281
282 def fig_influence():
283     im = pd.read_csv(OUTPUT_DIR / "influence_maximization.csv")
284     order = ["CELF greedy", "Top ExpertiseRank", "Top out-degree", "Top betweenness", "Top PageRank",
285            "Random"]
286     fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=False)
287     for ax, model, title in [(axes[0], "IC", "Independent Cascade"), (axes[1], "LT", "Linear
288            Threshold")]:
289         d = im[im.model == model]
290         for i, strat in enumerate(order):
291             s = d[d.strategy == strat].sort_values("k")
292             color = OTHER if strat == "Random" else SERIES[i]
293             ax.plot(s["k"], s["spread_pct_of_users"], color=color, marker="o", markersize=4,
294                    label=strat, linestyle="--" if strat == "Random" else "-")
295         ax.set_title(title, fontsize=11)
296         ax.set_xlabel("Number of seed users (k)")
297         ax.set_ylabel("% of users reached")
298         ax.set_xticks(range(1, 11))
299         axes[1].legend(loc="center left", bbox_to_anchor=(1.01, 0.5), fontsize=9)
300     fig.suptitle("Influence maximisation: which seed users spread knowledge furthest",
301                x=0.02, ha="left", fontweight="bold", fontsize=12)
302     save(fig, "12_influence_maximization")
303
304
305 def fig_diffusion_validation():
306     v = pd.read_csv(OUTPUT_DIR / "diffusion_validation.csv")
307     rho = json.load(open(OUTPUT_DIR / "diffusion_summary.json"))["validation"]["simulated_ic_spread"]
308     fig, ax = plt.subplots(figsize=(6.5, 4.5))
309     ax.scatter(v["simulated_ic_spread"], v["observed_reach"], s=36, color=SERIES[0],

```

```

301         edgecolors=SURFACE, linewidths=1.5, alpha=0.85)
302     ax.set_xscale("symlog")
303     ax.set_yscale("symlog")
304     ax.set_xlabel("Simulated IC spread from this user (users reached)")
305     ax.set_ylabel("Observed time-respecting reach")
306     ax.set_title(f"Simulated vs observed diffusion\n(Spearman  $\rho$  = {rho:.2f}, {len(v)} most active
helpers)")
307     save(fig, "13_diffusion_validation")
308
309
310 # ----- prediction & recommender
311
312 def fig_prediction():
313     roc = pd.read_csv(OUTPUT_DIR / "prediction_roc.csv")
314     met = pd.read_csv(OUTPUT_DIR / "prediction_metrics.csv").set_index("model")
315     fig, ax = plt.subplots(figsize=(6, 5))
316     for i, model in enumerate([m for m in met.index if not m.startswith("Baseline")]):
317         d = roc[roc.model == model]
318         ax.plot(d["fpr"], d["tpr"], color=SERIES[i], label=f"{model} (AUC {met.at[model,
'roc_auc']:.2f})")
319     ax.plot([0, 1], [0, 1], color=OTHER, linestyle="--", linewidth=1, label="Chance (AUC 0.50)")
320     ax.set_xlabel("False positive rate")
321     ax.set_ylabel("True positive rate")
322     ax.set_title("Will a new question be answered? (future test set)")
323     ax.legend(loc="lower right", fontsize=9)
324     save(fig, "14_prediction_roc")
325
326     imp = pd.read_csv(OUTPUT_DIR / "prediction_importance.csv").head(12).iloc[::-1]
327     fig, ax = plt.subplots(figsize=(7.5, 4.5))
328     ax.barh(imp["feature"].str.replace("_", " "), imp["importance_mean"], xerr=imp["importance_std"],
329            color=SERIES[0], height=0.65, error_kw={"ecolor": TEXT_2, "elinewidth": 1})
330     ax.set_xlabel("Drop in ROC-AUC when the feature is shuffled")
331     fig.suptitle(f"What predicts an answer ({imp['model'].iloc[0]}, permutation importance)",
332                x=0.02, ha="left", fontweight="bold", fontsize=12)
333     ax.grid(axis="y", visible=False)
334     save(fig, "15_prediction_importance")
335
336
337 def fig_recommender():
338     r = pd.read_csv(OUTPUT_DIR / "recommender_eval.csv")
339     ks = ["hit@1", "hit@5", "hit@10"]
340     x = np.arange(len(r))
341     w = 0.26
342     fig, ax = plt.subplots(figsize=(10, 4))
343     for i, k in enumerate(ks):
344         bars = ax.bar(x + (i - 1) * (w + 0.015), r[k] * 100, w, color=SERIES[i],
label=k.replace("hit@", "Hit@"))
345         ax.bar_label(bars, fmt="%0f", padding=2, fontsize=8, color=TEXT)
346     ax.set_xticks(x)
347     ax.set_xticklabels(r["method"].str.replace("(", "\n(", regex=False), fontsize=9)
348     ax.set_ylabel("% of test questions")
349     ax.set_ylim(0, 100)
350     ax.set_title("Expert finder: is a real answerer among the top-k recommendations?")
351     ax.legend(loc="upper left", ncol=3)
352     ax.grid(axis="x", visible=False)
353     save(fig, "16_recommender_eval")
354
355
356 FIGURES = [fig_activity, fig_lda_selection, fig_model_comparison, fig_topic_difficulty,
fig_topic_trends,
357            fig_help_concentration, fig_degree_distribution, fig_network_graph, fig_network_over_time,
358            fig_exposure, fig_learner_to_helper, fig_influence, fig_diffusion_validation,
359            fig_prediction, fig_recommender]
360
361
362 def run():
363     ok = 0
364     for f in FIGURES:
365         try:
366             f()
367             ok += 1
368         except FileNotFoundError as exc:
369             print(f"[figures] skipped {f.__name__}: missing {exc.filename}")
370     print(f"[figures] {ok}/{len(FIGURES)} figure groups written to {FIGURE_DIR}")
371
372
373 if __name__ == "__main__":
374     run()

```

Output

```

=== figures: report figures
[figures] 15/15 figure groups written to C:\Users\pc\python codes\Capstone project\outputs\figures
=== figures done in 5s
Pipeline finished in 9.2 min. Results are in outputs/.

```

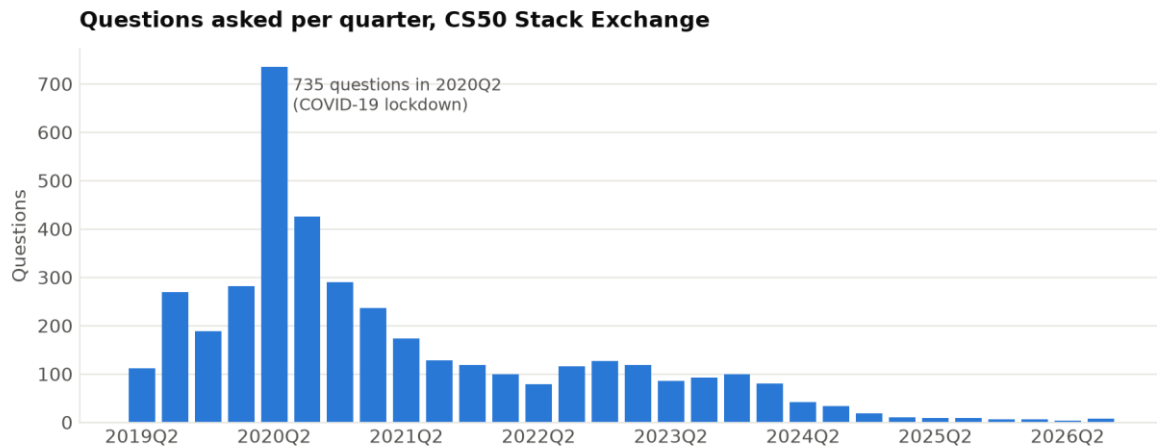


Figure 16. Questions per quarter (drawn by figures.py)

Source code: src/run_pipeline.py (55 lines)

```

1  """
2  Run the whole analysis in order.
3
4  Usage:
5      python src/run_pipeline.py          # everything
6      python src/run_pipeline.py --from network # start at a later step
7      python src/run_pipeline.py --only figures # a single step
8
9  Input is read from PostgreSQL when .env is configured and reachable, otherwise from the
10 CSV files in data/raw/. Results go to outputs/ (and to PostgreSQL via load_to_postgres.py).
11 """
12
13 import argparse
14 import time
15
16 import diffusion
17 import figures
18 import network
19 import predict
20 import preprocess
21 import recommender
22 import topic_model
23 from data_loader import data_source
24
25 STEPS = [
26     ("preprocess", preprocess.run, "clean text and build question features"),
27     ("topics", topic_model.run, "LDA + BERTopic topic modeling (~5 min)"),
28     ("network", network.run, "knowledge network, centrality, communities (~1 min)"),
29     ("diffusion", diffusion.run, "diffusion tests and IC/LT simulation (~8 min)"),
30     ("predict", predict.run, "answer prediction models (~1 min)"),
31     ("recommender", recommender.run, "expert recommender evaluation"),
32     ("figures", figures.run, "report figures"),
33 ]
34
35
36 def main():
37     names = [s[0] for s in STEPS]
38     parser = argparse.ArgumentParser(description=__doc__,
39     formatter_class=argparse.RawDescriptionHelpFormatter)
40     parser.add_argument("--from", dest="start", choices=names, default=names[0])
41     parser.add_argument("--only", choices=names)
42     args = parser.parse_args()
43
44     selected = [s for s in STEPS if s[0] == args.start] if args.start else
45 STEPS[names.index(args.start):]
46     print(f>Data source: {data_source()}\n")

```

```

45     total = time.time()
46     for name, func, desc in selected:
47         print(f"=== {name}: {desc}")
48         t = time.time()
49         func()
50         print(f"=== {name} done in {time.time() - t:.0f}s\n")
51     print(f"Pipeline finished in {(time.time() - total) / 60:.1f} min. Results are in outputs/.")
52
53
54 if __name__ == "__main__":
55     main()

```

Output of the complete pipeline run

```

Data source: CSV files
=== preprocess: clean text and build question features
[preprocess] 4000 questions cleaned (80% contain code, 46% mention an error)
=== preprocess done in 29s
=== topics: LDA + BERTopic topic modeling (~5 min)
[topics] 3960 questions, 82% mapped to a course unit
      LDA k= 6  NPMI=0.063  diversity=0.89
      LDA k= 8  NPMI=0.142  diversity=0.84
      LDA k=10  NPMI=0.119  diversity=0.82
      LDA k=12  NPMI=0.176  diversity=0.77
      LDA k=14  NPMI=0.185  diversity=0.78
      LDA k=16  NPMI=0.176  diversity=0.77
      LDA k=18  NPMI=0.173  diversity=0.74
      LDA k=20  NPMI=0.243  diversity=0.73
      model      n_topics      npmi      diversity      nmi_vs_course_tags
outlier_share_before_reduction
      LDA (k=20)      20 0.2426      0.7300      0.3839
0.0000
BERTopic (MiniLM + UMAP + HDBSCAN)      19 0.3099      0.8653      0.5613
0.1192
=== topics done in 134s
=== network: knowledge network, centrality, communities (~1 min)
[network] 2891 users, 4245 helper->learner links
[network] 118 communities (modularity 0.60); top 10% of users give 92% of all help
=== network done in 8s
=== diffusion: diffusion tests and IC/LT simulation (~8 min)
[diffusion] exposure test: 3/19 topics show more contact with earlier adopters than chance
[diffusion] influence maximisation (IC) ...
[diffusion] influence maximisation (LT) ...
[diffusion] validating simulated influence against observed reach ...
[diffusion] simulated influence vs observed reach: Spearman rho = 0.86
=== diffusion done in 336s
=== predict: answer prediction models (~1 min)
[predict] train 3168 questions (until 2022-08), test 792 (answered rate 59%)
      Baseline (majority class)      test ROC-AUC 0.500      CV 0.500
      Logistic Regression      test ROC-AUC 0.673      CV 0.592
      Random Forest      test ROC-AUC 0.675      CV 0.602
      XGBoost      test ROC-AUC 0.656      CV 0.599
[predict] best model: Random Forest
=== predict done in 29s
=== recommender: expert recommender evaluation
[recommender] 698 answered test questions, 77% answered by someone already active before the cut-off
      Content (tfidf)      Hit@5 0.689      Hit@10 0.710      MRR 0.596
      Hybrid (tfidf)      Hit@5 0.717      Hit@10 0.798      MRR 0.628
      Popularity (most answers)      Hit@5 0.702      Hit@10 0.702      MRR 0.582
      Content (embedding)      Hit@5 0.708      Hit@10 0.727      MRR 0.611
      Hybrid (embedding)      Hit@5 0.725      Hit@10 0.811      MRR 0.637
=== recommender done in 13s
=== figures: report figures
[figures] 15/15 figure groups written to C:\Users\pc\python codes\Capstone project\outputs\figures
=== figures done in 5s
Pipeline finished in 9.2 min. Results are in outputs/.

```

12. SQL Analysis Queries (queries.sql)

These queries run in pgAdmin (**Tools → Query Tool**) after [load_to_postgres.py](#). Queries 1–6 use the raw tables; queries 7–11 use the result tables. All queries were checked with the PostgreSQL parser. Run each query on the laptop and paste the result screenshot in the box below it.

Query 1. Dataset overview

```

1 SELECT
2   (SELECT COUNT(*) FROM users)      AS users,
3   (SELECT COUNT(*) FROM questions)  AS questions,
4   (SELECT COUNT(*) FROM answers)    AS answers,
5   (SELECT COUNT(*) FROM comments)   AS comments,
6   (SELECT COUNT(*) FROM interactions) AS interactions

```

Paste screenshot: pgAdmin result of query 1

Query 2. Top 10 helpers: most answers, and how many were accepted

```

1 SELECT u.user_name,
2        u.user_reputation,
3        COUNT(*)      AS answers_given,
4        COUNT(*) FILTER (WHERE a.is_accepted) AS accepted_answers
5 FROM answers a
6 JOIN users u ON u.user_id = a.user_id
7 GROUP BY u.user_id, u.user_name, u.user_reputation
8 ORDER BY answers_given DESC
9 LIMIT 10

```

Paste screenshot: pgAdmin result of query 2

Query 3. Most discussed tags and how often their questions go unanswered

```

1 SELECT tag,
2        COUNT(*) AS questions,
3        ROUND(100.0 * AVG(CASE WHEN q.is_answered THEN 0 ELSE 1 END), 1) AS pct_unanswered
4 FROM questions q,
5      unnest(string_to_array(q.tags, '|')) AS tag

```

```

6 GROUP BY tag
7 HAVING COUNT(*) >= 10
8 ORDER BY questions DESC
9 LIMIT 20

```

Paste screenshot: pgAdmin result of query 3

Query 4. Median hours until the first answer, per month

```

1 SELECT date_trunc('month', q.created_at)::date AS month,
2        COUNT(*) AS answered_questions,
3        ROUND((PERCENTILE_CONT(0.5) WITHIN GROUP (
4            ORDER BY EXTRACT(EPOCH FROM fa.first_answer - q.created_at) / 3600))::numeric, 1)
5        AS median_hours_to_first_answer
6 FROM questions q
7 JOIN (SELECT question_id, MIN(created_at) AS first_answer
8      FROM answers GROUP BY question_id) fa ON fa.question_id = q.post_id
9 GROUP BY month
10 ORDER BY month

```

Paste screenshot: pgAdmin result of query 4

Query 5. Knowledge diffusion: learners who first asked, and later became helpers

```

1 WITH first_question AS (
2     SELECT user_id, MIN(created_at) AS first_asked FROM questions GROUP BY user_id
3 ), first_answer AS (
4     SELECT user_id, MIN(created_at) AS first_answered FROM answers GROUP BY user_id
5 )
6 SELECT u.user_name,
7        fq.first_asked::date,
8        fa.first_answered::date,
9        fa.first_answered::date - fq.first_asked::date AS days_to_become_helper
10 FROM first_question fq
11 JOIN first_answer fa ON fa.user_id = fq.user_id AND fa.first_answered > fq.first_asked
12 JOIN users u ON u.user_id = fq.user_id
13 ORDER BY days_to_become_helper

```

Paste screenshot: pgAdmin result of query 5

Query 6. Strongest helper -> learner links in the knowledge network

```

1 SELECT s.user_name AS helper,
2        t.user_name AS learner,
3        COUNT(*)     AS interactions
4 FROM interactions i
5 JOIN users s ON s.user_id = i.source_user
6 LEFT JOIN users t ON t.user_id = i.target_user
7 GROUP BY s.user_name, t.user_name
8 ORDER BY interactions DESC
9 LIMIT 15

```

Paste screenshot: pgAdmin result of query 6

Query 7. Hardest topics: unanswered rate, waiting time and confusion

```

1 SELECT label, course_unit, n_questions,
2        ROUND(pct_unanswered::numeric, 1) AS pct_unanswered,
3        ROUND(median_hours_to_answer::numeric, 1) AS median_hours_to_answer,
4        ROUND(difficulty_index::numeric, 2) AS difficulty_index
5 FROM topics
6 ORDER BY difficulty_index DESC

```

Paste screenshot: pgAdmin result of query 7

Query 8. Top 3 experts for every topic (who answers the most questions in it)

```

1 WITH per_topic AS (
2   SELECT qt.topic, a.user_id, COUNT(*) AS answers,
3         ROW_NUMBER() OVER (PARTITION BY qt.topic ORDER BY COUNT(*) DESC) AS rnk
4   FROM answers a
5   JOIN question_topics qt ON qt.post_id = a.question_id
6   GROUP BY qt.topic, a.user_id
7 )
8 SELECT t.label AS topic, u.user_name, p.answers
9   FROM per_topic p
10  JOIN topics t ON t.topic = p.topic
11  JOIN users u ON u.user_id = p.user_id
12  WHERE p.rnk <= 3
13  ORDER BY t.label, p.answers DESC

```

Paste screenshot: pgAdmin result of query 8

Query 9. Network roles: how many users play each role, and how much help they give

```

1 SELECT role,
2        COUNT(*) AS users,
3        SUM(out_strength) AS help_given,
4        ROUND(100.0 * SUM(out_strength) / SUM(SUM(out_strength)) OVER (), 1) AS pct_of_all_help
5   FROM user_metrics
6   GROUP BY role
7   ORDER BY help_given DESC

```

Paste screenshot: pgAdmin result of query 9

Query 10. Topic popularity by year (share of each year's questions)

```

1 SELECT LEFT(tt.period, 4) AS year, t.label AS topic, SUM(tt.n_questions) AS questions
2   FROM topic_trends tt
3  JOIN topics t ON t.topic = tt.topic
4  GROUP BY LEFT(tt.period, 4), t.label
5  ORDER BY year, questions DESC

```

Paste screenshot: pgAdmin result of query 10

Query 11. Model results at a glance

```
1 SELECT 'prediction' AS task, model AS method, ROUND(roc_auc::numeric, 3) AS score, 'ROC-AUC' AS metric
2 FROM prediction_metrics
3 UNION ALL
4 SELECT 'expert finder', method, ROUND("hit@10"::numeric, 3), 'Hit@10'
5 FROM recommender_eval
6 ORDER BY task, score DESC
```

Paste screenshot: pgAdmin result of query 11

13. Interactive Dashboard (streamlit_app.py)

Aim: present all results in a web app with seven pages. The **Answer prediction** page trains a Random Forest on all questions and estimates the chance that a newly typed question gets answered, with tips. The **Expert finder** page recommends experts and similar past questions for a problem description. The **Database** page runs the SQL queries when PostgreSQL is connected.

Run: `streamlit run app/streamlit_app.py`, then open <http://localhost:8501>

Source code: `app/streamlit_app.py` (559 lines)

```

1  """
2  Learning Community Analytics – interactive dashboard.
3
4  Run from the project folder:
5      streamlit run app/streamlit_app.py
6  """
7
8  import json
9  import re
10 import sys
11 from datetime import datetime
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16 import plotly.express as px
17 import plotly.graph_objects as go
18 import streamlit as st
19
20 ROOT = Path(__file__).resolve().parent.parent
21 sys.path.insert(0, str(ROOT / "src"))
22
23 from config import OUTPUT_DIR, PROCESSED_DIR, TRAIN_FRACTION # noqa: E402
24 from data_loader import data_source, get_db_engine, load_raw # noqa: E402
25
26 st.set_page_config(page_title="Learning Community Analytics", page_icon="📊", layout="wide")
27
28 SERIES = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300", "#4a3aa7", "#e34948"]
29 OTHER = "#b4b2aa"
30 BLUES = ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#256abf", "#184f95", "#0d366b"]
31
32
33 # ===== data
34
35 @st.cache_data(show_spinner="Loading community data ...")
36 def raw_data():
37     return load_raw()
38
39
40 @st.cache_data
41 def out(name):
42     path = OUTPUT_DIR / name
43     if not path.exists():
44         return None
45     if path.suffix == ".json":
46         return json.loads(path.read_text())
47     return pd.read_csv(path)
48
49
50 @st.cache_data
51 def processed(name):
52     path = PROCESSED_DIR / name
53     return pd.read_parquet(path) if path.exists() else None
54
55
56 def need(*frames):
57     if any(f is None for f in frames):
58         st.warning("Results are missing. Run `python src/run_pipeline.py` first.")
59         st.stop()
60
61
62 def plot(fig, height=None):
63     fig.update_layout(margin=dict(l=10, r=10, t=40, b=10), legend_title_text="",
64                       font=dict(size=13), hoverlabel=dict(font_size=13))
65     if height:
66         fig.update_layout(height=height)

```

```

67     st.plotly_chart(fig, width="stretch", theme="streamlit")
68
69
70 def topic_labels():
71     t = out("topics.csv")
72     return {} if t is None else dict(zip(t["topic"], t["label"]))
73
74
75 # ===== text helpers
76
77 @st.cache_resource(show_spinner="Loading language model ...")
78 def encoder():
79     """Sentence-transformer if installed, else None (TF-IDF fallback)."""
80     try:
81         from sentence_transformers import SentenceTransformer
82         from config import EMBEDDING_MODEL
83         return SentenceTransformer(EMBEDDING_MODEL)
84     except Exception:
85         return None
86
87
88 @st.cache_resource
89 def text_tools():
90     from preprocess import DOMAIN_STOPWORDS, TOKEN_RE, _nltk_resources
91     stop, lemmatizer, vader = _nltk_resources()
92     stop = stop | DOMAIN_STOPWORDS
93
94     def tokenize(text):
95         toks = [lemmatizer.lemmatize(t) for t in TOKEN_RE.findall(text.lower())]
96         return " ".join(t for t in toks if t not in stop and len(t) > 2)
97     return tokenize, vader
98
99
100 @st.cache_resource(show_spinner="Building the expert finder ...")
101 def finder_and_vectors():
102     from recommender import ExpertFinder
103     from sklearn.feature_extraction.text import TfidfVectorizer
104     q = processed("questions_clean.parquet")
105     ids = np.load(PROCESSED_DIR / "embedding_post_ids.npy")
106     q = q[q["post_id"].isin(ids)].reset_index(drop=True)
107     if encoder() is not None:
108         emb = np.load(PROCESSED_DIR / "embeddings.npy")
109         idx = pd.Series(np.arange(len(ids)), index=ids)
110         vectors, mode, tfidf = emb[idx[q["post_id"]].to_numpy()], "sentence embeddings", None
111     else:
112         tfidf = TfidfVectorizer(token_pattern=r"\S+", ngram_range=(1, 2), min_df=2,
113 sublinear_tf=True)
114         vectors, mode = tfidf.fit_transform(q["tokens"]).toarray().astype(np.float32), "TF-IDF"
115         answers = raw_data()["answers"]
116         return ExpertFinder(q, answers, vectors), q, vectors, mode, tfidf
117
118 def embed_query(text, tfidf):
119     enc = encoder()
120     if enc is not None:
121         return enc.encode([text], normalize_embeddings=True)[0]
122     tokenize, _ = text_tools()
123     return tfidf.transform([tokenize(text)]).toarray()[0]
124
125
126 # ===== pages
127
128 def page_overview():
129     raw = raw_data()
130     q, a, u = raw["questions"], raw["answers"], raw["users"]
131     st.title("📖 Learning Community Analytics")
132     st.caption("NLP topics and knowledge-diffusion networks in the CS50 Stack Exchange learning community")
133
134     c = st.columns(5)
135     c[0].metric("Questions", f"{len(q):,}")
136     c[1].metric("Answers", f"{len(a):,}")
137     c[2].metric("Comments", f"{len(raw['comments']):,}")
138     c[3].metric("Users", f"{len(u):,}")
139     c[4].metric("Time span", f"{q.created_at.min():%Y} - {q.created_at.max():%Y}")
140
141     per = q.groupby(q["created_at"].dt.to_period("Q")).size()
142     fig = go.Figure(go.Bar(x=per.index.astype(str), y=per.values, marker_color=SERIES[0],
143 hovertemplate="%{x}: %{y} questions<extra></extra>"))
144     fig.update_layout(title="Questions per quarter (peak = COVID-19 lockdown, 2020)",
145 yaxis_title="Questions")
146     plot(fig, 360)

```

```

147     st.subheader("Key findings")
148     comp, stats, diff = out("topic_model_comparison.csv"), out("network_stats.json"),
out("diffusion_summary.json")
149     topics, pred, rec = out("topics.csv"), out("prediction_metrics.csv"), out("recommender_eval.csv")
150     lines = []
151     if comp is not None:
152         b = comp.iloc[1]
153         lines.append(f"***Topics:** BERTopic found {int(b.n_topics)} topics that match the course
syllabus "
154                     f"(NMI {b.nmi_vs_course_tags:.2f} vs {comp.iloc[0].nmi_vs_course_tags:.2f} for
LDA).")
155     if topics is not None:
156         hard = topics.sort_values("difficulty_index", ascending=False).iloc[0]
157         unit = str(hard.course_unit).replace(" ", "").rstrip("")
158         lines.append(f"***Hardest topic:** {hard.label}* ({unit}): "
159                     f"{hard.pct_unanswered:.0f}% of its questions stay unanswered.")
160     if stats is not None:
161         lines.append(f"***Network:** the top 1% of users give {stats['help_share_top1pct']:.0f}% of all
help; "
162                     f"{stats['n_communities_10plus']} sub-communities (modularity
{stats['modularity']:.2f}).")
163     if diff is not None:
164         l2h = diff["learner_to_helper"]
165         lines.append(f"***Diffusion:** a modest social-influence effect overall (combined z = "
166                     f"{diff['exposure_stouffer_z']:.2f}, p = {diff['exposure_stouffer_p']:.3f}),
significant in "
167                     f"{diff['exposure_topics_significant']} of {diff['exposure_topics_tested']}
topics: "
168                     f"{', '.join(diff['exposure_significant_topics'])}.")
169         lines.append(f"***Simulation check:** simulated influence matches observed time-respecting
reach "
170                     f"(Spearman  $\rho$  = {diff['validation']['simulated_ic_spread']:.2f}).")
171         lines.append(f"***Learners  $\rightarrow$  helpers:** {l2h['pct_became_helpers']:.1f}% of learners later
helped others "
172                     f"(median {l2h['median_days_to_become_helper']:.0f} days); getting an answer to
the first "
173                     f"question made {'a significant' if l2h['p_value'] < 0.05 else 'no significant'}
difference "
174                     f"(p = {l2h['p_value']:.2f}).")
175     if pred is not None:
176         best = pred.iloc[1:].sort_values("roc_auc").iloc[-1]
177         lines.append(f"***Prediction:** {best.model} predicts whether a new question gets answered "
178                     f"(ROC-AUC {best.roc_auc:.2f} on future questions).")
179     if rec is not None:
180         best = rec.sort_values("hit@10").iloc[-1]
181         lines.append(f"***Expert finder:** a real answerer is in the top 10 for {best['hit@10']:.0f}%
of new questions.")
182     for line in lines:
183         st.markdown(f"- {line}")
184
185
186     def page_topics():
187         st.title("Discussion topics")
188         topics, comp, tmap, trends = out("topics.csv"), out("topic_model_comparison.csv"),
out("topic_map.csv"), out("topic_trends.csv")
189         need(topics, comp, tmap, trends)
190         names = topic_labels()
191
192         st.subheader("Model comparison: LDA vs BERTopic")
193         st.dataframe(comp.rename(columns={"npmi": "NPMI coherence", "diversity": "Topic diversity",
194                                         "nmi_vs_course_tags": "NMI vs course tags"}),
195                     hide_index=True, width="stretch")
196
197         st.subheader("Topic map")
198         choice = st.selectbox("Highlight a topic", topics["topic"], format_func=lambda t: f"{t}:
{names[t]}")
199         tmap = tmap.assign(highlight=np.where(tmap["topic"] == choice, names[choice], "Other topics"))
200         fig = px.scatter(tmap, x="x", y="y", color="highlight", hover_name="title",
201                         color_discrete_map={names[choice]: SERIES[0], "Other topics": OTHER},
202                         category_orders={"highlight": ["Other topics", names[choice]]})
203         fig.update_traces(marker=dict(size=6, line=dict(width=0.5, color="white")))
204         fig.update_xaxes(visible=False)
205         fig.update_yaxes(visible=False)
206         fig.update_layout(title="Every question, placed by meaning (UMAP of sentence embeddings)")
207         plot(fig, 500)
208
209         row = topics.set_index("topic").loc[choice]
210         c = st.columns(4)
211         c[0].metric("Questions", int(row.n_questions))
212         c[1].metric("Unanswered", f"{row.pct_unanswered:.0f}%")
213         c[2].metric("Median wait for answer", f"{row.median_hours_to_answer:.1f} h")
214         # Name a course unit only when most of the topic's tagged questions come from it.
215         c[3].metric("Course unit", str(row.course_unit) if row.unit_purity >= 0.5 else "Mixed")

```

```

216     st.markdown(f"""Top words:** {row.top_words}""")
217     q = processed("questions_clean.parquet")
218     qt = processed("question_topics.parquet")
219     sample = q.merge(qt[qt.topic == choice][["post_id"]], on="post_id").nlargest(8, "view_count")
220     st.markdown(f"""Most viewed questions in this topic""")
221     for r in sample.itertuples():
222         st.markdown(f"- [{r.title}](https://cs50.stackexchange.com/q/{r.post_id}) · {r.view_count:,} views")
223
224     st.subheader("Topic difficulty")
225     d = topics.sort_values("difficulty_index")
226     fig = go.Figure(go.Bar(x=d["difficulty_index"], y=d["label"], orientation="h",
227         marker_color=[SERIES[7] if v > 0 else SERIES[0] for v in
d["difficulty_index"],
228         customdata=d[["pct_unanswered", "median_hours_to_answer", "n_questions"]],
229         hovertemplate="%{y}<br>difficulty %{x:.2f}<br>unanswered
%{customdata[0]:.0f}%"  

230                                     "<br>median wait %{customdata[1]:.1f} h<br>%{customdata[2]}
questions<extra></extra>"))
231     fig.update_layout(title="Difficulty index (unanswered rate + wait time + confusion), higher =
harder")
232     plot(fig, 560)
233
234     st.subheader("How topics changed over time")
235     trends["year"] = trends["period"].str[:4]
236     y = trends.groupby(["topic", "year"])["n_questions"].sum().reset_index()
237     per_year = y.groupby("year")["n_questions"].transform("sum")
238     y = y[per_year >= 100] # years with very few questions give misleading shares
239     y["share"] = 100 * y["n_questions"] / y.groupby("year")["n_questions"].transform("sum")
240     y["label"] = y["topic"].map(names)
241     heat = y.pivot(index="label", columns="year", values="share").fillna(0)
242     fig = px.imshow(heat, color_continuous_scale=["#ffffff"] + BLUES, aspect="auto",
243         labels=dict(color="% of year's questions"))
244     fig.update_traces(hovertemplate="%{y}<br>%{x}: %{z:.1f}% of questions<extra></extra>")
245     st.caption("Only years with at least 100 questions are shown.")
246     plot(fig, 560)
247
248
249     def network_html(metrics, edges, n_top):
250         from pyvis.network import Network
251         top = metrics.nlargest(n_top, "out_strength")
252         ids = set(top["user_id"])
253         sub = edges[edges.source_user.isin(ids) & edges.target_user.isin(ids)]
254         comms = metrics["community"].value_counts().index[:3].tolist()
255         color = {c: SERIES[i] for i, c in enumerate(comms)}
256         net = Network(height="620px", width="100%", directed=True, bgcolor="#ffffff",
font_color="#0b0b0b",
257             cdn_resources="in_line")
258         net.barnes_hut(gravity=-6000, spring_length=140)
259         for r in top.itertuples():
260             net.add_node(int(r.user_id), label=str(r.user_name), size=8 + 3 * np.sqrt(r.out_strength),
261                 color=color.get(r.community, OTHER),
262                 title=f"{r.user_name}\nrole: {r.role}\nhelped: {r.out_strength}\ncommunity:
{r.community}")
263         for r in sub.itertuples():
264             net.add_edge(int(r.source_user), int(r.target_user), value=int(r.weight), color="#cccccc")
265         # generate_html avoids writing a file (pyvis would use the Windows code page and fail on names).
266         return net.generate_html(notebook=False), comms
267
268
269     def page_network():
270         st.title("Knowledge network")
271         stats, m, edges, over, val = (out("network_stats.json"), out("user_metrics.csv"),
out("help_edges.csv"),
272             out("network_over_time.csv"), out("network_validation.csv"))
273         need(stats, m, edges, over, val)
274         c = st.columns(5)
275         c[0].metric("Users in network", f"{stats['n_users']:,}")
276         c[1].metric("Helper → learner links", f"{stats['n_edges']:,}")
277         c[2].metric("Help from top 1%", f"{stats['help_share_top1pct']:.0%}")
278         c[3].metric("Communities (10+ users)", stats["n_communities_10plus"])
279         c[4].metric("Modularity", f"{stats['modularity']:.2f}")
280
281         st.subheader("Interactive network of the most active helpers")
282         n_top = st.slider("Number of users to show", 30, 200, 80, step=10)
283         html, comms = network_html(m, edges, n_top)
284         st.caption("Drag nodes, scroll to zoom, hover for details. Colours = the three largest
communities ")
285         f("{'', '.join(map(str, comms))}); grey = other communities. Node size = help given.")
286         st.components.v1.html(html, height=640, scrolling=False)
287
288         left, right = st.columns([3, 2])
289         with left:

```

```

290     st.subheader("Top experts (ExpertiseRank)")
291     names = topic_labels()
292     show = m.head(20)[["user_name", "role", "out_strength", "n_answers", "n_accepted",
293                       "expertise_rank", "betweenness", "community", "main_help_topic"]].copy()
294     show["main_help_topic"] = show["main_help_topic"].map(names)
295     st.dataframe(show, hide_index=True, width="stretch")
296     with right:
297         st.subheader("User roles")
298         roles = m["role"].value_counts()
299         fig = go.Figure(go.Bar(x=roles.values, y=roles.index, orientation="h",
marker_color=SERIES[0],
300                             hovertemplate="%{y}: %{x} users<extra></extra>"))
301         plot(fig, 330)
302
303     st.subheader("The network over time")
304     metric = st.radio("Measure", ["active_users", "n_helpers", "help_share_top10pct", "modularity",
"reciprocity"],
305                       horizontal=True, format_func=lambda s: s.replace("_", " "))
306     fig = go.Figure(go.Scatter(x=over["year"], y=over[metric], mode="lines+markers",
line=dict(color=SERIES[0], width=2),
307                             marker=dict(size=8)))
308     fig.update_layout(yaxis_title=metric.replace("_", " "))
309     plot(fig, 330)
310
311     st.subheader("Validation: do network scores agree with Stack Exchange reputation?")
312     st.dataframe(val, hide_index=True, width="stretch")
313
314     def page_diffusion():
315         st.title("Knowledge diffusion")
316         summ, exp, im, curves, valid = (out("diffusion_summary.json"),
317 out("diffusion_exposure_test.csv"),
318 out("topic_adoption_curves.csv"),
319 out("diffusion_validation.csv"))
320         need(summ, exp, im, curves, valid)
321         l2h = summ["learner_to_helper"]
322         c = st.columns(4)
323         c[0].metric("Topics with social-influence signal", f"{summ['exposure_topics_significant']} /
{summ['exposure_topics_tested']}"),
324                     f"combined p = {summ['exposure_stouffer_p']:.3f}", delta_color="off")
325         c[1].metric("Learners who became helpers", f"{l2h['pct_became_helpers']:.0%}")
326         c[2].metric("...if first question answered", f"{l2h['pct_helper_if_first_question_answered']:.1%}",
327                     f"vs {l2h['pct_helper_if_first_question_unanswered']:.1%} unanswered",
delta_color="off")
328         c[3].metric("Simulation vs reality (p)", f"{summ['validation']['simulated_ic_spread']:.2f}")
329
330     st.subheader("Social exposure test")
331     st.caption("For each topic: the share of users who, before first joining the topic, had talked
with someone "
332               "who had already joined it – compared with 200 random re-orderings of the joining
times.")
333     e = exp.sort_values("observed_exposed_share")
334     fig = go.Figure()
335     fig.add_trace(go.Scatter(x=100 * e["null_mean"], y=e["label"], mode="markers", name="Expected by
chance",
336                             marker=dict(color=OTHER, size=10)))
337     fig.add_trace(go.Scatter(x=100 * e["observed_exposed_share"], y=e["label"], mode="markers",
name="Observed",
338                             marker=dict(color=SERIES[0], size=10), customdata=e[["p_value"]],
339                             hovertemplate="%{y}<br>observed %{x:.1f}%<br>p =
%{customdata[0]:.3f}<extra></extra>"))
340     fig.update_layout(xaxis_title="% of adopters exposed to an earlier adopter")
341     plot(fig, 560)
342
343     st.subheader("Topic adoption curves")
344     names = topic_labels()
345     pick = st.multiselect("Topics", sorted(curves["topic"].unique()),
default=sorted(curves["topic"].unique())[:3],
346                  format_func=lambda t: names.get(t, t), max_selections=8)
347     fig = go.Figure()
348     for i, t in enumerate(pick):
349         d = curves[curves.topic == t]
350         fig.add_trace(go.Scatter(x=d["period"], y=d["cumulative_adopters"], name=names.get(t, t),
351                                 mode="lines", line=dict(color=SERIES[i], width=2)))
352     fig.update_layout(yaxis_title="Users who have joined the topic", hovermode="x unified")
353     plot(fig, 380)
354
355     st.subheader("Influence maximisation (simulation)")
356     model = st.radio("Diffusion model", ["IC", "LT"], horizontal=True,
357                     format_func=lambda s: {"IC": "Independent Cascade", "LT": "Linear
Threshold"}[s])
358     d = im[im.model == model]

```

```

359     order = ["CELF greedy", "Top ExpertiseRank", "Top out-degree", "Top betweenness", "Top PageRank",
"Random"]
360     fig = go.Figure()
361     for i, s in enumerate(order):
362         x = d[d.strategy == s].sort_values("k")
363         fig.add_trace(go.Scatter(x=x["k"], y=x["spread_pct_of_users"], name=s, mode="lines+markers",
364                                line=dict(color=OTHER if s == "Random" else SERIES[i], width=2,
365                                           dash="dash" if s == "Random" else "solid"))))
366     fig.update_layout(xaxis_title="Seed users", yaxis_title="% of users reached", hovermode="x
unified")
367     plot(fig, 400)
368     seeds = json.loads((OUTPUT_DIR / "seed_sets.json").read_text())[model]["CELF greedy"]
369     m = out("user_metrics.csv").set_index("user_id")
370     st.markdown("**Best seed users (CELF):** " + ", ".join(str(m.at[s, "user_name"]) for s in seeds
if s in m.index))
371
372     st.subheader("Does the simulation match what really happened?")
373     fig = px.scatter(valid, x="simulated_ic_spread", y="observed_reach", hover_name="user_name",
374                    log_x=True, log_y=True, color_discrete_sequence=[SERIES[0]])
375     fig.update_layout(xaxis_title="Simulated spread from user", yaxis_title="Observed time-respecting
reach")
376     plot(fig, 420)
377
378
379 @st.cache_resource(show_spinner="Training the answer-prediction model ...")
380 def answer_model():
381     from sklearn.ensemble import RandomForestClassifier
382     from sklearn.pipeline import Pipeline
383     from predict import CATEGORICAL, NUMERIC, TARGET, models
384     df = processed("prediction_features.parquet")
385     model = models()["Random Forest"]
386     model.fit(df[NUMERIC + CATEGORICAL], df[TARGET].astype(int))
387     return model, df
388
389
390 def page_prediction():
391     st.title("Will this question get answered?")
392     met, imp, roc = out("prediction_metrics.csv"), out("prediction_importance.csv"),
out("prediction_roc.csv")
393     need(met, imp, roc)
394
395     st.subheader("Check your question")
396     title = st.text_input("Title", "Segmentation fault in recover when reading the memory card")
397     body = st.text_area("Question body", "I'm stuck on recover. My code compiles but I get a
segmentation fault ")
398     "when I run it. I use fread into a buffer of 512 bytes. What am I doing
wrong?", height=120)
399     tags = st.text_input("Tags (separated by spaces)", "pset4 recover segmentation-fault")
400     has_code = st.checkbox("I include my code", value=True)
401     if st.button("Predict", type="primary"):
402         from predict import CATEGORICAL, NUMERIC
403         from preprocess import CONFUSION_TERMS, ERROR_RE
404         from topic_model import course_unit
405         model, df = answer_model()
406         tokenize, vader = text_tools()
407         text = f"{title}. {body}"
408         finder, q, vectors, mode, tfidf = finder_and_vectors()
409         qt = processed("question_topics.parquet").set_index("post_id")
410         rows, _ = finder.similar(embed_query(text, tfidf), n=15)
411         topic = qt.loc[q.loc[rows, "post_id"], "topic"].mode().iloc[0]
412         words = max(len(text.split()), 1)
413         tag_str = "|".join(tags.split())
414         topics = out("topics.csv").set_index("topic")
415         recent = df.iloc[-200:]
416         feat = pd.DataFrame([
417             "n_words": words, "title_words": len(title.split()), "n_code_lines": 20 if has_code else
0,
418             "n_question_marks": text.count("?"), "n_tags": len(tags.split()),
419             "sentiment": vader.polarity_scores(text)["compound"],
420             "confusion_score": sum(text.lower().count(t) for t in CONFUSION_TERMS) / np.sqrt(words) *
10,
421             "has_code": int(has_code), "has_error_msg": int(bool(ERROR_RE.search(text))),
422             "has_check50_fail": int("50" in text), "asker_prior_questions": 0, "asker_prior_answers":
0,
423             "asker_is_new": 1, "asker_prior_answered_rate": -1,
424             "questions_last_30d": recent["questions_last_30d"].median(),
425             "helpers_last_30d": recent["helpers_last_30d"].median(),
426             "topic_prior_answered_rate": 1 - topics.at[topic, "pct_unanswered"] / 100,
427             "hour": datetime.now().hour, "weekday": datetime.now().weekday(),
428             "topic": str(topic), "course_unit": course_unit(tag_str) or "Unknown",
429         ])
430     p = model.predict_proba(feat[NUMERIC + CATEGORICAL])[0, 1]
431     c = st.columns(3)

```

```

432     c[0].metric("Chance of a good answer", f"{p:.0%}")
433     c[1].metric("Detected topic", topic_labels()[topic])
434     c[2].metric("Course unit", course_unit(tag_str) or "Unknown")
435     tips = []
436     if not has_code:
437         tips.append("Include the relevant part of your code: questions with code are answered
more often.")
438     if len(tags.split()) < 3:
439         tips.append("Add more tags (problem set + concept), e.g. `pset4 recover malloc`.")
440     if words < 40:
441         tips.append("Describe what you expected, what happened, and what you already tried.")
442     for t in tips:
443         st.info(t)
444     st.caption("Assumes a first-time asker posting now. Model: Random Forest trained on all
questions.")
445
446     st.subheader("Model evaluation on future questions")
447     st.dataframe(met, hide_index=True, width="stretch")
448     left, right = st.columns(2)
449     with left:
450         fig = go.Figure()
451         for i, mname in enumerate([m for m in met.model if not m.startswith("Baseline")]):
452             d = roc[roc.model == mname]
453             fig.add_trace(go.Scatter(x=d.fpr, y=d.tpr, name=mname, mode="lines",
line=dict(color=SERIES[i], width=2)))
454             fig.add_trace(go.Scatter(x=[0, 1], y=[0, 1], name="Chance", mode="lines",
line=dict(color=OTHER, dash="dash")))
455             fig.update_layout(title="ROC curves", xaxis_title="False positive rate", yaxis_title="True
positive rate")
456             plot(fig, 420)
457     with right:
458         d = imp.head(12).iloc[::-1]
459         fig = go.Figure(go.Bar(x=d.importance_mean, y=d.feature, orientation="h",
marker_color=SERIES[0],
error_x=dict(type="data", array=d.importance_std, color="#52514e")))
460         fig.update_layout(title="Feature importance (permutation)", xaxis_title="Drop in ROC-AUC")
461         plot(fig, 420)
462
463
464
465     def page_experts():
466         st.title("🔍 Expert finder")
467         finder, q, vectors, mode, tfidf = finder_and_vectors()
468         st.caption(f"Matching by {mode}. Experts are scored on similar past questions they answered, "
469                 "how much they help overall and how recently they were active.")
470         text = st.text_area("Describe your problem", "My hash table in speller leaks memory according to
valgrind", height=100)
471         if st.button("Find experts", type="primary") and text.strip():
472             vec = embed_query(text, tfidf)
473             ranking = finder.score(vec, method="hybrid").head(10)
474             m = out("user_metrics.csv").set_index("user_id")
475             names = topic_labels()
476             experts = pd.DataFrame({"score": ranking.round(3)})
477             experts = experts.join(m[["user_name", "role", "n_answers", "n_accepted",
"main_help_topic"]], how="left")
478             experts["main_help_topic"] = experts["main_help_topic"].map(names)
479             left, right = st.columns([3, 2])
480             with left:
481                 st.subheader("Recommended experts")
482                 st.dataframe(experts.reset_index(drop=True), hide_index=True, width="stretch")
483             with right:
484                 st.subheader("Similar past questions")
485                 rows, sims = finder.similar(vec, n=8)
486                 for r, s in zip(rows, sims):
487                     st.markdown(f"- [{q.at[r, 'title']}]({https://cs50.stackexchange.com/q/{q.at[r,
'post_id']}) · {s:.2f}")
488
489             ev = out("recommender_eval.csv")
490             if ev is not None:
491                 st.subheader("How good are the recommendations? (future questions)")
492                 long = ev.melt(id_vars="method", value_vars=["hit@1", "hit@5", "hit@10"], var_name="metric",
value_name="value")
493                 fig = px.bar(long, x="method", y="value", color="metric", barmode="group",
color_discrete_sequence=SERIES[:3])
494                 fig.update_layout(yaxis_tickformat=".0%", xaxis_title="", yaxis_title="Real answerer in top
k")
495                 plot(fig, 400)
496                 st.dataframe(ev, hide_index=True, width="stretch")
497
498
499
500     def sql_queries():
501         text = (ROOT / "sql" / "queries.sql").read_text(encoding="utf-8")
502         blocks = re.split(r"\n(?:--- \d+\.\. )", text)
503         out_ = {}

```

```

504     for b in blocks:
505         m = re.match(r"-- (\d+\. .+)\n", b)
506         if m:
507             out_[m.group(1)] = b[m.end():].split(";")[0].strip()
508     return out_
509
510
511 def page_database():
512     st.title("PostgreSQL database")
513     engine = get_db_engine()
514     if engine is None:
515         st.info("No PostgreSQL connection. The dashboard is using the CSV files.\n\n"
516                "To connect: create the `learning_community` database in pgAdmin, fill in `.env`,
then run "
517                "`python src/load_to_postgres.py` and restart the dashboard.")
518     return
519     from sqlalchemy import inspect, text
520     st.success("Connected to PostgreSQL")
521     tables = inspect(engine).get_table_names()
522     with engine.connect() as conn:
523         counts = {t: conn.execute(text(f'SELECT COUNT(*) FROM "{t}"')).scalar() for t in
sorted(tables)}
524     st.dataframe(pd.DataFrame({"table": counts.keys(), "rows": counts.values()}), hide_index=True)
525
526     queries = sql_queries()
527     name = st.selectbox("Ready-made analysis query", list(queries))
528     sql = st.text_area("SQL (read-only)", queries[name], height=220)
529     if st.button("Run query", type="primary"):
530         if not re.match(r"^\s*(select|with)\b", sql, re.I) or ";" in sql.strip().rstrip(";"):
531             st.error("Only a single SELECT query is allowed here.")
532         else:
533             with engine.connect() as conn:
534                 conn.execute(text("SET TRANSACTION READ ONLY"))
535                 st.dataframe(pd.read_sql(text(sql), conn), hide_index=True, width="stretch")
536
537
538 PAGES = {
539     "Overview": page_overview,
540     "Topics (NLP)": page_topics,
541     "Knowledge network": page_network,
542     "Knowledge diffusion": page_diffusion,
543     "Answer prediction": page_prediction,
544     "Expert finder": page_experts,
545     "Database (SQL)": page_database,
546 }
547
548 with st.sidebar:
549     st.header("Learning Community Analytics")
550     # ?page=<name> in the URL opens that page directly (handy for links and screenshots).
551     requested = st.query_params.get("page", "")
552     names = list(PAGES)
553     start = next((i for i, p in enumerate(names) if p.lower().startswith(requested.lower())), 0) if
requested else 0
554     page = st.radio("Page", names, index=start, label_visibility="collapsed")
555     st.divider()
556     st.caption(f"Data source: **{data_source()}**")
557     st.caption("CS50 Stack Exchange · 2019–2026")
558
559 PAGES[page]()

```

Output: dashboard screenshots

The screenshots below were taken from the running dashboard with the CSV data source. On the laptop, after connecting PostgreSQL, the sidebar shows **Data source: PostgreSQL** and the Database page lists the tables.

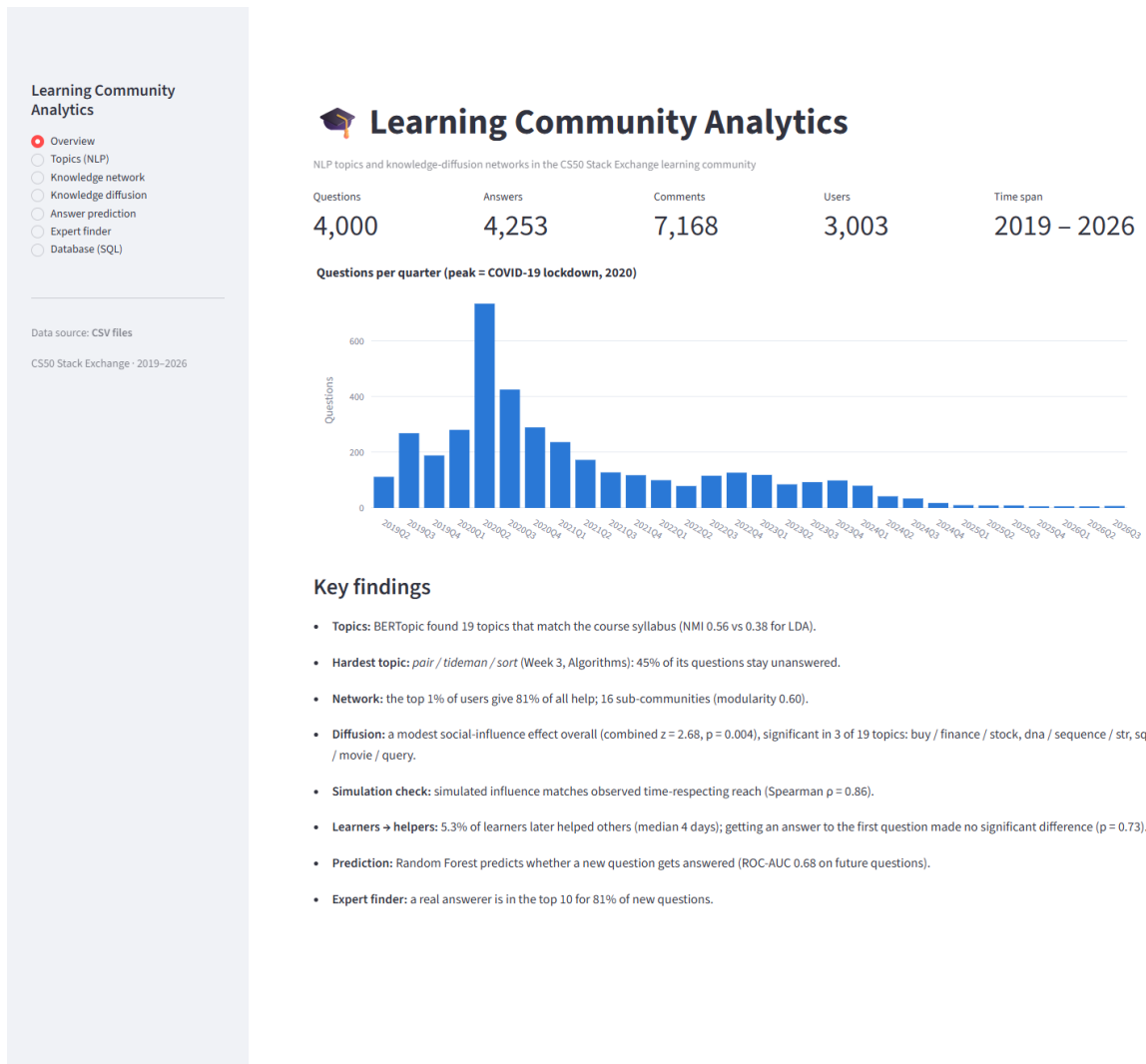


Figure 17. Dashboard: Overview page

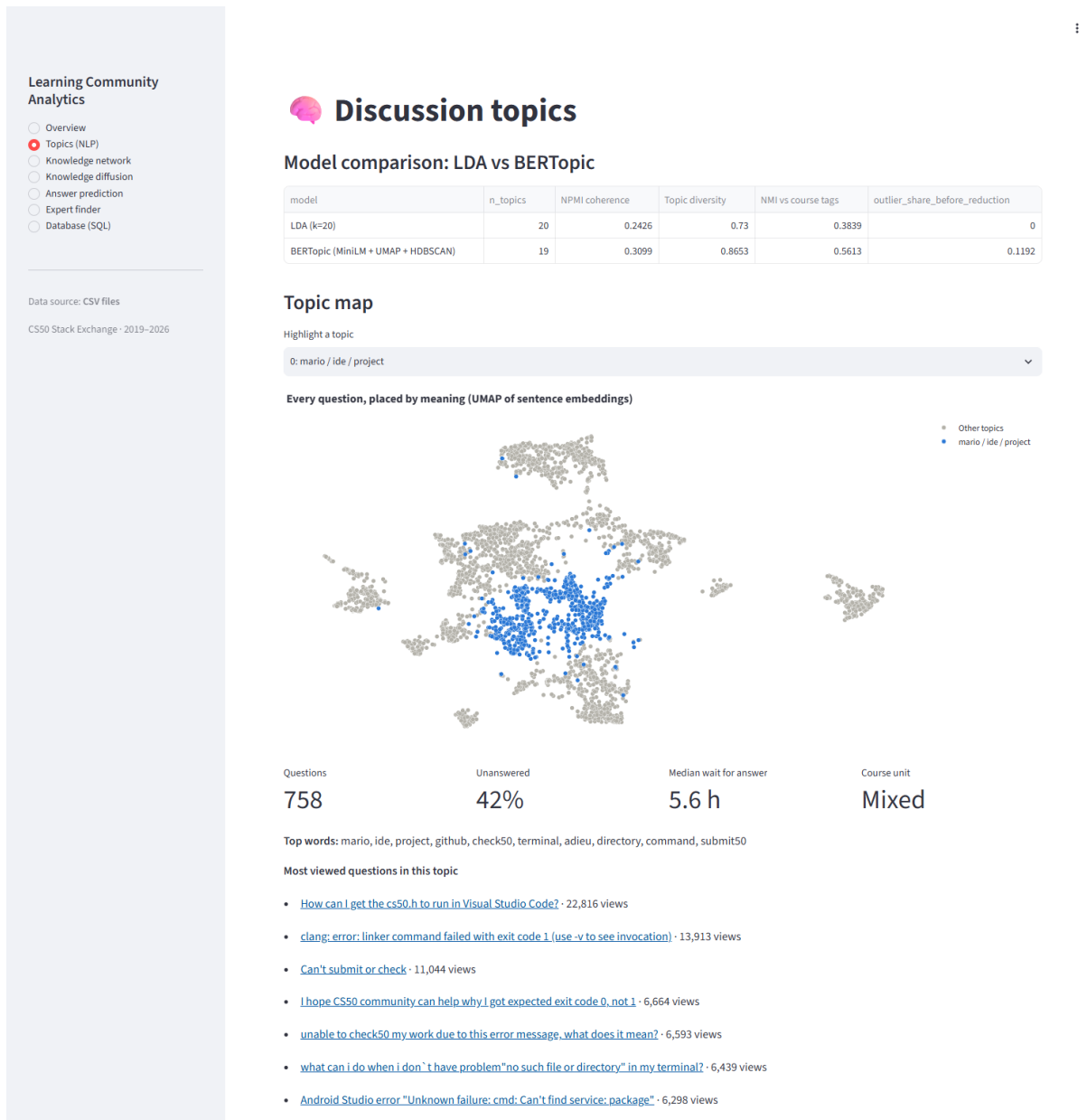
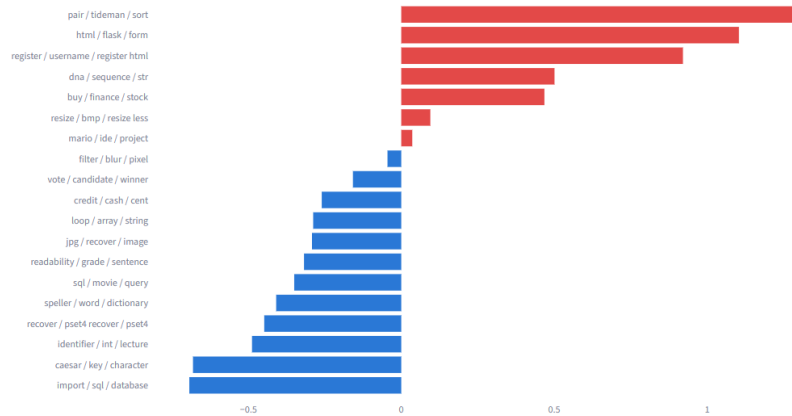


Figure 18. Dashboard: Topics page (topic map, difficulty, trends) (part 1 of 2)

- [How can I submit project0 using github?](#) · 6,281 views

Topic difficulty

Difficulty index (unanswered rate + wait time + confusion), higher = harder



How topics changed over time

Only years with at least 100 questions are shown.

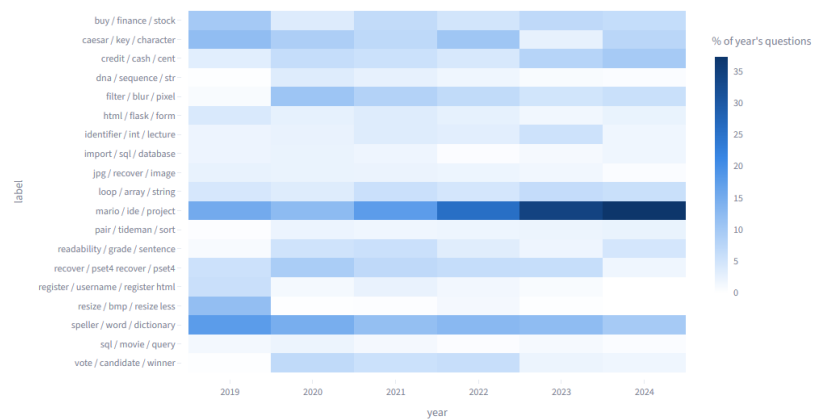


Figure 19. Dashboard: Topics page (topic map, difficulty, trends) (part 2 of 2)

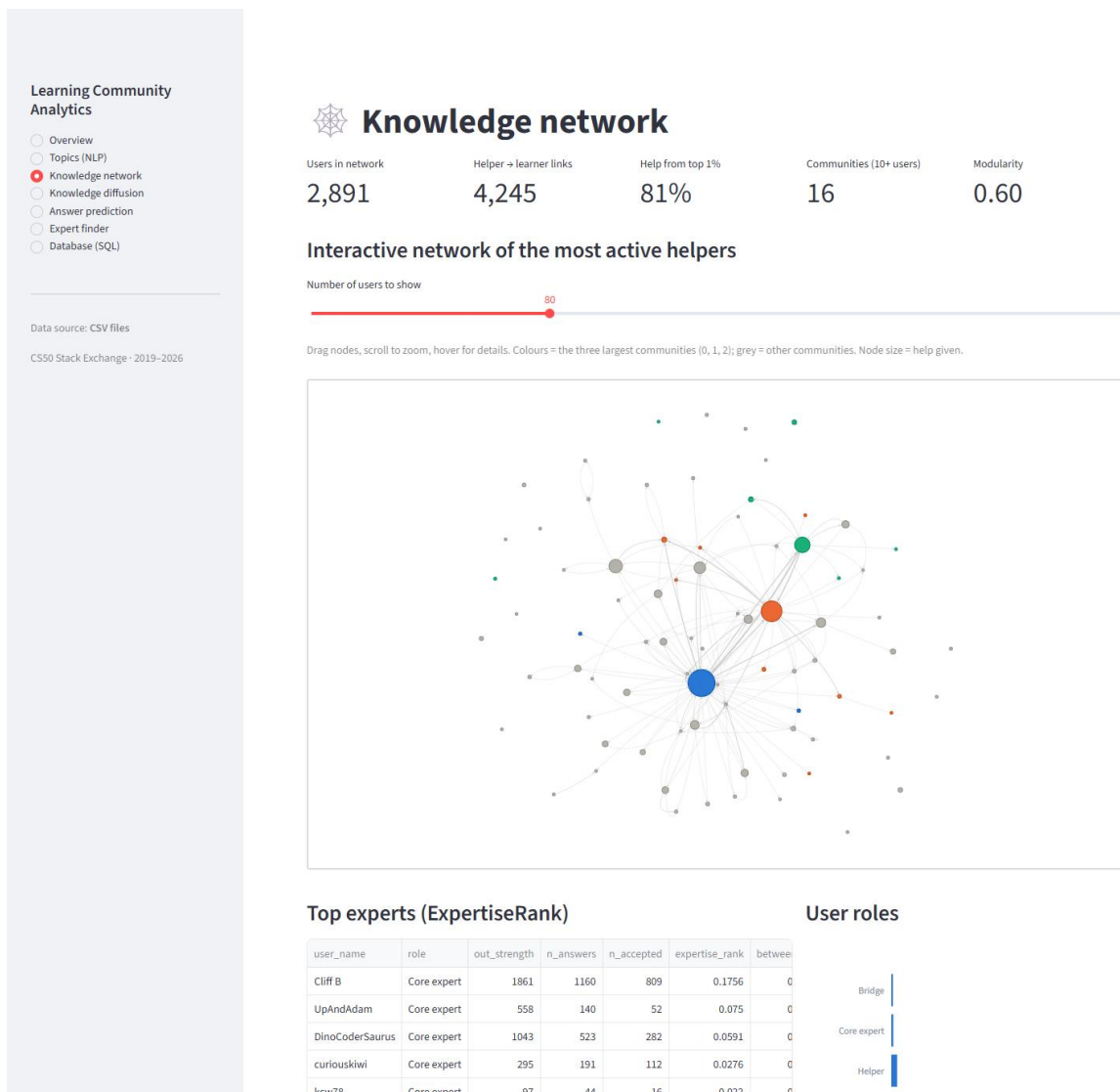


Figure 20. Dashboard: Knowledge network page (interactive graph) (part 1 of 2)

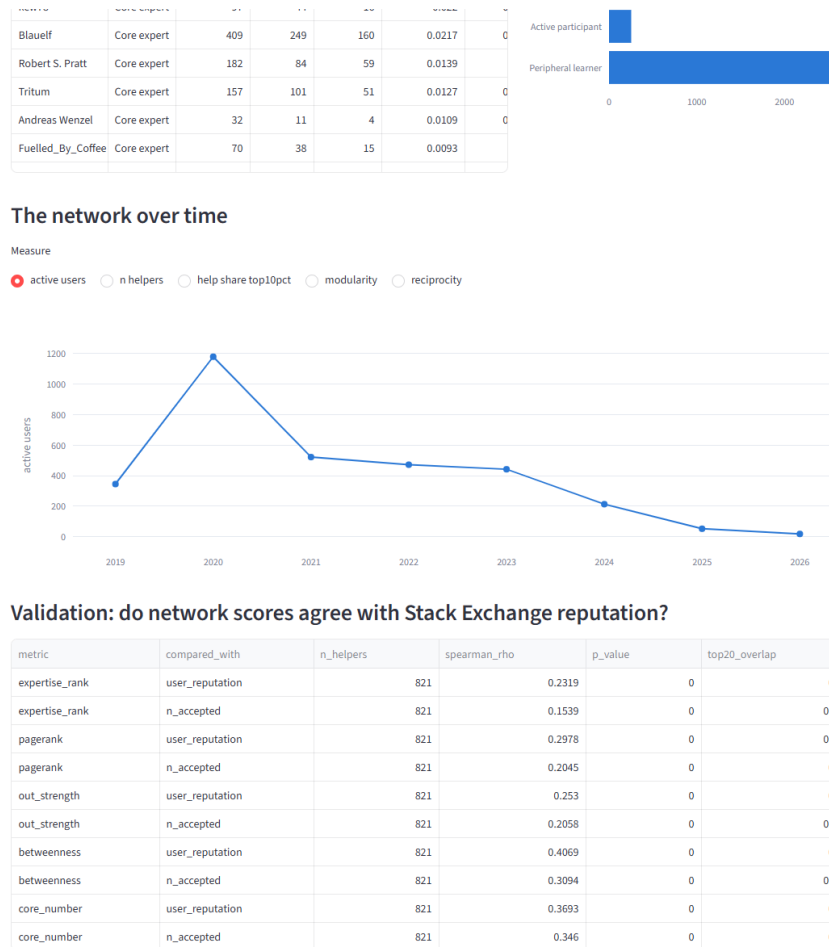


Figure 21. Dashboard: Knowledge network page (interactive graph) (part 2 of 2)

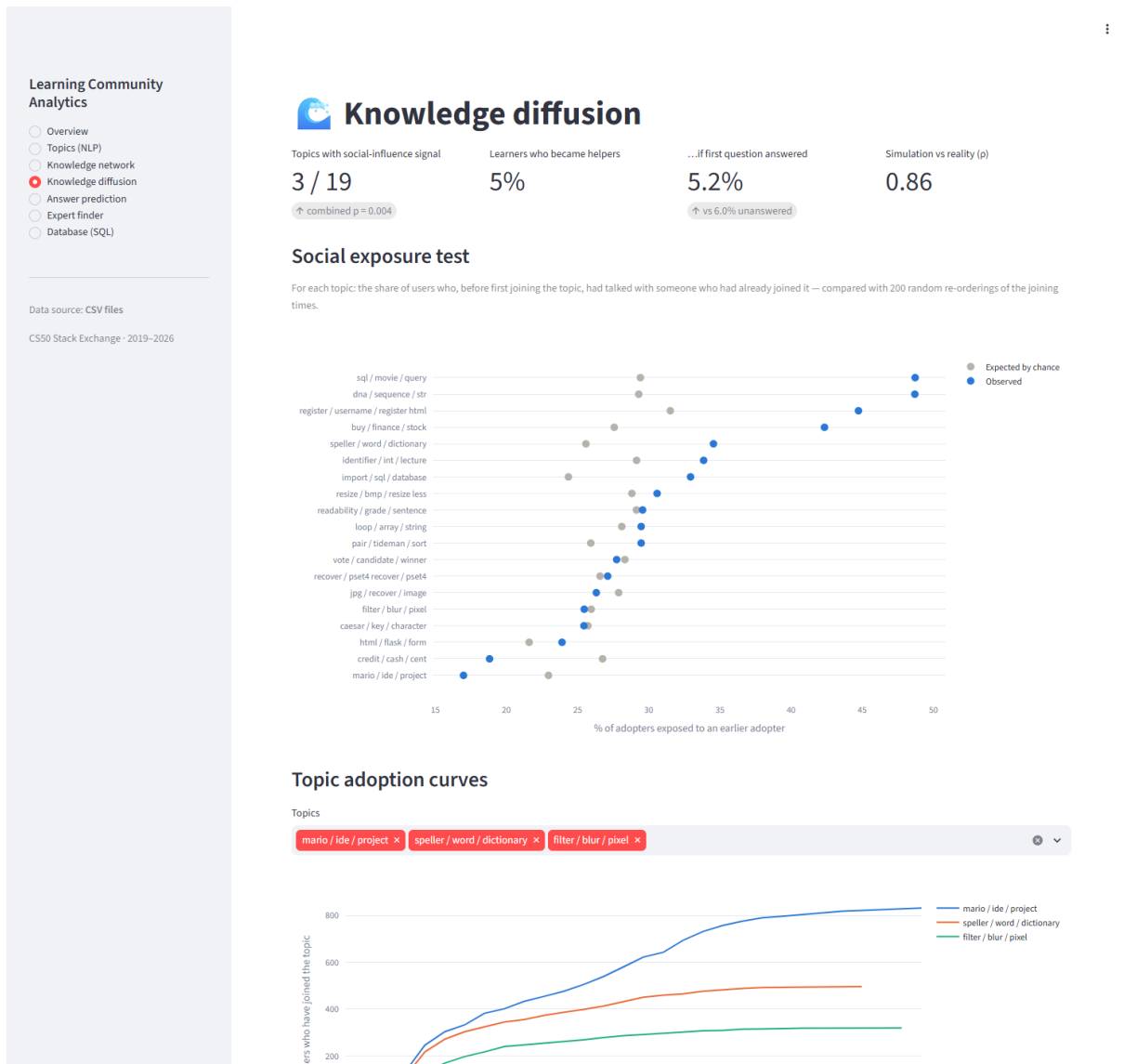


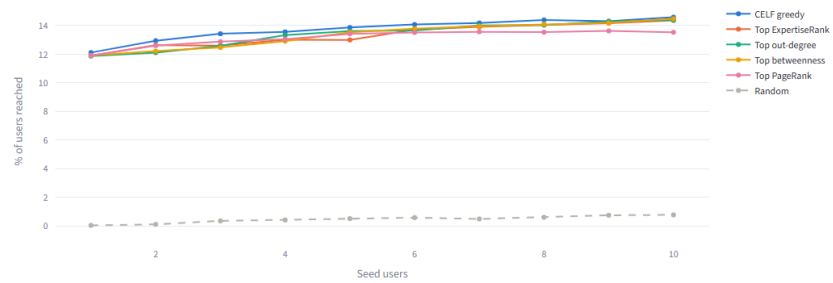
Figure 22. Dashboard: Knowledge diffusion page (part 1 of 2)



Influence maximisation (simulation)

Diffusion model

☒ Independent Cascade ☐ Linear Threshold



Best seed users (CELF): UpAndAdam, Blaueif, curiouskiwi, Tritum, Fuelled_By_Coffee, Ojou Nii, kcw78, Community, Astha Jain, Sentox

Does the simulation match what really happened?

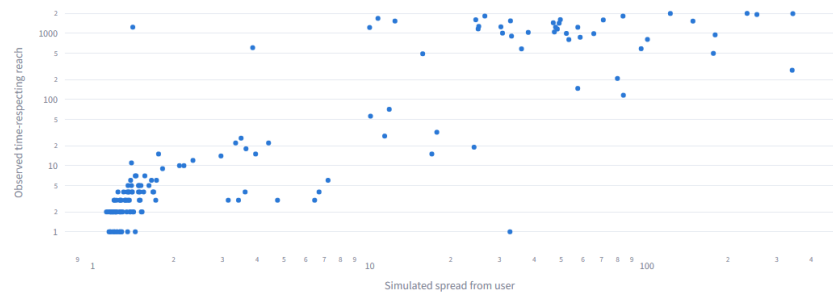
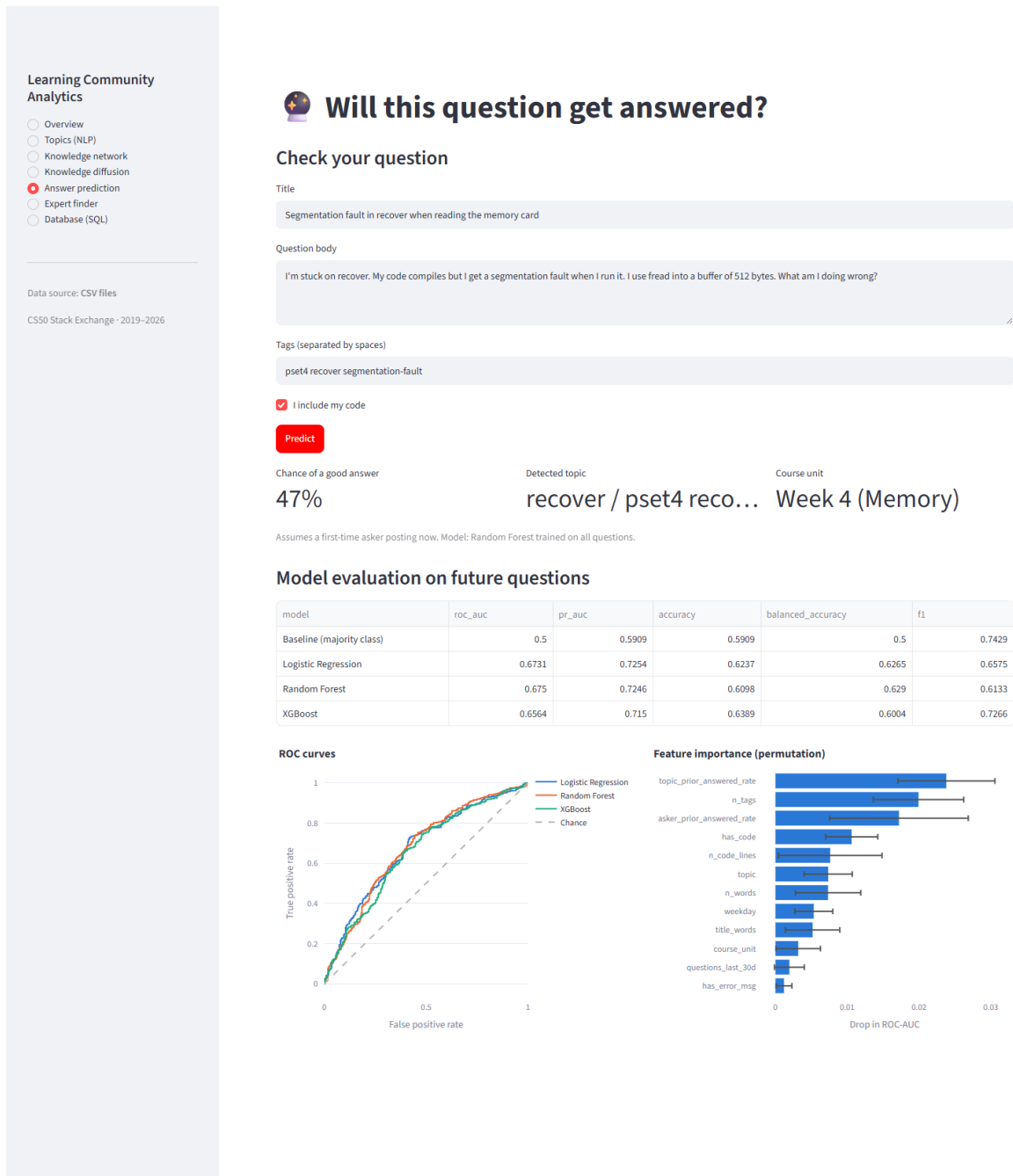


Figure 23. Dashboard: Knowledge diffusion page (part 2 of 2)



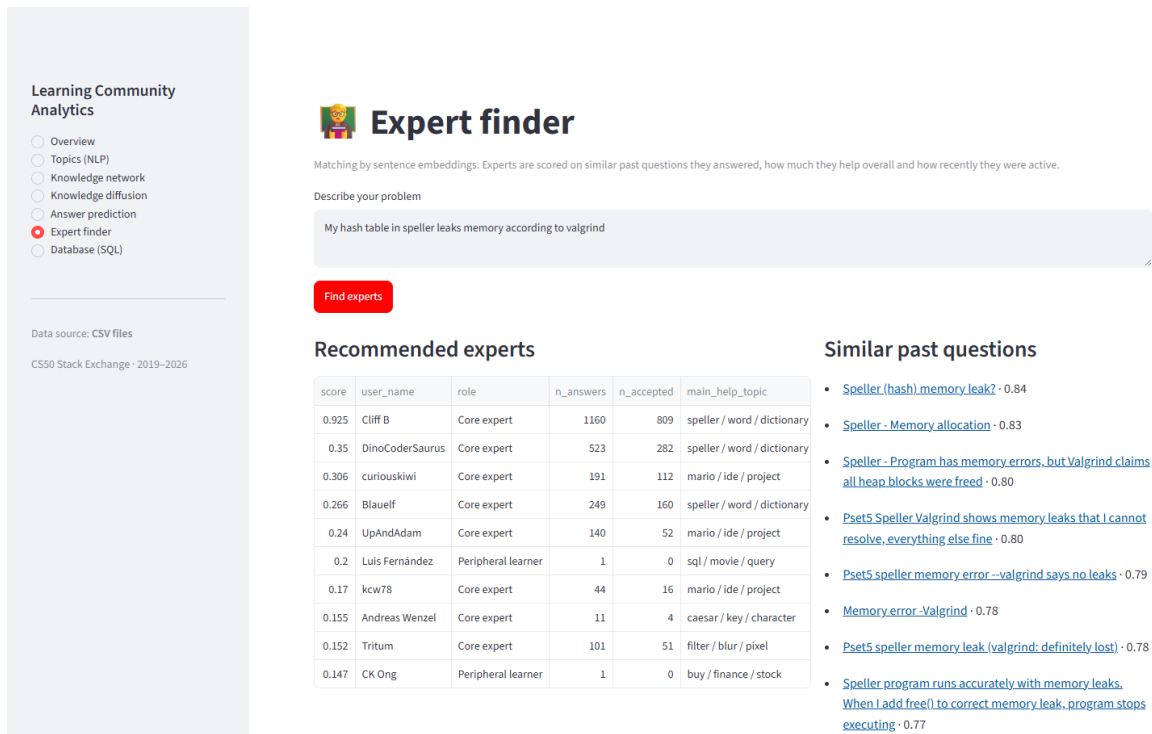


Figure 25. Dashboard: Expert finder, result for a sample problem (part 1 of 2)

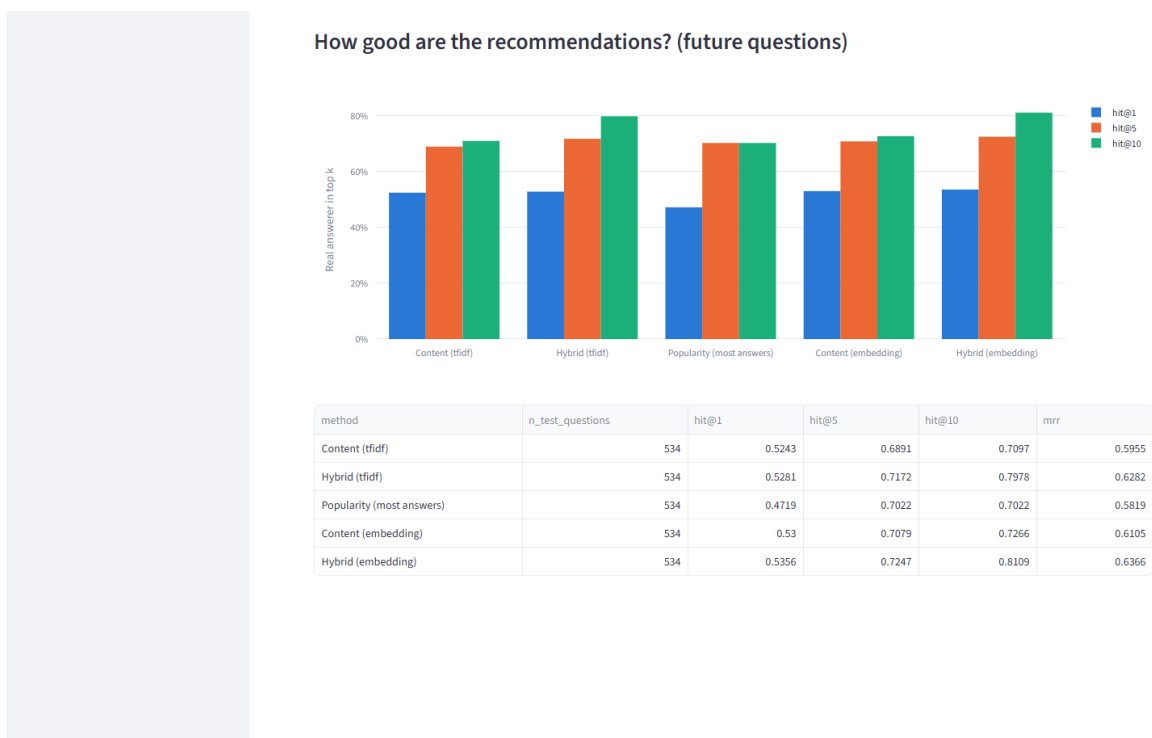


Figure 26. Dashboard: Expert finder, result for a sample problem (part 2 of 2)

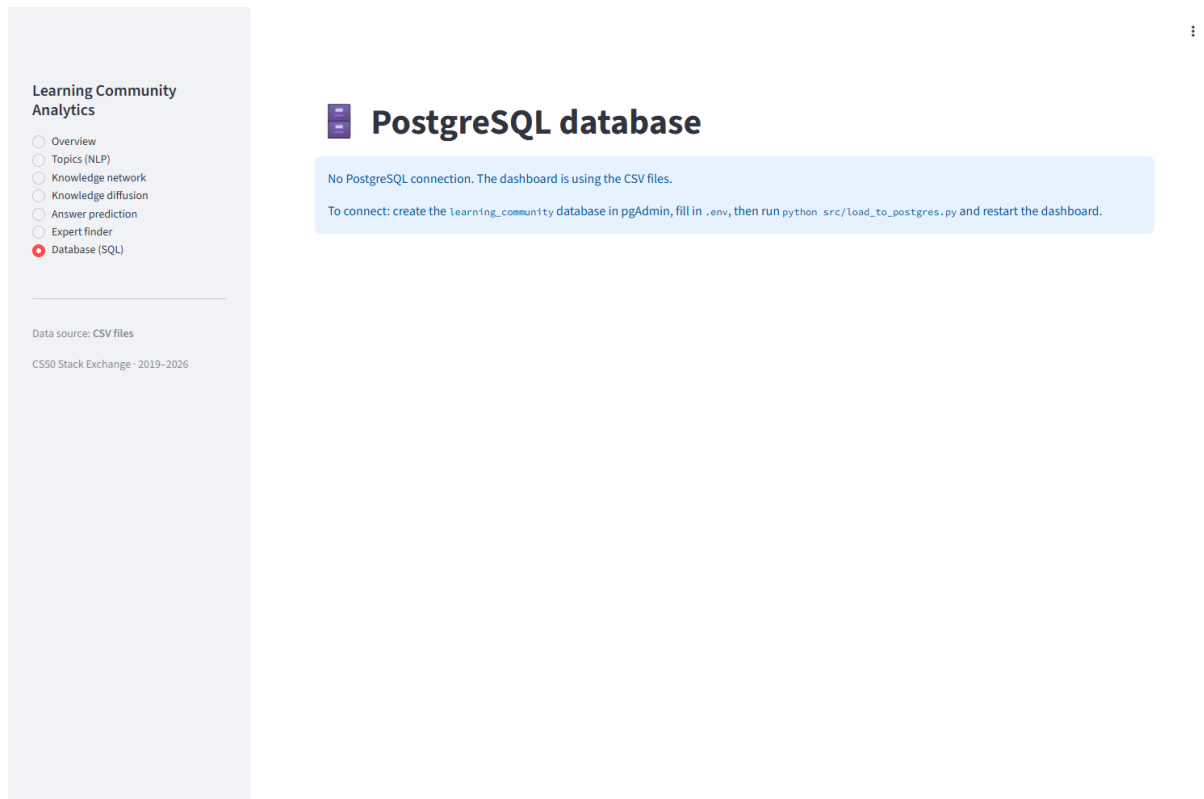


Figure 27. Dashboard: Database page without a PostgreSQL connection

